

From CLIP to Vision-Language Models

Following our CLIP lecture · ES 667 · Nipun Batra · IIT Gandhinagar

How Images Become Context for Language Generation

Last time: compare an image with descriptions.

Today: use the image to generate an answer.



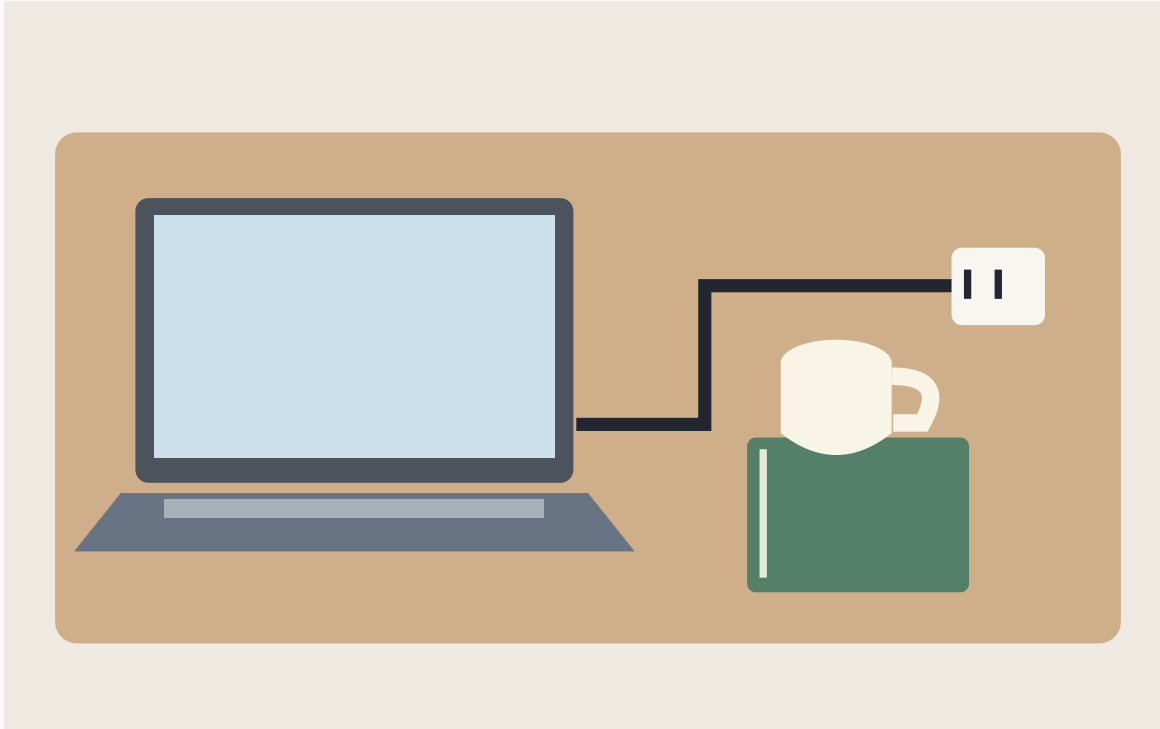
| How does visual information become context for next-token prediction?

SECTION 01

What can a VLM do?

Start with the evidence an answer needs.

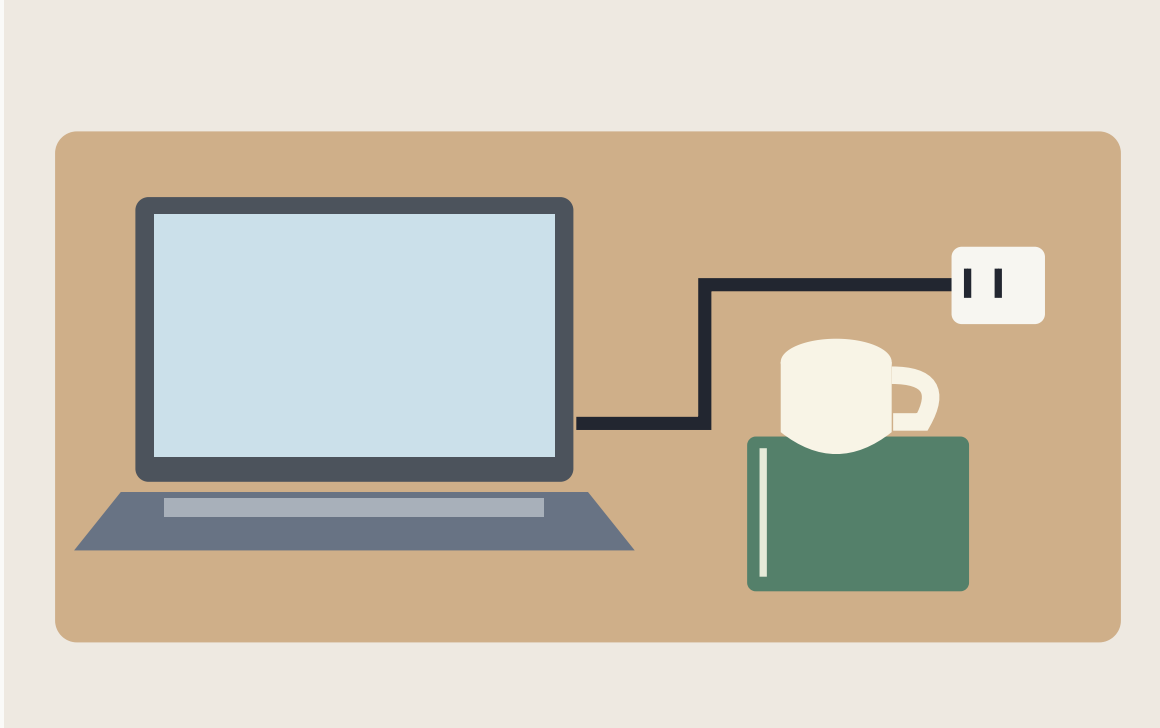
Can the answer use visible evidence?



QUESTION

Is the laptop plugged in?

Can the answer use visible evidence?

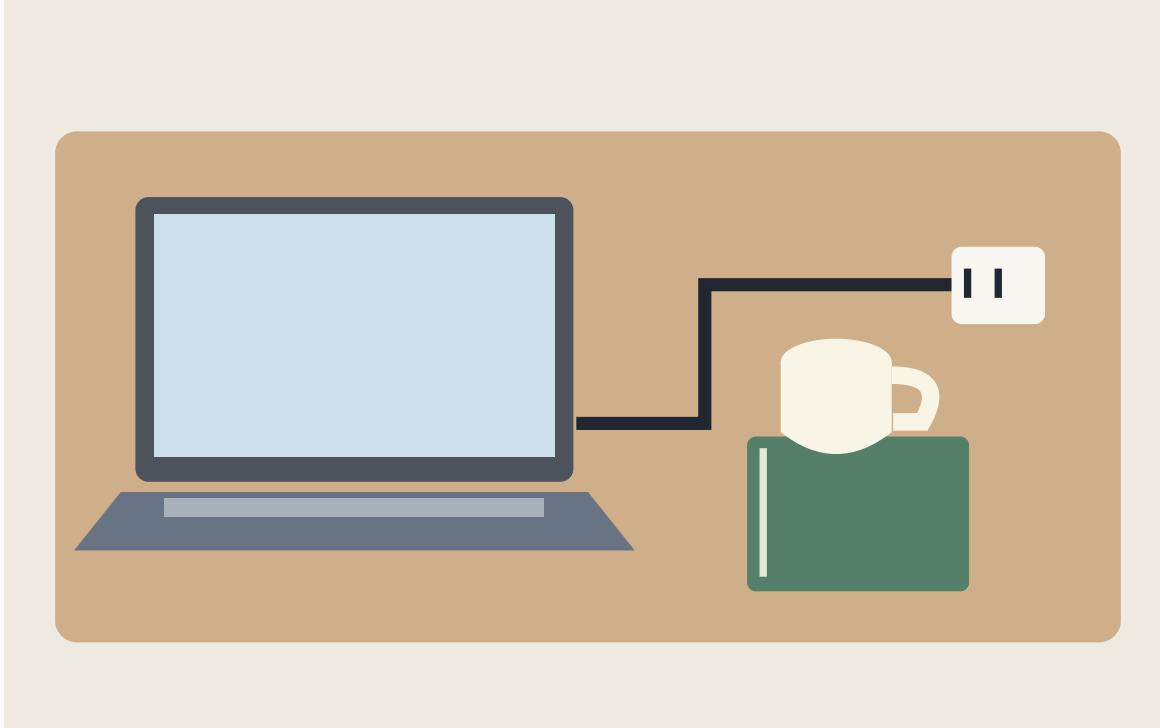


REFERENCE ANSWER

Is the laptop plugged in?

Yes. A cable connects the laptop to the socket.

Can the answer use visible evidence?



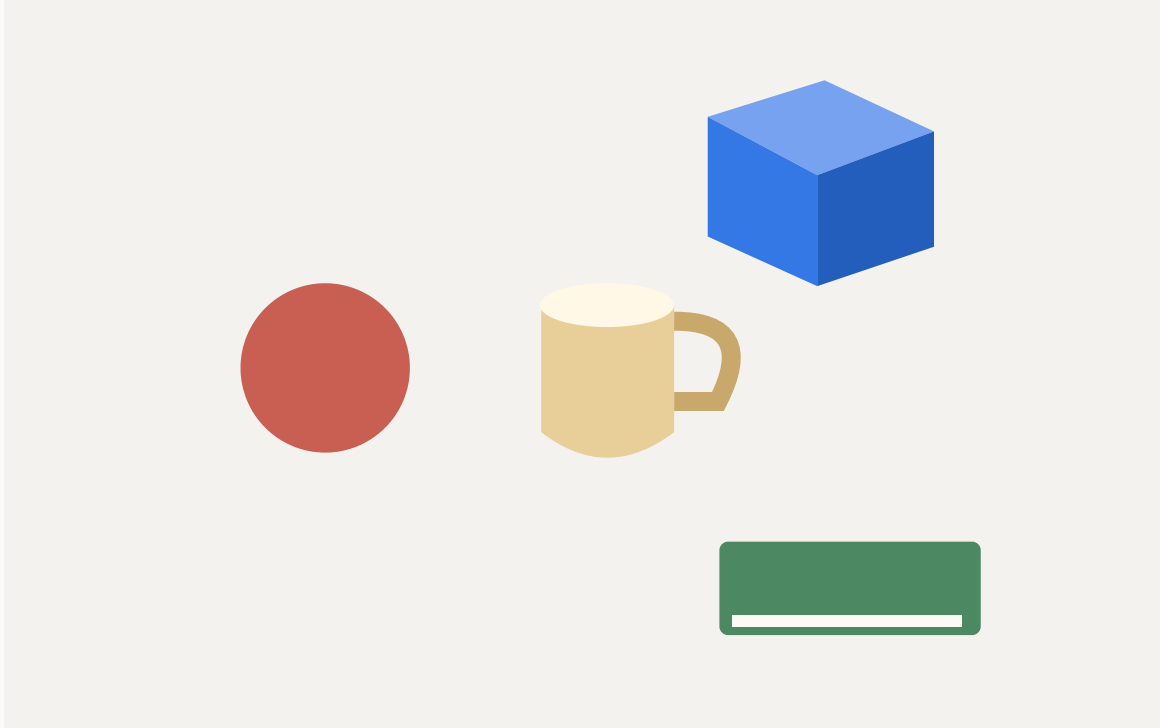
ARCHITECTURAL REQUIREMENT

Is the laptop plugged in?

Yes. A cable connects the laptop to the socket.

| Trace a connection between objects.

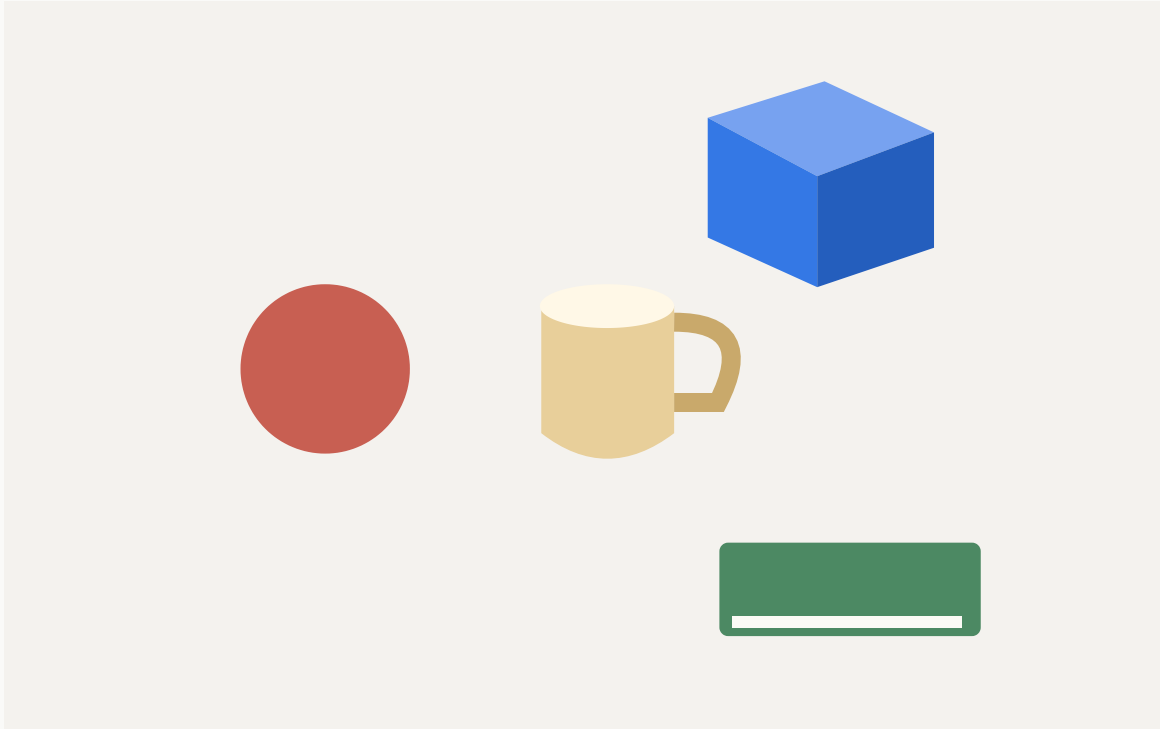
Identity is only part of the question



QUESTION

What is immediately left of the mug?

Identity is only part of the question

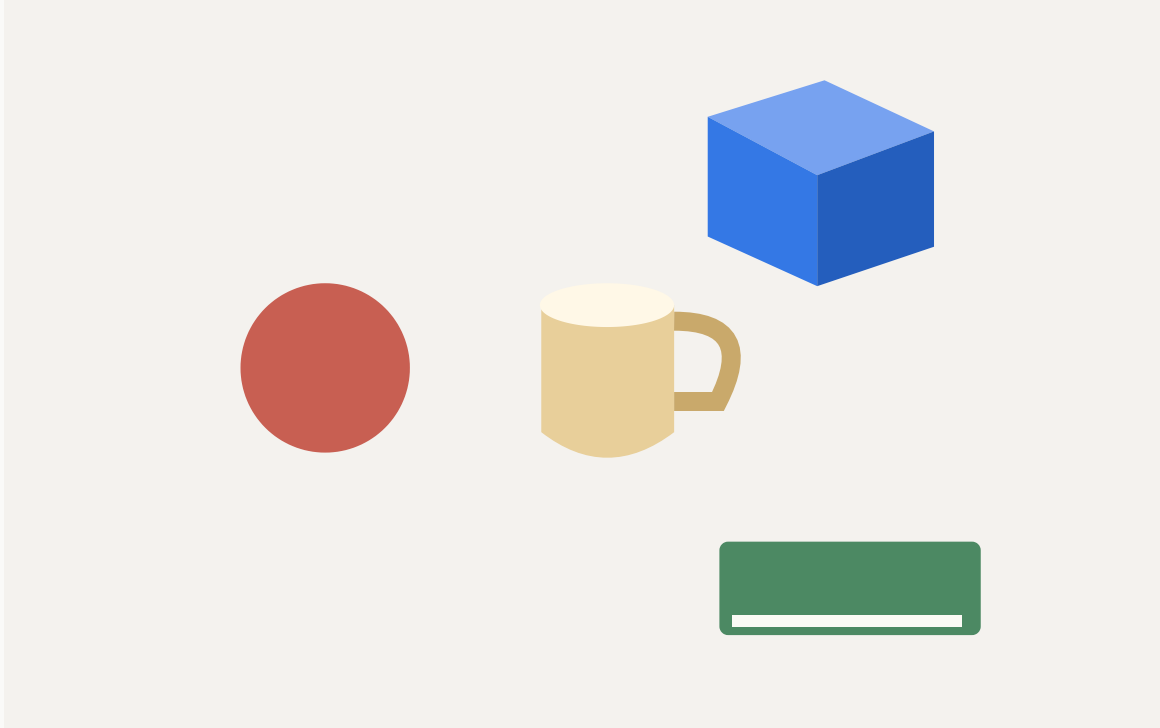


REFERENCE ANSWER

What is immediately left of the mug?

The red ball.

Identity is only part of the question



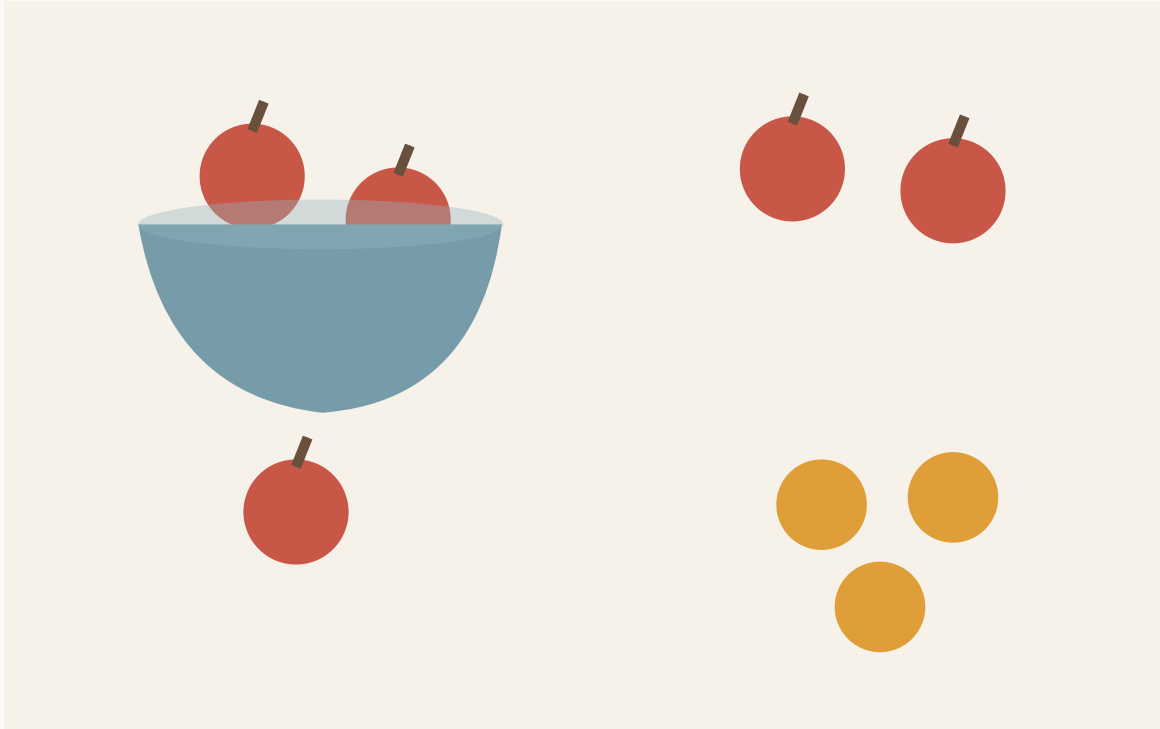
ARCHITECTURAL REQUIREMENT

What is immediately left of the mug?

The red ball.

Keep object identity and spatial relations.

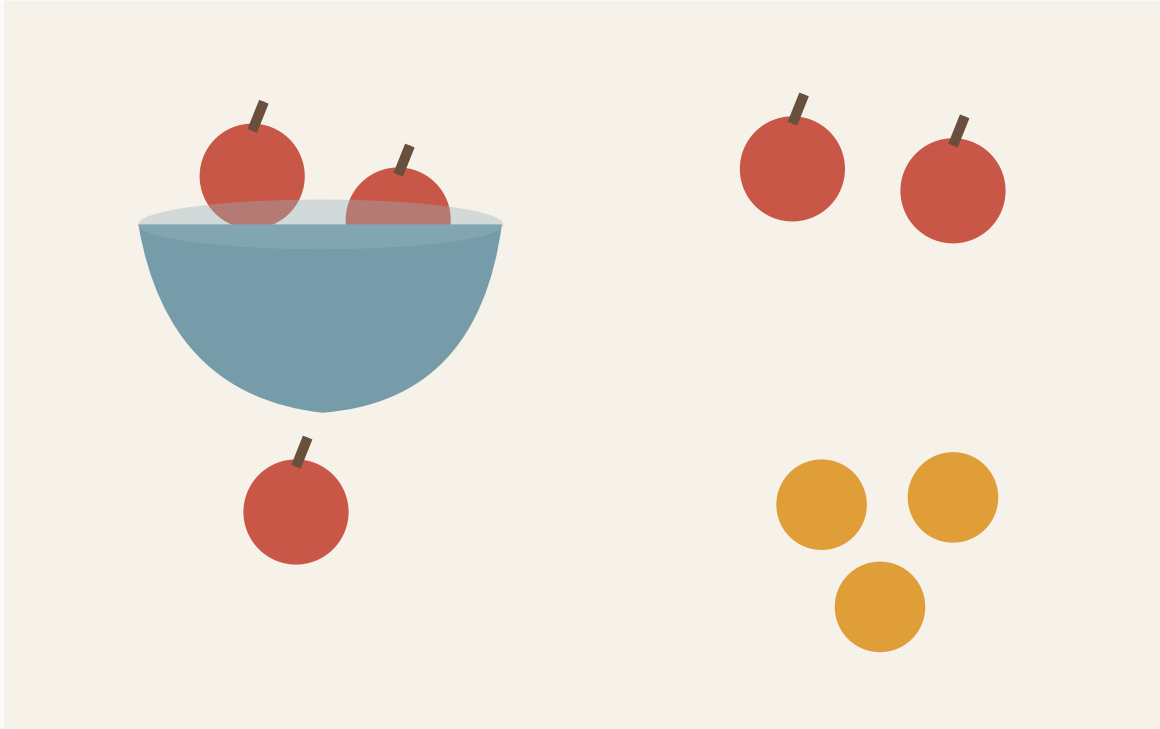
The question selects a subset



QUESTION

How many apples are outside the bowl?

The question selects a subset

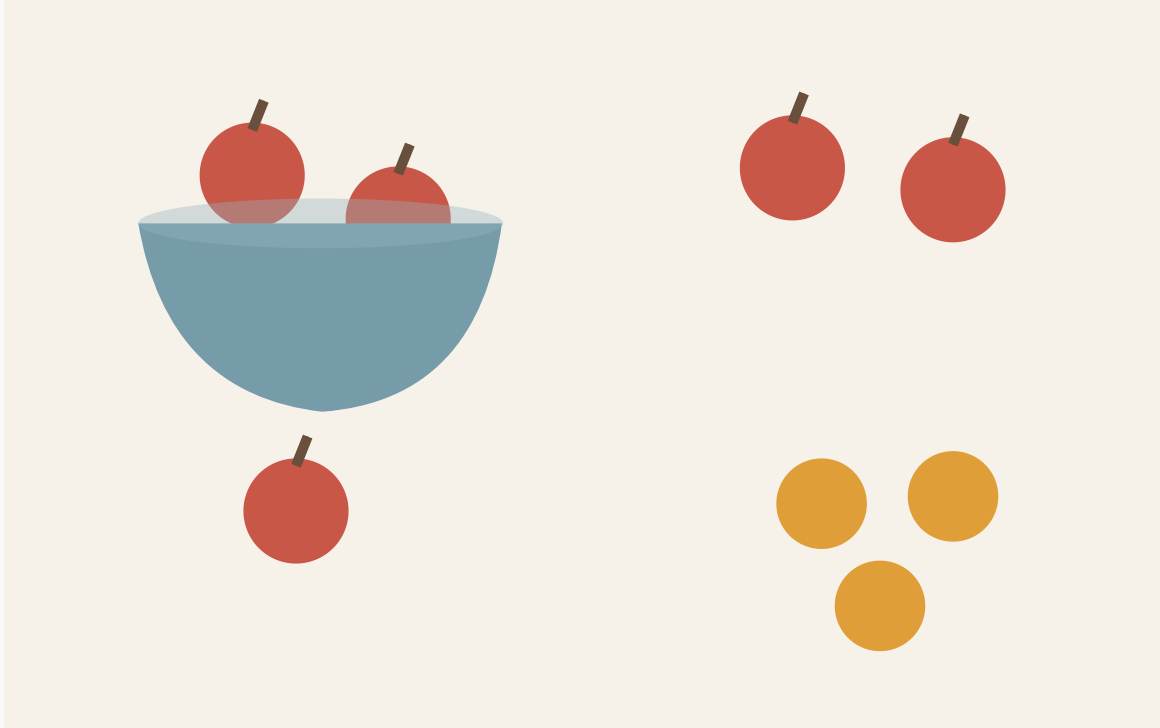


REFERENCE ANSWER

How many apples are outside the bowl?

Three.

The question selects a subset



ARCHITECTURAL REQUIREMENT

How many apples are outside the bowl?

Three.

Represent multiple objects and the region that selects them.

Read first, then calculate

CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

Subtotal ₹1,000

GST ₹180

TOTAL ₹1,180

QUESTION

What is the invoice ID?

What percentage of the subtotal is GST?

Read first, then calculate

CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

Subtotal ₹1,000

GST ₹180

TOTAL ₹1,180

REFERENCE ANSWER

What is the invoice ID?

What percentage of the subtotal is GST?

ES667-042. GST is $180 / 1000 \times 100 = 18\%$.

Read first, then calculate

CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

Subtotal ₹1,000

GST ₹180

TOTAL ₹1,180

ARCHITECTURAL REQUIREMENT

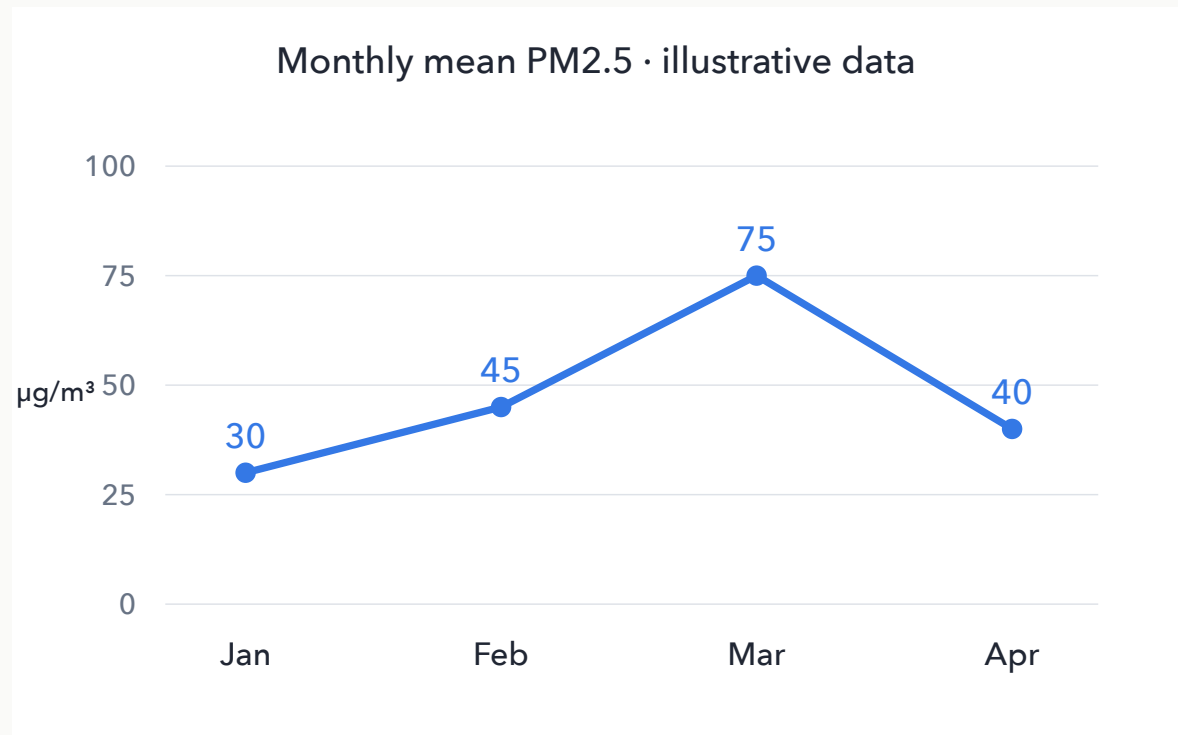
What is the invoice ID?

What percentage of the subtotal is GST?

ES667-042. GST is $180 / 1000 \times 100 = 18\%$.

Preserve fine text, numbers and their roles.

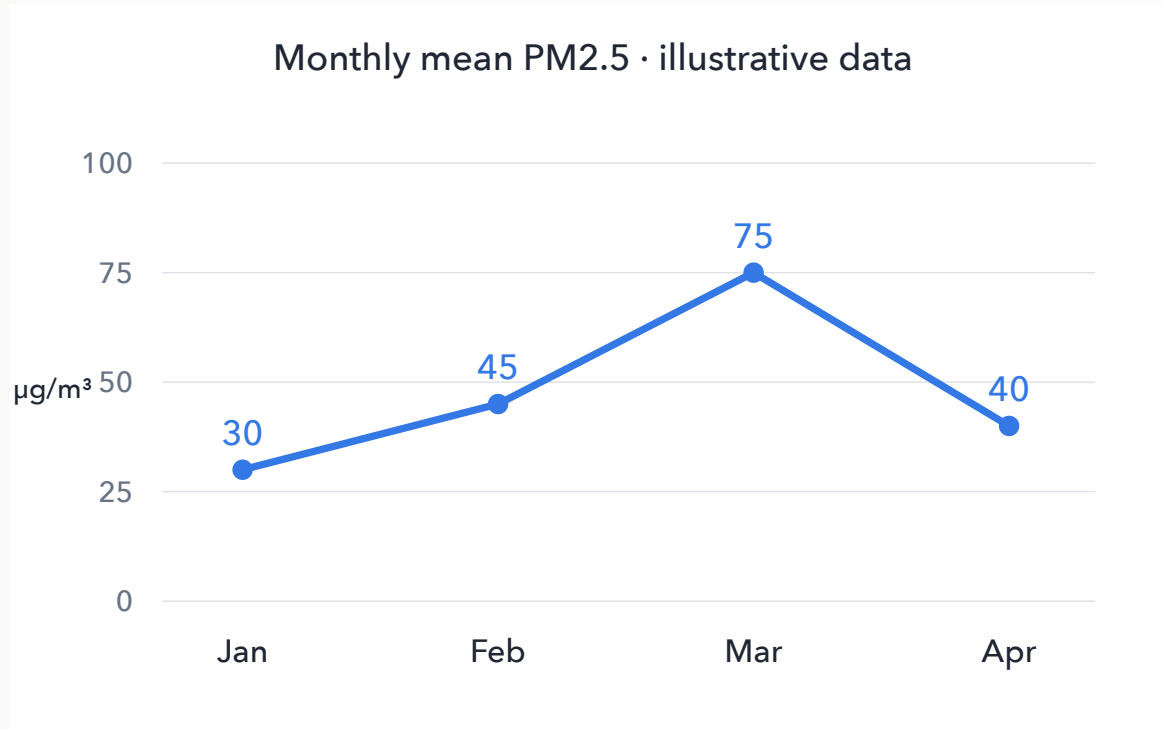
The labels change the answer



QUESTION

Which month is highest?
How much higher is March than January?

The labels change the answer

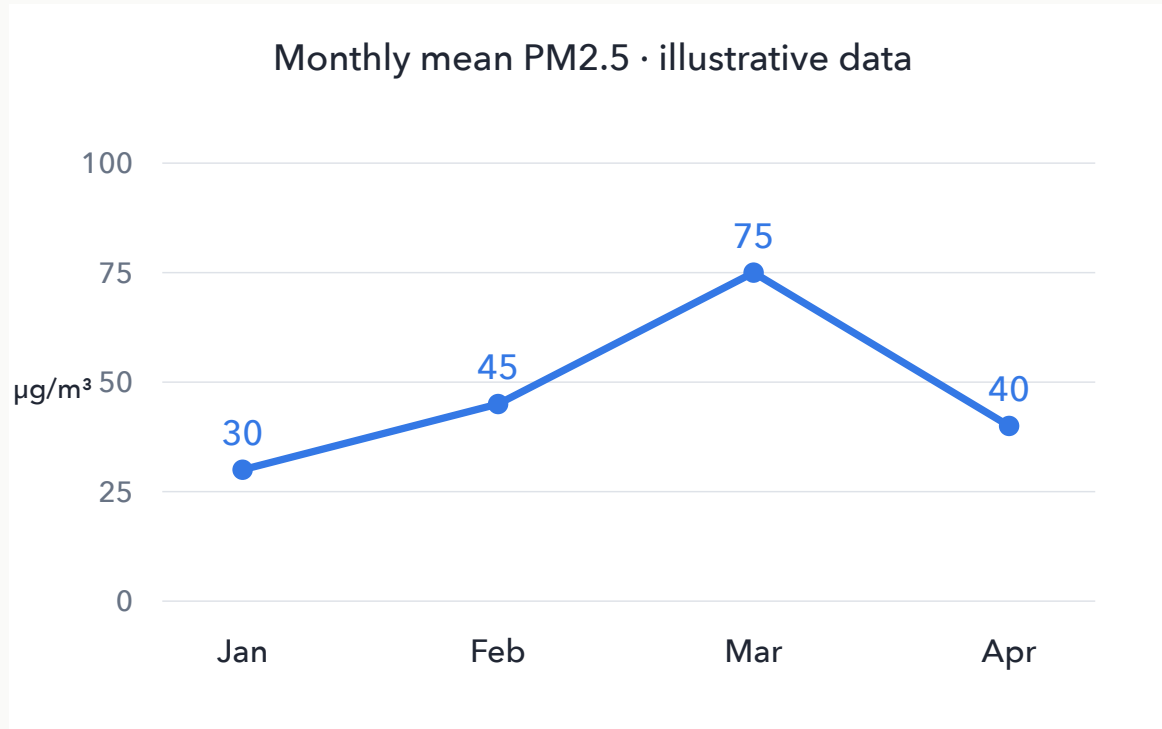


REFERENCE ANSWER

Which month is highest?
How much higher is March than January?

March. The difference is 45 $\mu\text{g}/\text{m}^3$.

The labels change the answer



ARCHITECTURAL REQUIREMENT

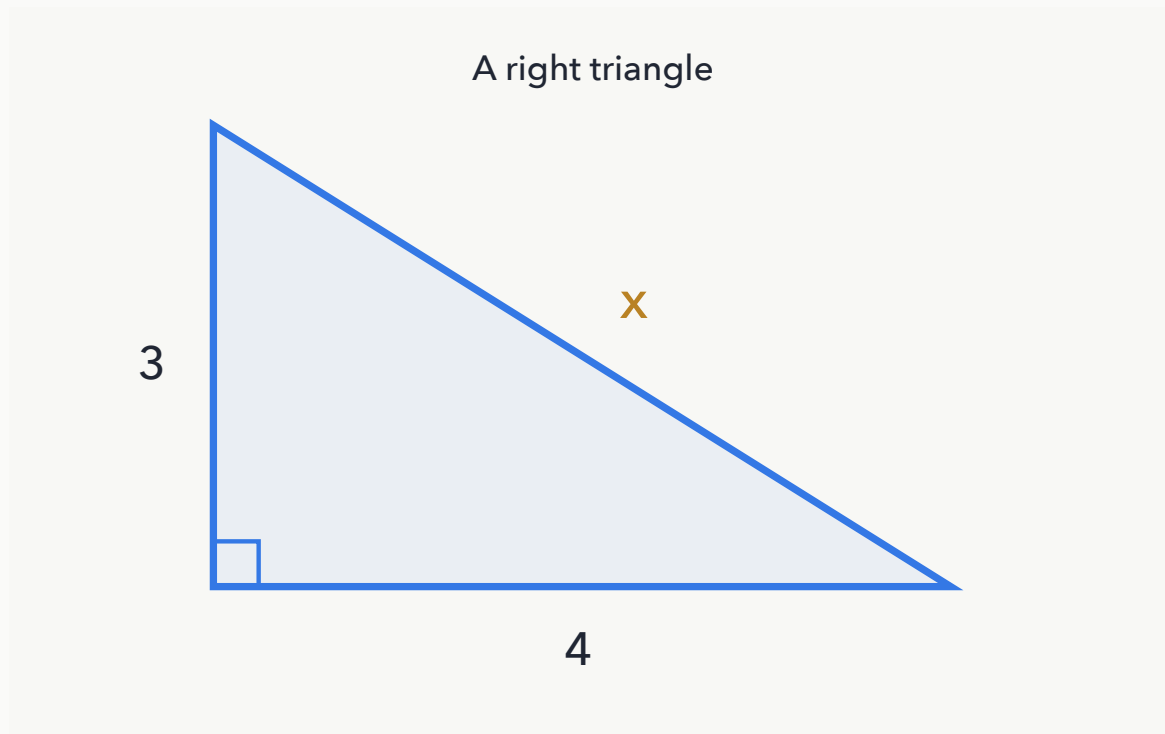
Which month is highest?

How much higher is March than January?

March. The difference is 45 μg/m³.

Bind chart labels to values and units.

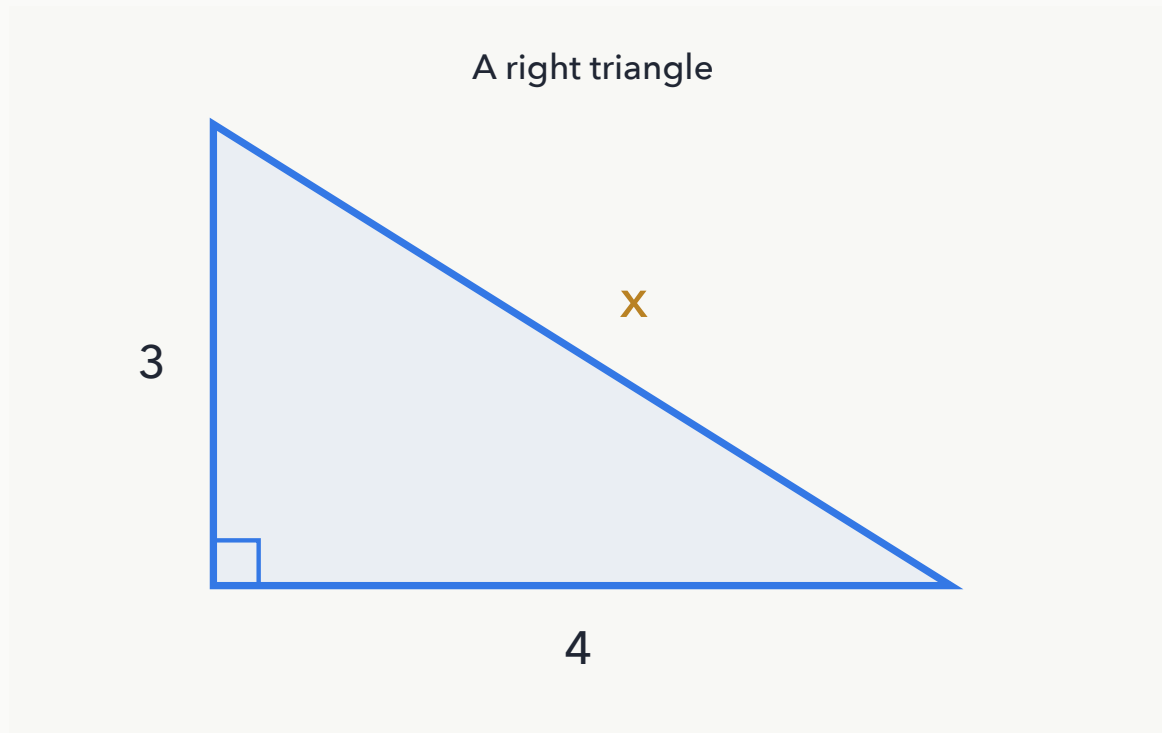
A diagram supplies conditions



QUESTION

What is x?

A diagram supplies conditions

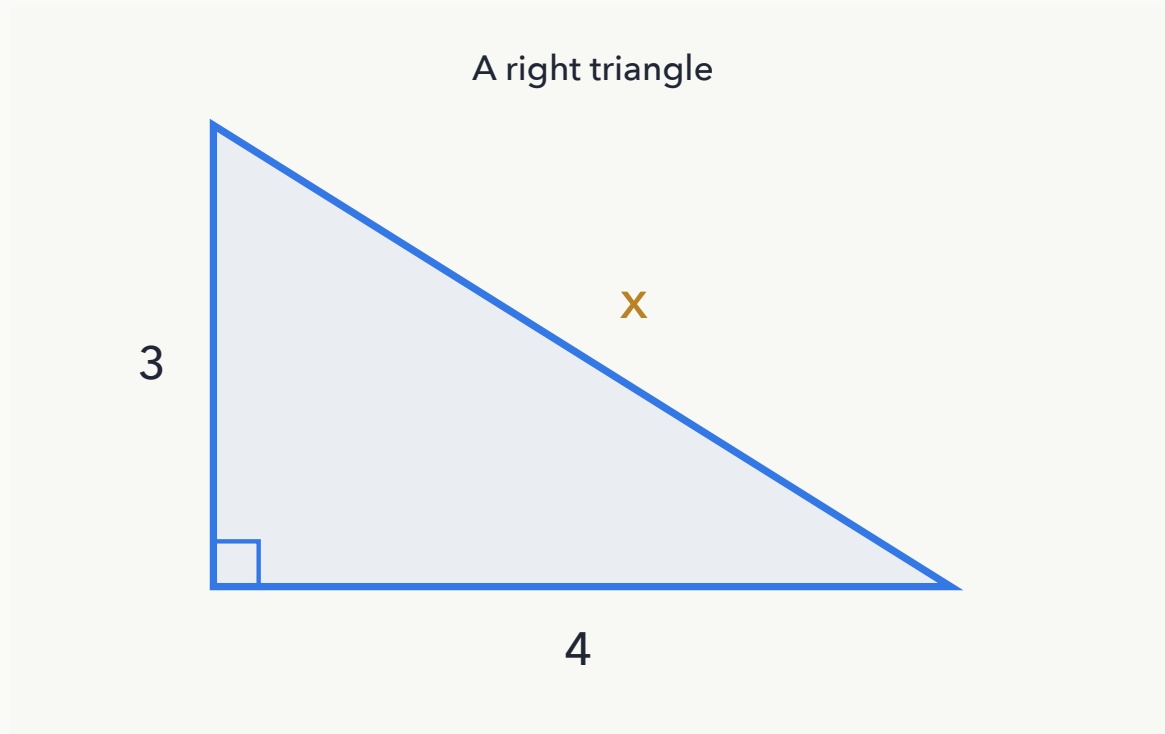


REFERENCE ANSWER

What is x?

$x = 5$, using the marked right angle and sides 3 and 4.

A diagram supplies conditions



ARCHITECTURAL REQUIREMENT

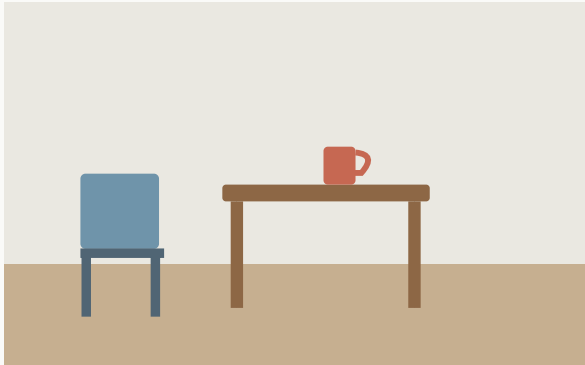
What is x?

$x = 5$, using the marked right angle and sides 3 and 4.

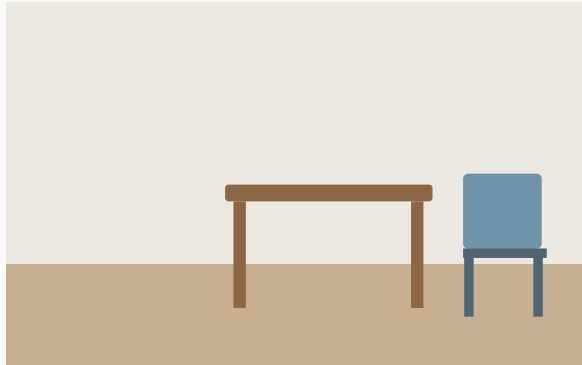
Combine diagram evidence with a mathematical rule.

Compare presence and position

A · BEFORE



B · AFTER



QUESTION

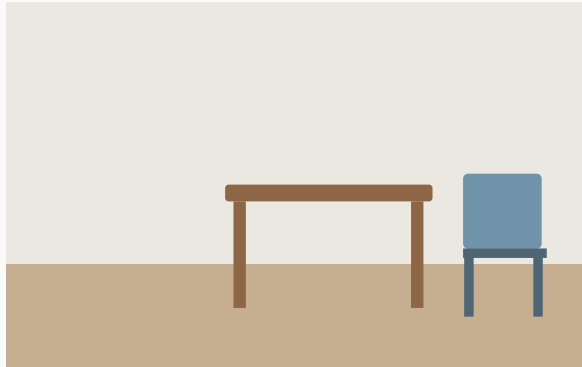
What changed from A to B?

Compare presence and position

A · BEFORE



B · AFTER



REFERENCE ANSWER

What changed from A to B?

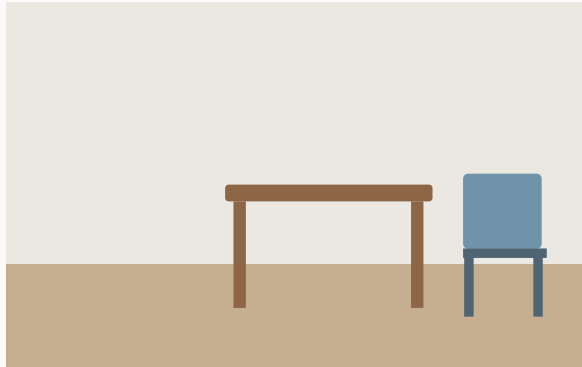
The chair moved to the right; the cup disappeared.

Compare presence and position

A · BEFORE



B · AFTER



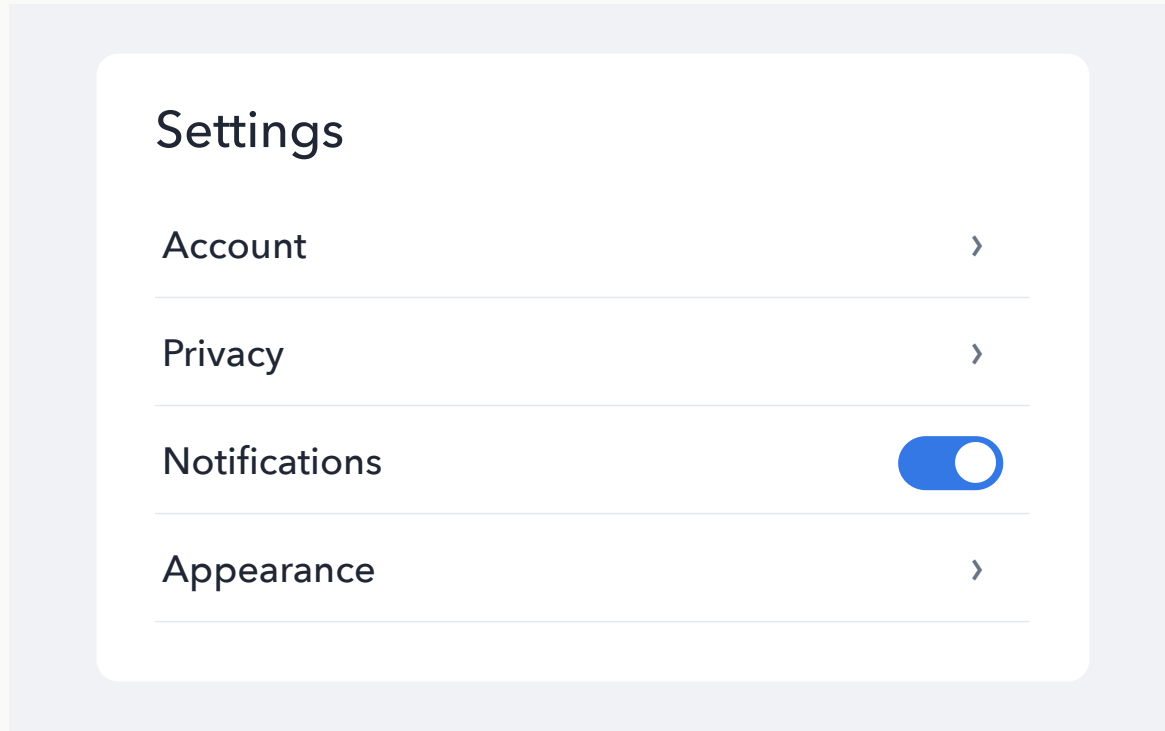
ARCHITECTURAL REQUIREMENT

What changed from A to B?

The chair moved to the right; the cup disappeared.

| Compare evidence across images.

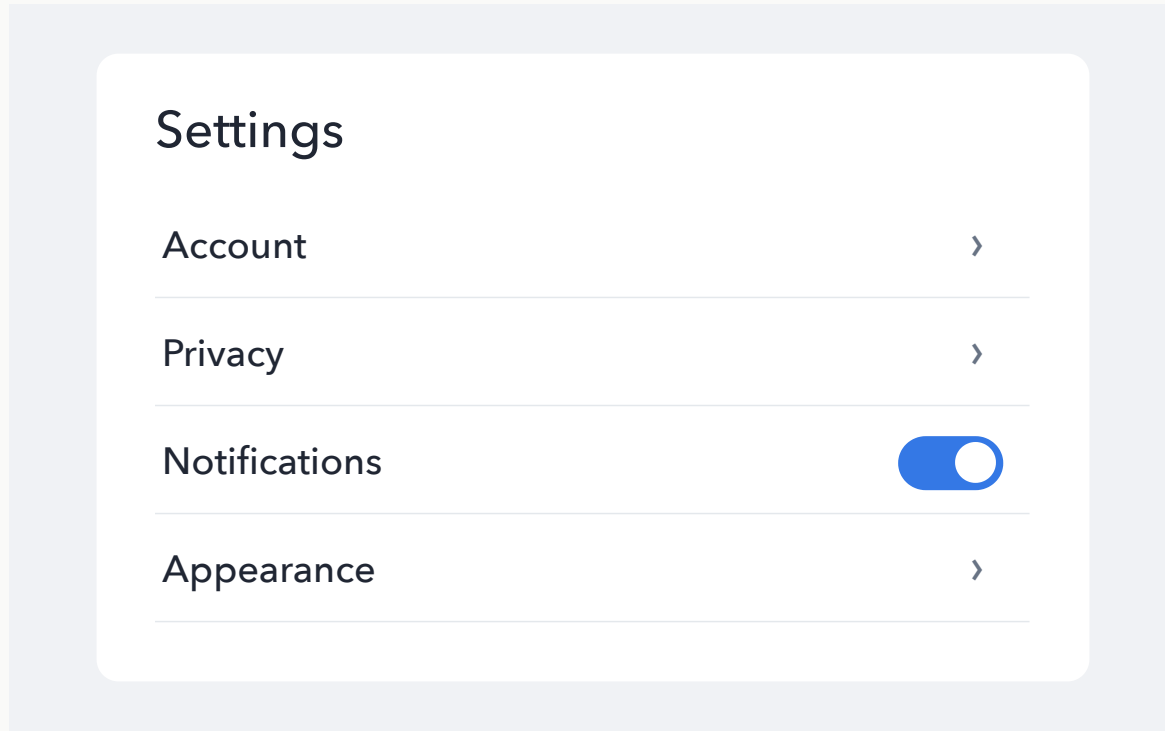
A screenshot is a spatial interface



QUESTION

Where should I click to disable notifications?

A screenshot is a spatial interface

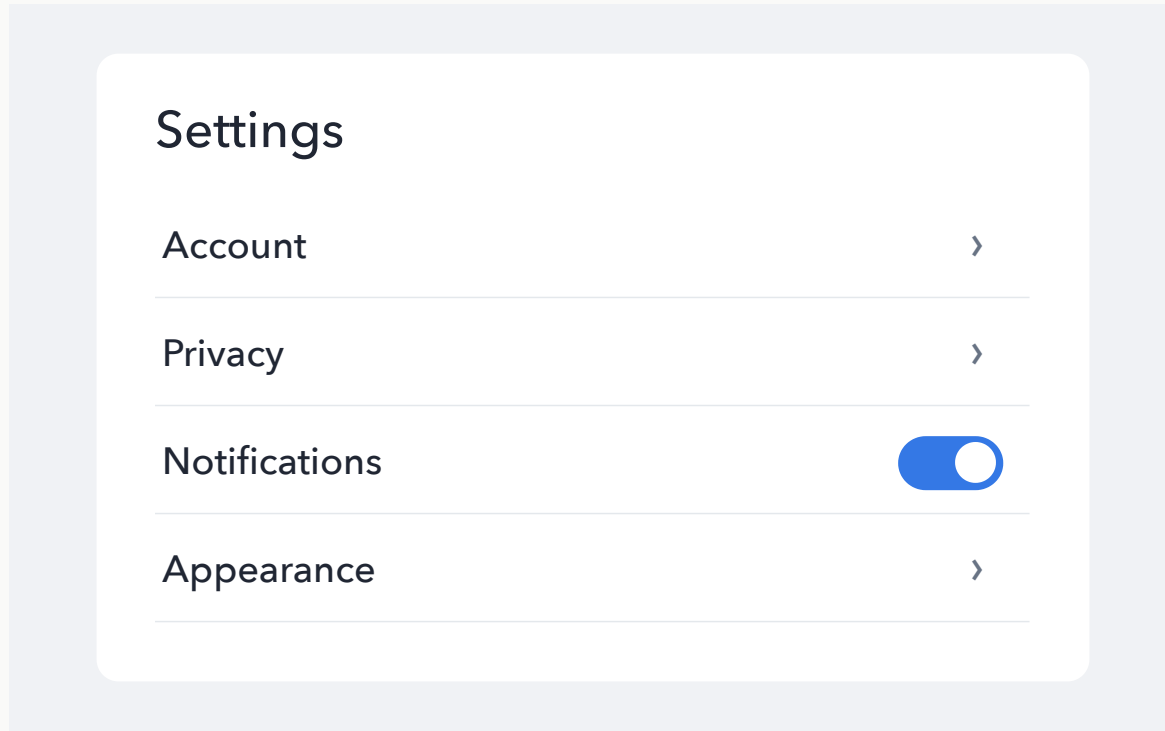


REFERENCE ANSWER

Where should I click to disable notifications?

The blue toggle on the Notifications row.

A screenshot is a spatial interface



ARCHITECTURAL REQUIREMENT

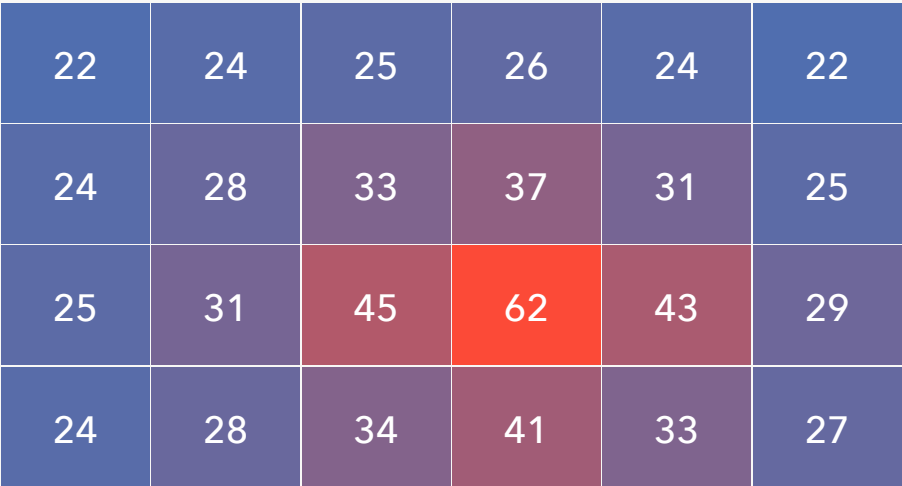
Where should I click to disable notifications?

The blue toggle on the Notifications row.

Locate a control relative to its label.

The image can be a scientific figure

Thermal map of a test plate (°C)



Synthetic measurements; no physical experiment claimed.

QUESTION

What is the hottest measured region?

The image can be a scientific figure

Thermal map of a test plate (°C)

22	24	25	26	24	22
24	28	33	37	31	25
25	31	45	62	43	29
24	28	34	41	33	27

Synthetic measurements; no physical experiment claimed.

REFERENCE ANSWER

What is the hottest measured region?

Row 3, column 4: 62 °C.

The image can be a scientific figure

Thermal map of a test plate (°C)

22	24	25	26	24	22
24	28	33	37	31	25
25	31	45	62	43	29
24	28	34	41	33	27

Synthetic measurements; no physical experiment claimed.

ARCHITECTURAL REQUIREMENT

What is the hottest measured region?

Row 3, column 4: 62 °C.

| Read a scale and local measurements.

The question can be wrong



QUESTION

What color is the bicycle?

The question can be wrong



REFERENCE ANSWER

What color is the bicycle?

No bicycle is visible.

The question can be wrong



ARCHITECTURAL REQUIREMENT

What color is the bicycle?

No bicycle is visible.

Let visual evidence override the wording of the question.

What must survive the visual representation?

QUESTION FAMILY	INFORMATION NEEDED
Counting / spatial	Multiple objects and their arrangement
OCR / charts	Fine detail, labels, values and units
UI / multiple images	Locations and correspondence across views
False premise	Evidence that can contradict the prompt

| Would one global CLIP vector always be enough?

Try the same questions on a small model



A reference answer is not a model prediction.

Predict → run → inspect the exact answer.

[Open the VLM lab ↗](#)

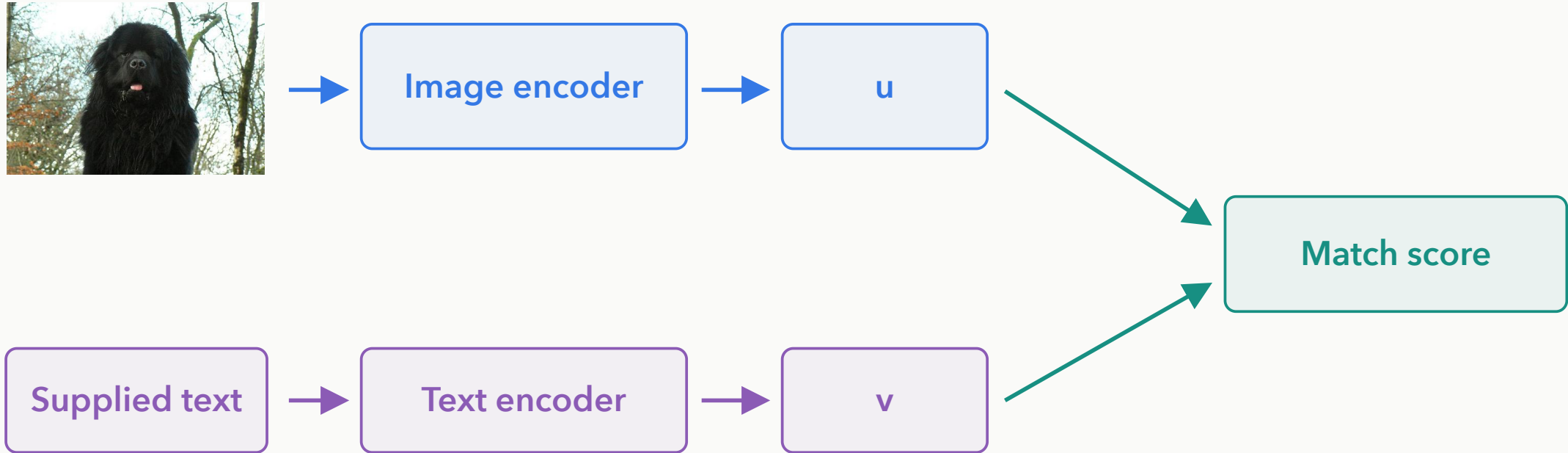
11 image presets, recorded outputs, and an image-swap experiment.

SECTION 02

From matching to generation

Matching supplied text leaves a generation problem.

Recall: CLIP compares an image with supplied text



| Image vector u · text vector v · similarity $u^T v$

Now remove the candidate description

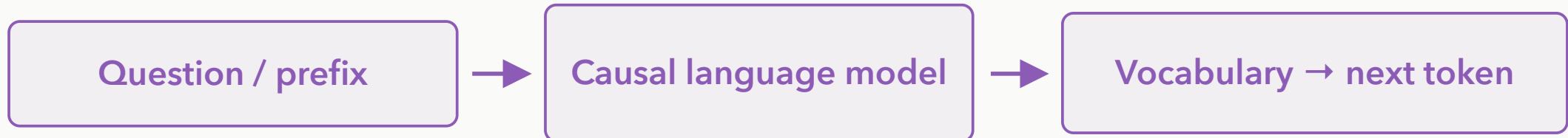


No candidate description supplied.

Where do new words come from?

| Desired output: ? ? ?

We already know how to generate words



| A causal language model maps a prefix to a next-token distribution.

We have two useful pieces



Visual representation

How do we connect them?

Question / prefix



Causal language model



Vocabulary → next token

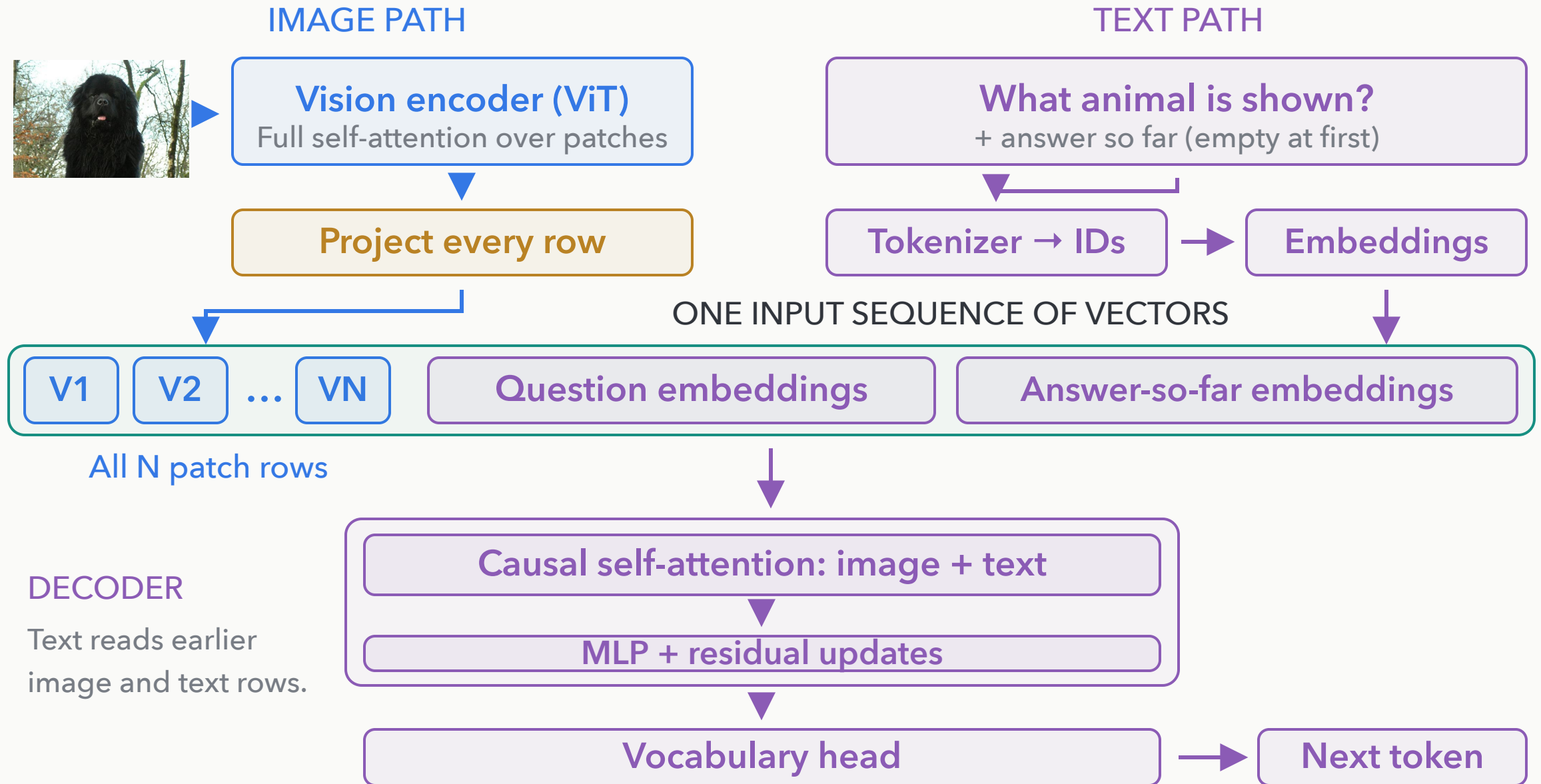
| How can visual states enter the language computation?

SECTION 03

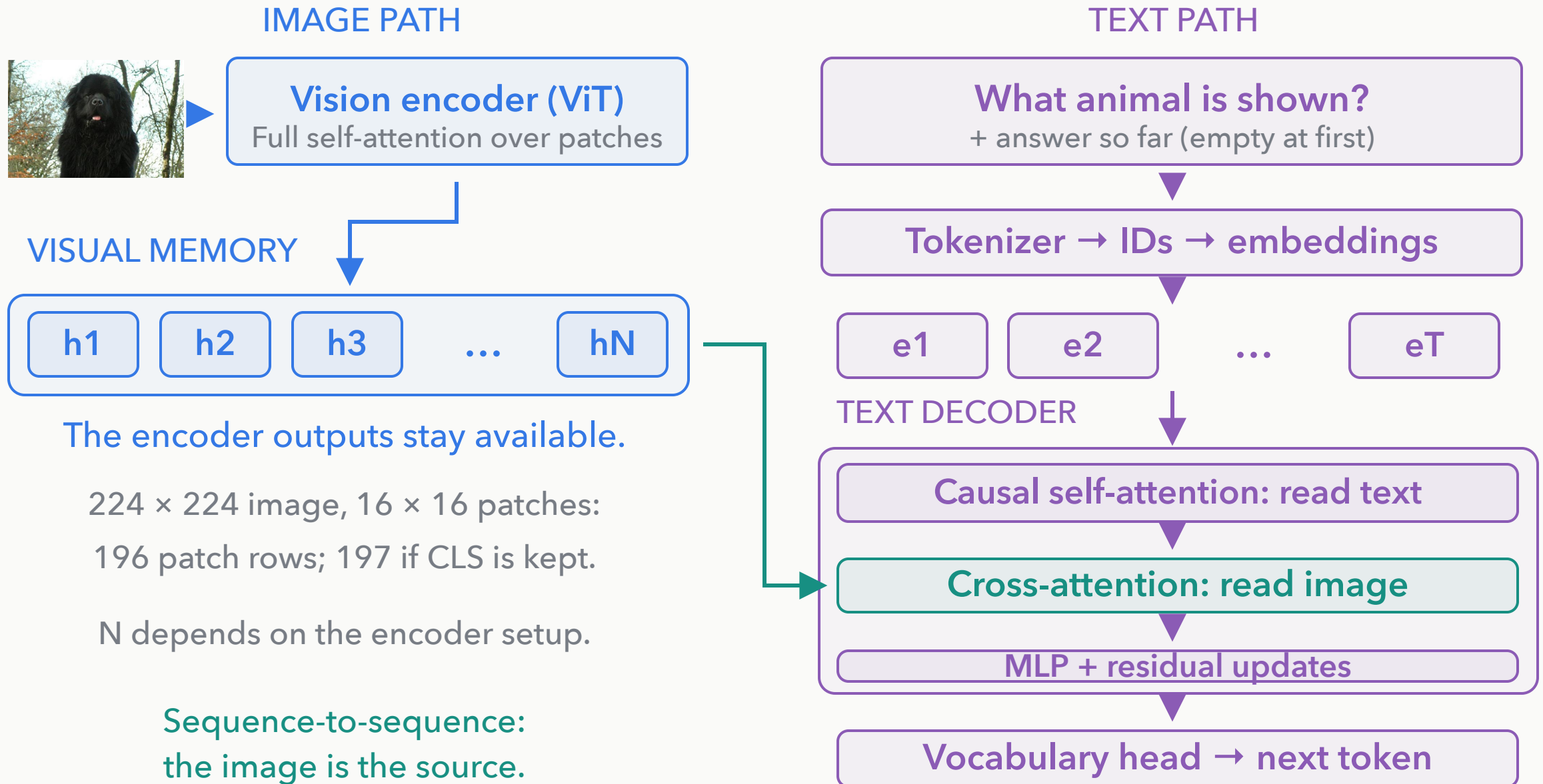
How can vision enter a language model?

First, see the three ideas. Then build each connection.

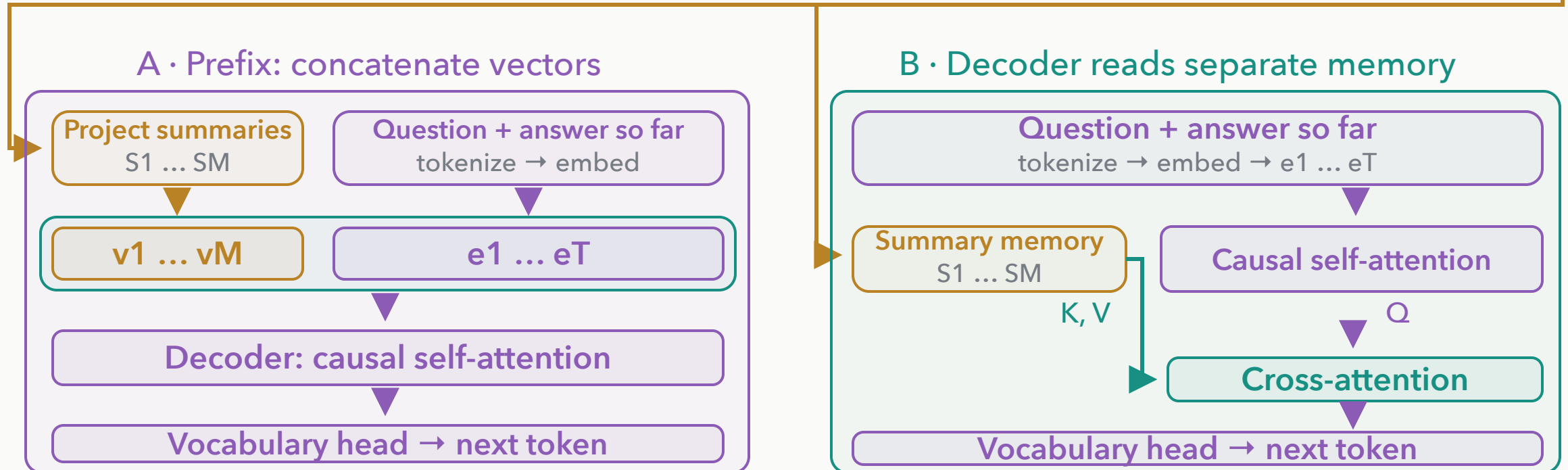
Idea 1 • prepend the image as visual tokens



Idea 2 • let the decoder look at a visual memory



Idea 3 · summarize the image, then use either connection

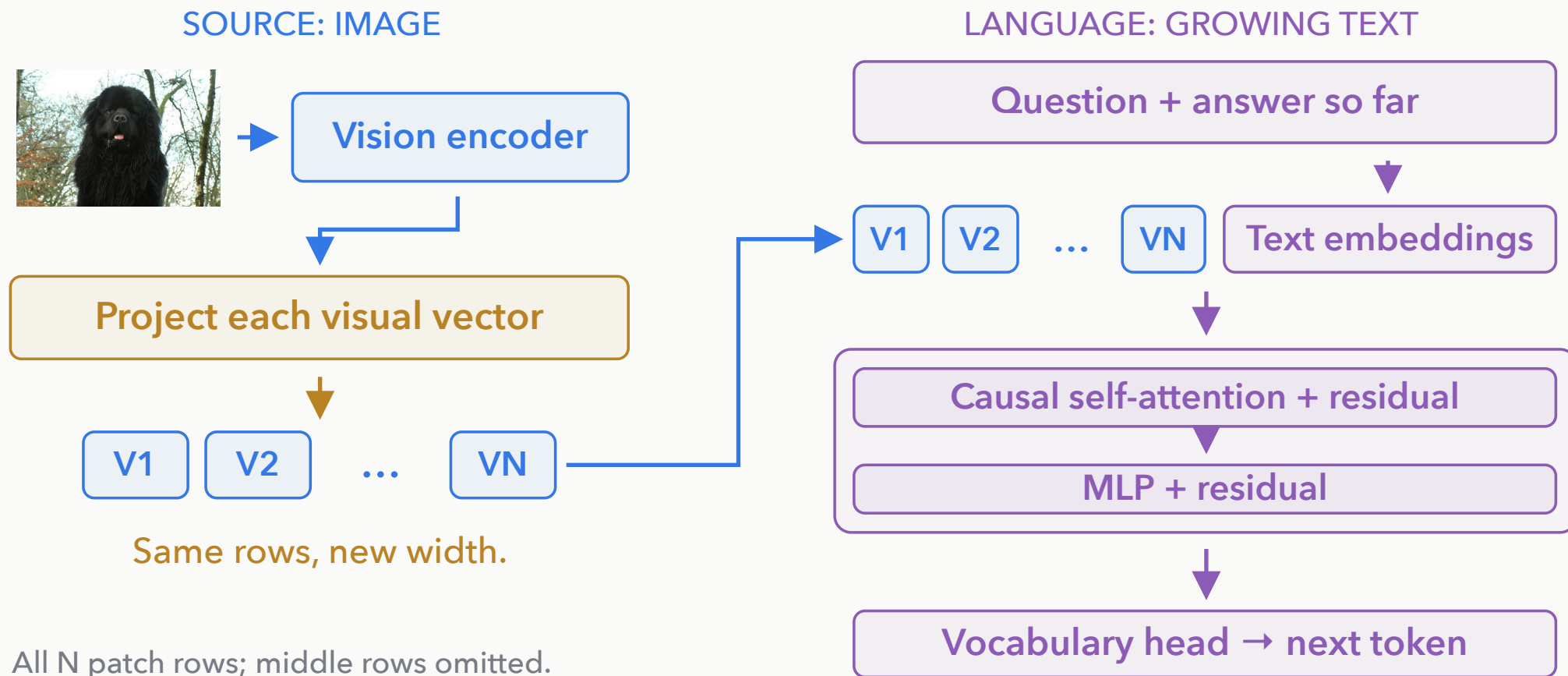


The map: two access routes, one compression choice

IDEA	HIGH-LEVEL DECISION	THEN WE WILL EXPLAIN...
1 · Visual prefix	Put visual vectors before text.	How to make them fit the decoder input.
2 · Separate memory	Let language read the image.	How a text state retrieves visual information.
3 · Shorter summary	Use fewer visual vectors.	How learned readers make the summaries.

Ideas 1 and 2 choose the access route. Idea 3 can be used with either.

Idea 1 in detail · build the visual prefix



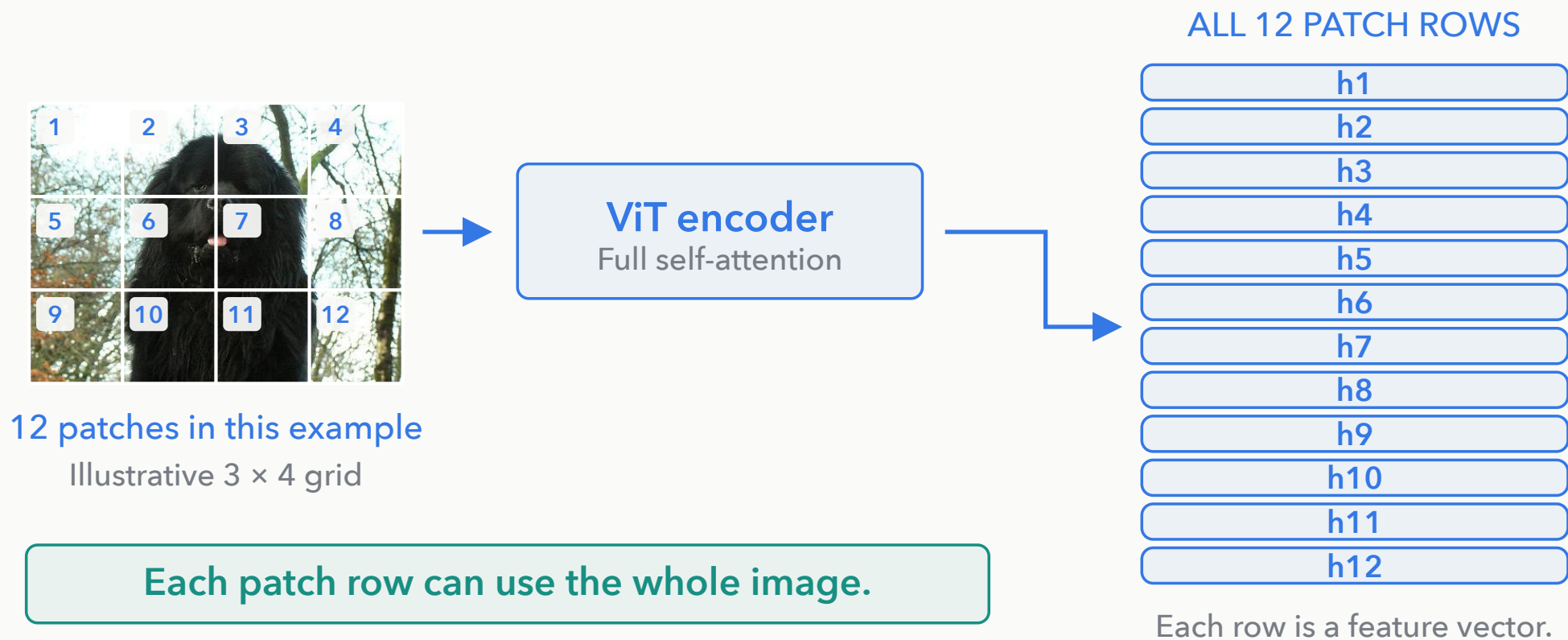
One sequence; the same causal self-attention can use both kinds of context.

What must we build to make this work?

JOB	WHY IT IS NEEDED
1. Keep image feature vectors	The decoder needs access to visual evidence.
2. Match the decoder input width	All input rows must have the same number of coordinates.
3. Place them before the question	Causal attention can read earlier context.

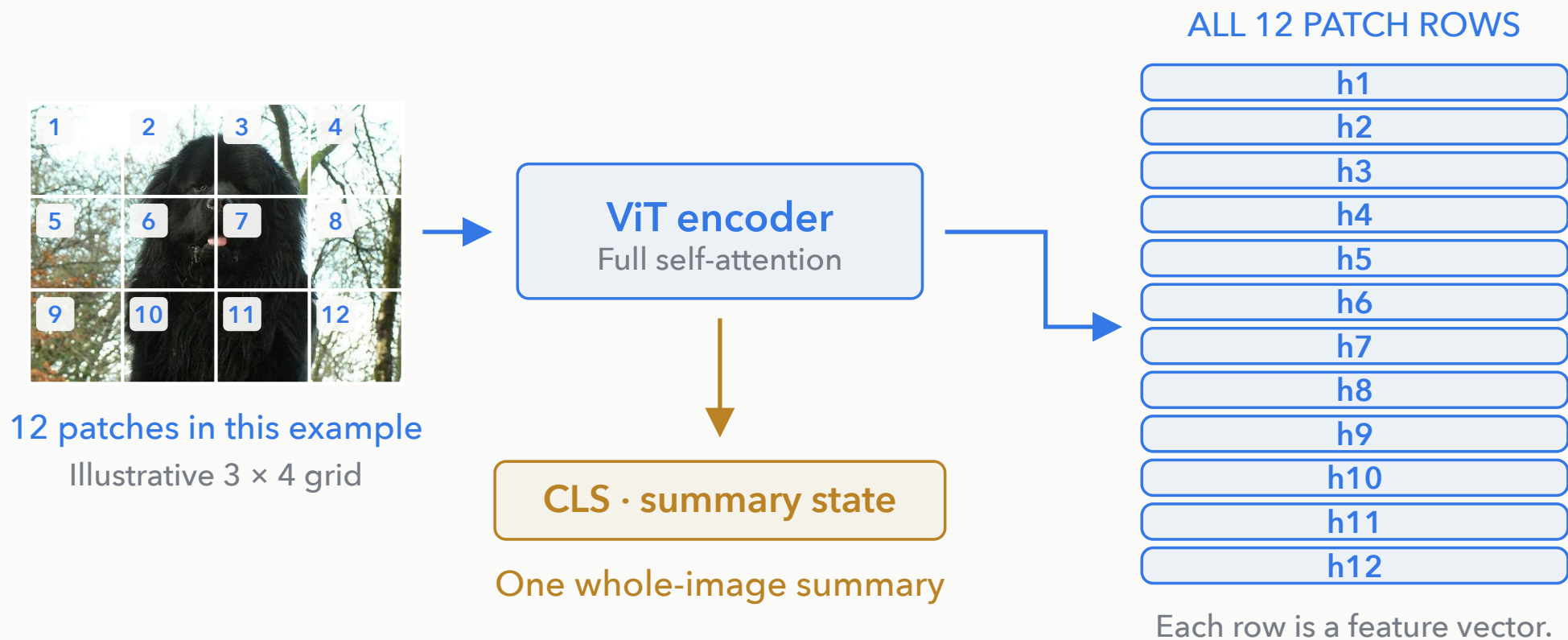
| We already have the decoder. Now build its visual input.

Recall ViT: every patch already has an output row



| Patch 1 has row h1, patch 2 has row h2, and so on through patch 12.

ViT outputs a CLS state and one row per patch



Here: 1 CLS row + 12 patch rows. We keep all 12 patch rows.

Why keep more than the global summary?

CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

Subtotal ₹1,000

GST ₹180

TOTAL ₹1,180

RECALL OUR OPENING QUESTION

What is the invoice ID?

What percentage is GST?

The answer needs small text, several numbers, and their roles.

Give language access to more visual evidence before asking it to answer.

A “visual state” is just one row of features

One image → many rows

$$H_V = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{bmatrix}$$

Read the notation in plain language

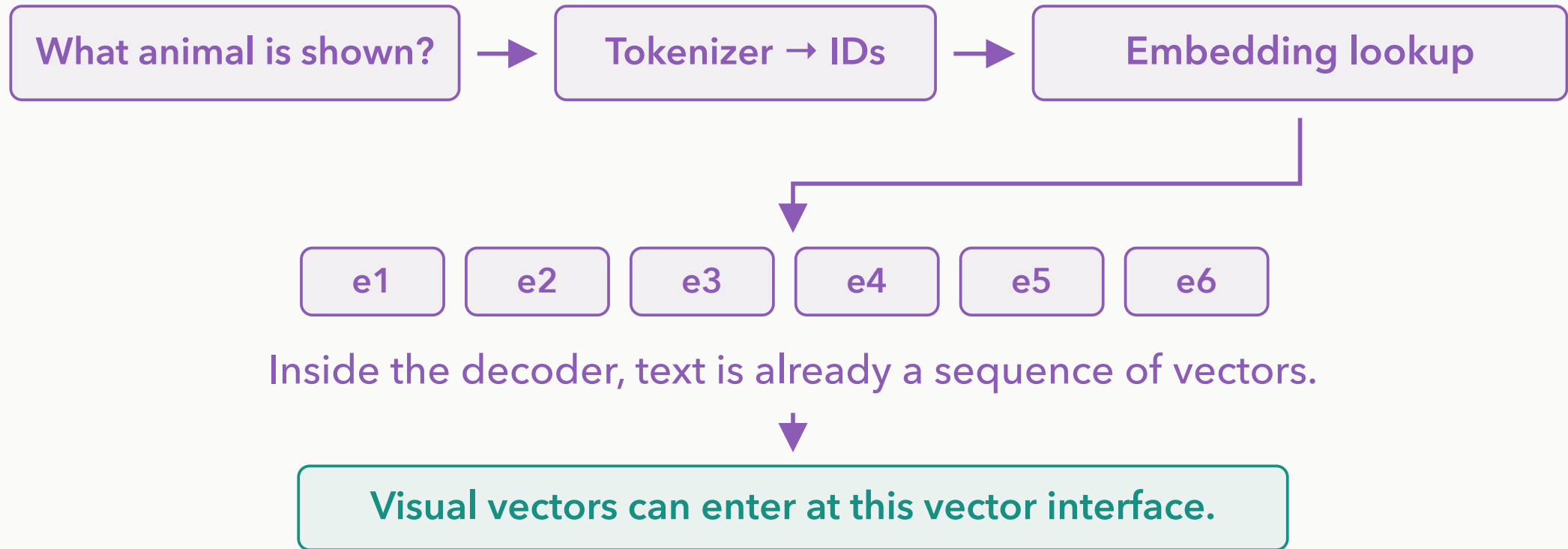
N = number of visual rows.

d_V = numbers in each row.

h_i = the feature row at patch position i .

Rows are vectors, not object labels or named semantic attributes.

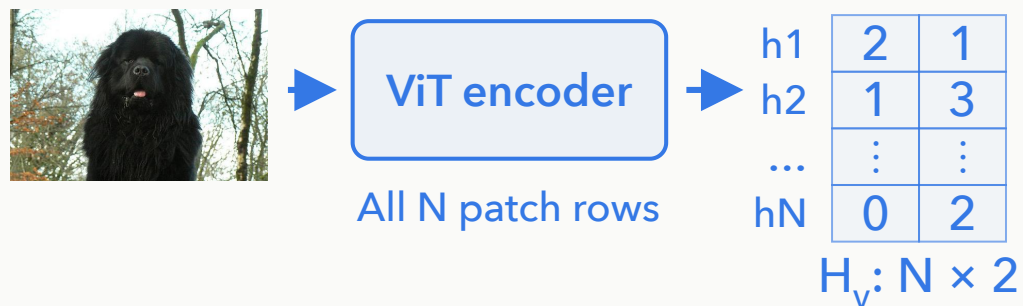
Recall: the decoder already consumes vectors



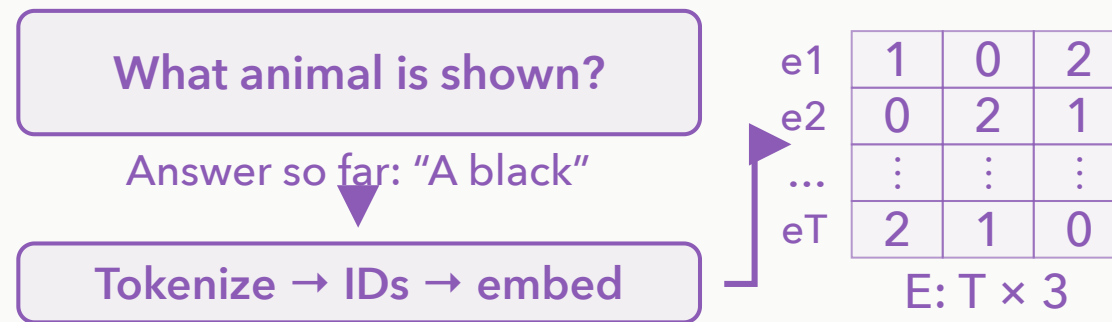
| Text IDs become embeddings before reaching the Transformer blocks.

The image rows have 2 coordinates; text rows have 3

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Can these rows enter the same decoder?

h_1

2	1
---	---

2 coordinates per image row

$\neq$ width

e_1

1	0	2
---	---	---

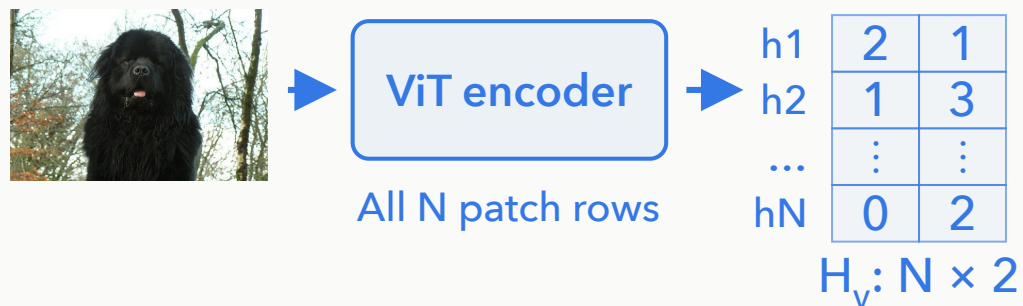
3 coordinates per text row

We need to turn each image row into a 3-coordinate vector.

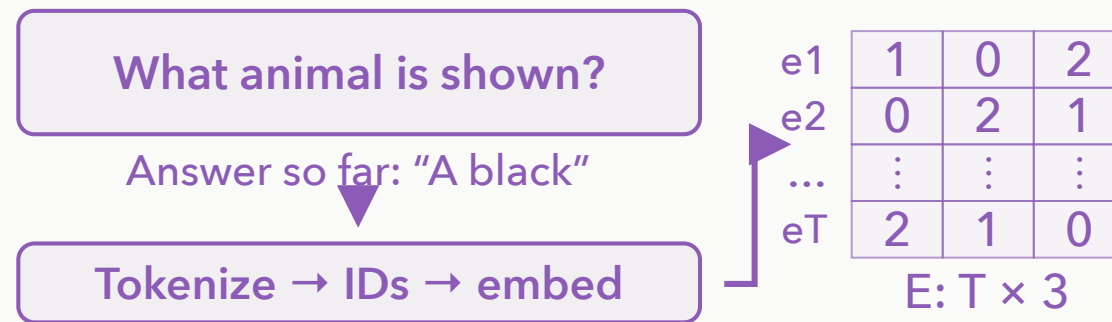
To form one input sequence, every vector must have the decoder's input width.

Add a projector to the image path

IMAGE → FEATURE VECTORS

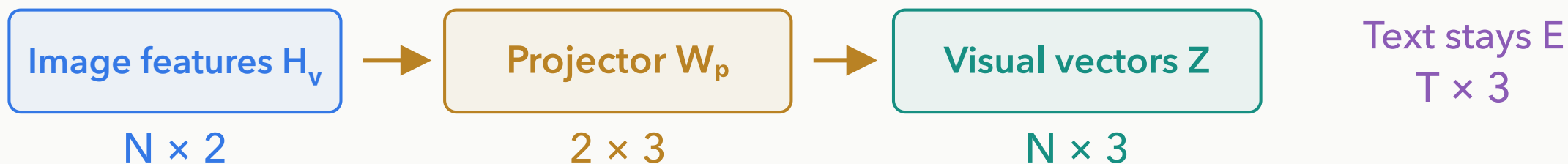


TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Insert a shared projector on the image path

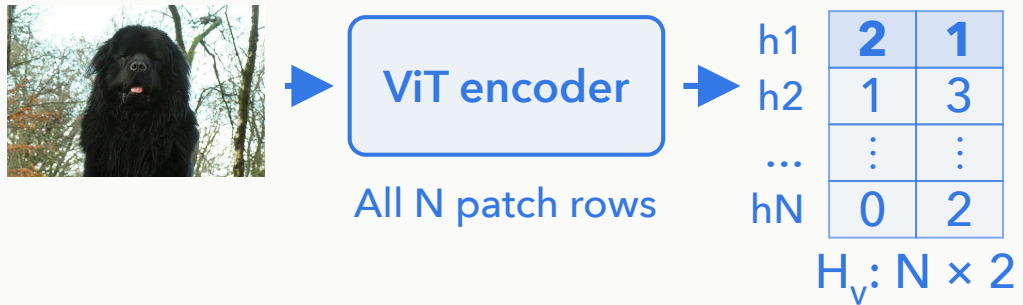


$Z = H_V W_p$ · Keep N rows. Change 2 coordinates to 3.

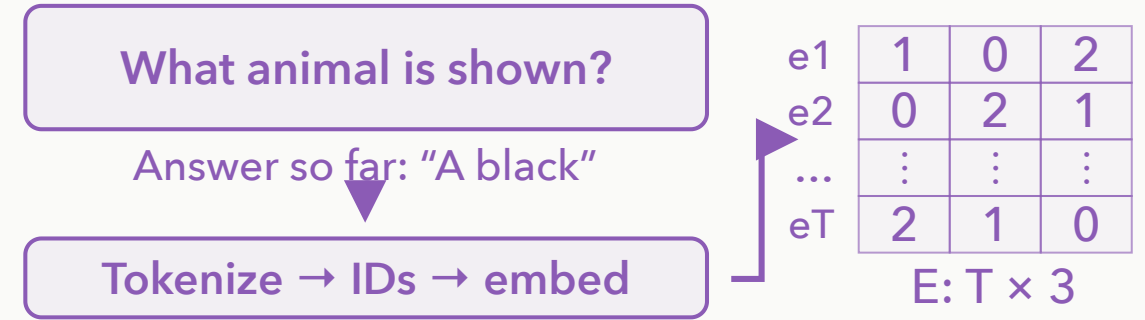
| The projector changes $N \times 2$ image features into $N \times 3$ visual vectors.

Zoom into one row from the image

IMAGE → FEATURE VECTORS

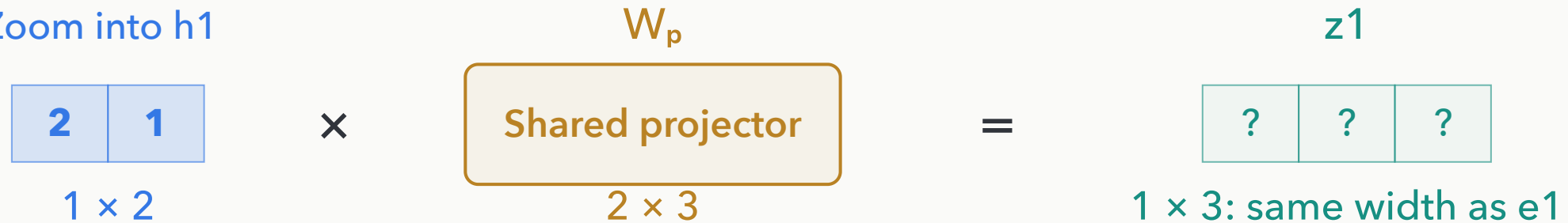


TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h_1

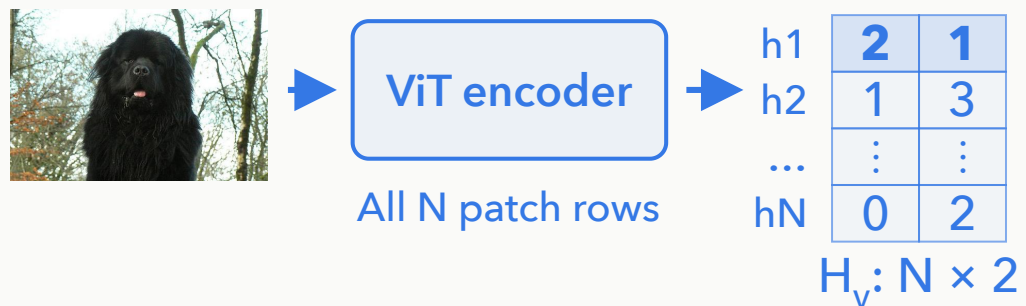


h_1 is one of the image rows above. Its two coordinates are 2 and 1.

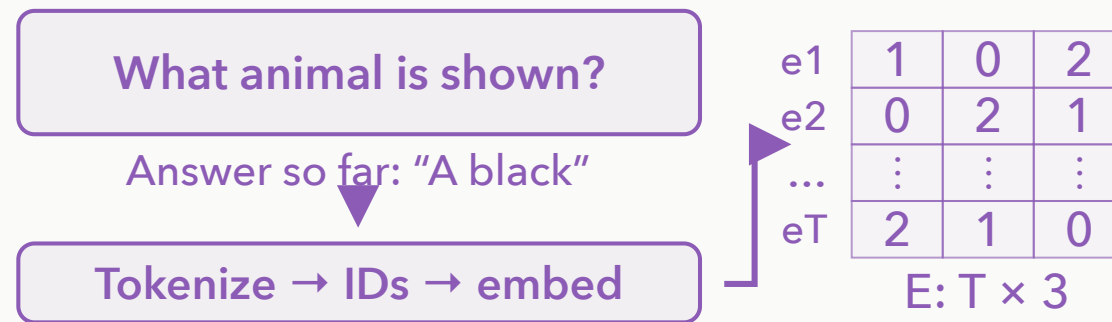
Take one 2-coordinate row from the image feature sequence.

Use a 2×3 matrix to change the width

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h_1

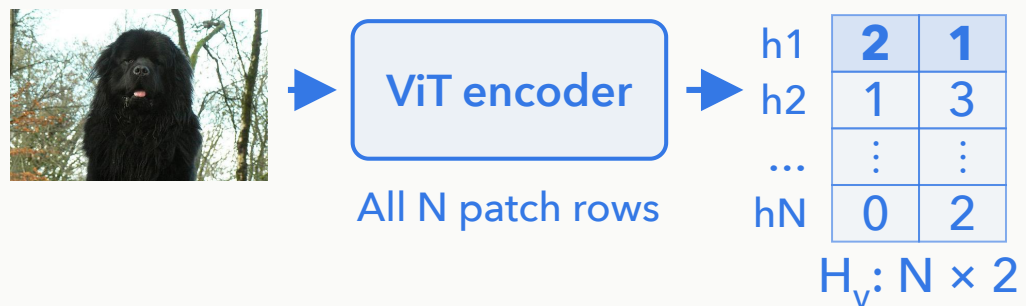
$$\begin{array}{c}
 \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\
 1 \times 2
 \end{array}
 \times
 \begin{array}{c}
 W_p \\
 \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\
 2 \times 3
 \end{array}
 =
 \begin{array}{c}
 z_1 \\
 \begin{array}{|c|c|c|} \hline ? & ? & ? \\ \hline \end{array} \\
 1 \times 3: \text{ same width as } e_1
 \end{array}$$

Each of the three columns of W_p will make one output coordinate.

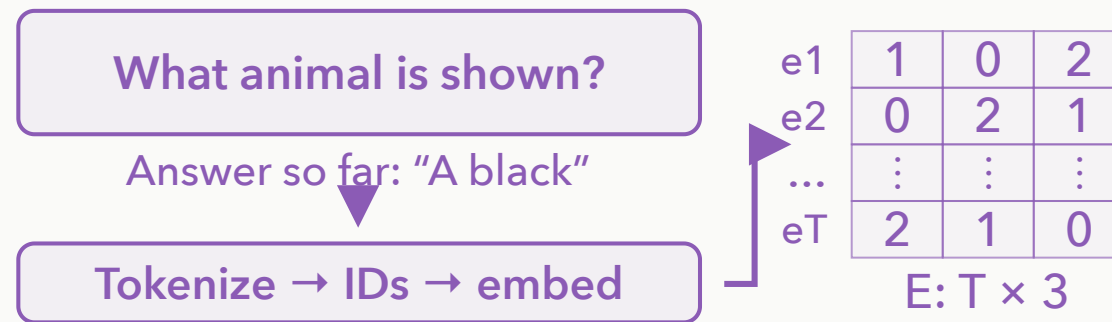
Each column makes one of the 3 decoder-input coordinates.

The first column makes the first coordinate

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h1

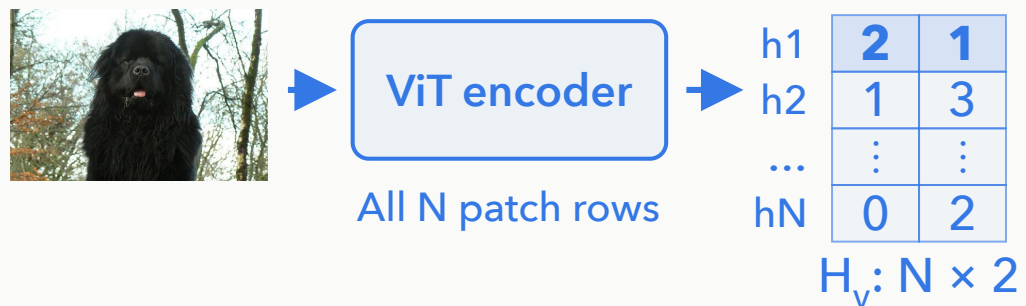
$$\begin{array}{c} \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\ 1 \times 2 \end{array} \times \begin{array}{c} W_p \\ \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\ 2 \times 3 \end{array} = \begin{array}{c} z1 \\ \begin{array}{|c|c|c|} \hline ? & ? & ? \\ \hline \end{array} \\ 1 \times 3: \text{ same width as } e1 \end{array}$$

First column: how much of 2, and how much of 1, should we use?

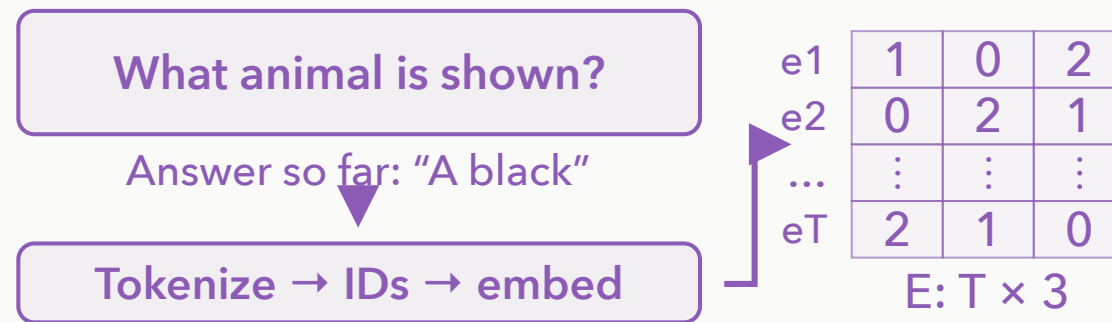
Which input coordinates does this column combine?

The first output coordinate is 2

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h1

2	1
---	---

1×2

×

W_p

1	0	1
0	1	1

2×3

=

2	?	?
---	---	---

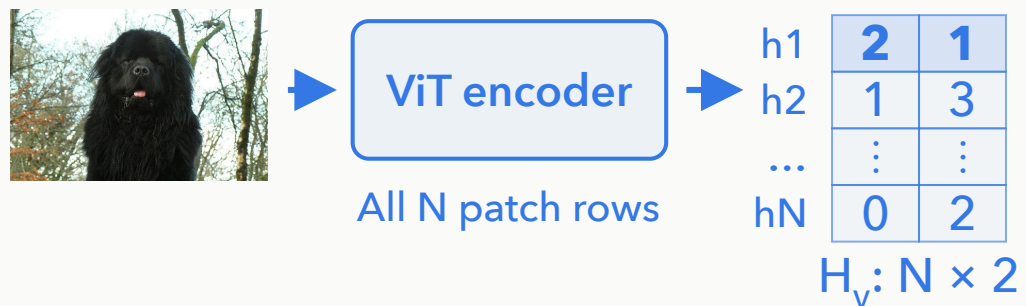
1×3 : same width as e1

First coordinate: $2 \times 1 + 1 \times 0 = 2$

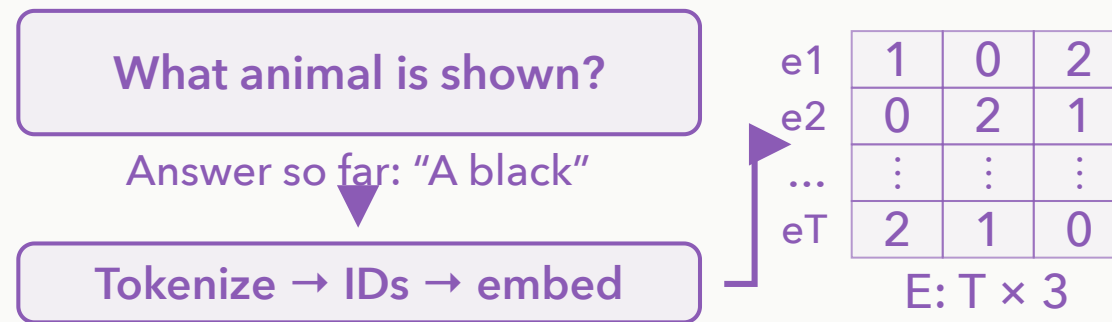
The first output coordinate copies the first input in this toy map.

The second column reuses the same image row

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

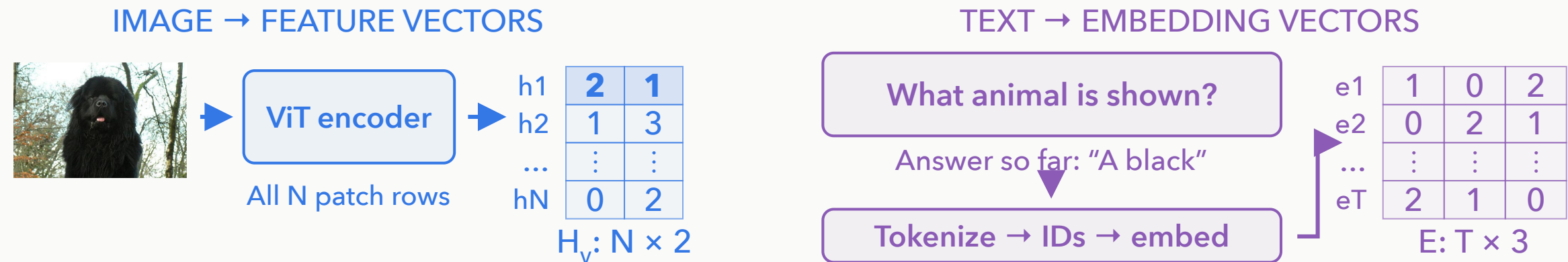
Zoom into h1

$$\begin{array}{c} \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\ 1 \times 2 \end{array} \times \begin{array}{c} W_p \\ \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\ 2 \times 3 \end{array} = \begin{array}{c} z1 \\ \begin{array}{|c|c|c|} \hline 2 & ? & ? \\ \hline \end{array} \\ 1 \times 3: \text{ same width as } e1 \end{array}$$

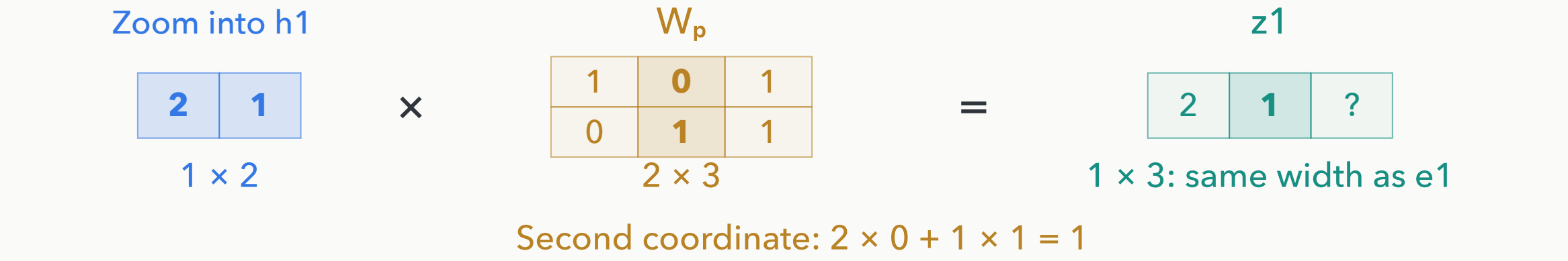
Second column: use the same input [2, 1] again.

Now use the same input row with the second column.

The second output coordinate is 1



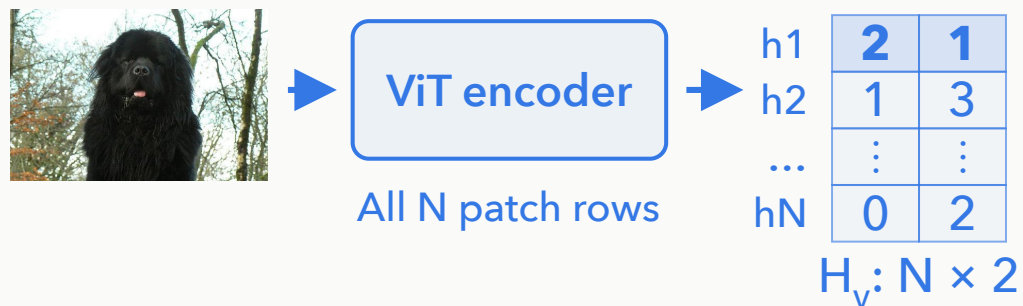
Toy widths and values; not measured from this image or text. Middle rows omitted.



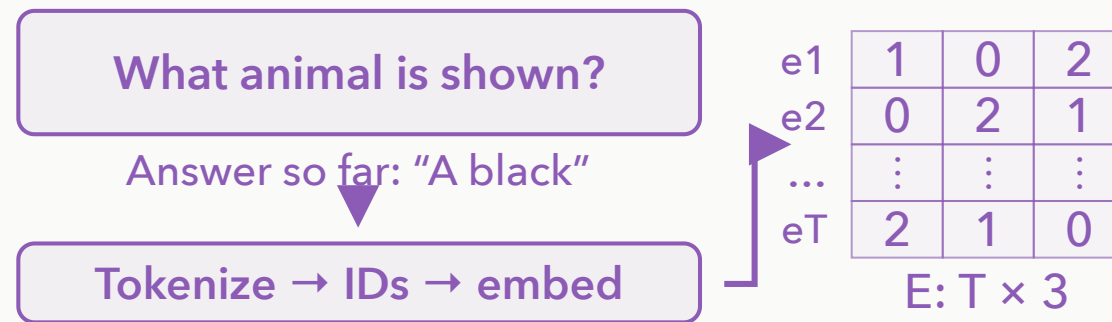
The second output coordinate copies the second input in this toy map.

The third column combines both input coordinates

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h_1

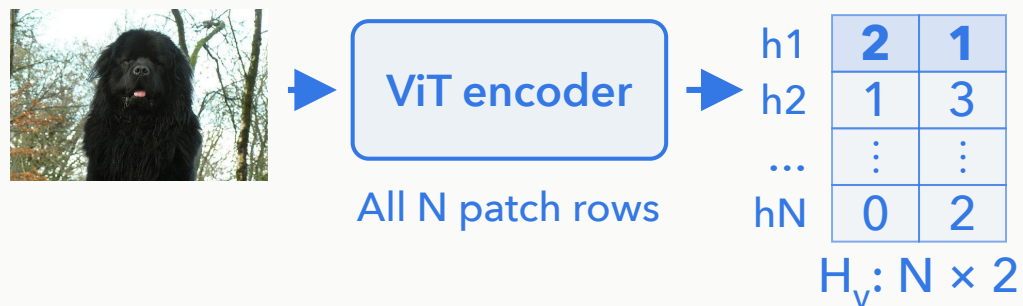
$$\begin{array}{c} \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\ 1 \times 2 \end{array} \times \begin{array}{c} W_p \\ \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\ 2 \times 3 \end{array} = \begin{array}{c} z_1 \\ \begin{array}{|c|c|c|} \hline 2 & 1 & 3 \\ \hline \end{array} \\ 1 \times 3: \text{ same width as } e_1 \end{array}$$

$$\text{Third coordinate: } 2 \times 1 + 1 \times 1 = 3$$

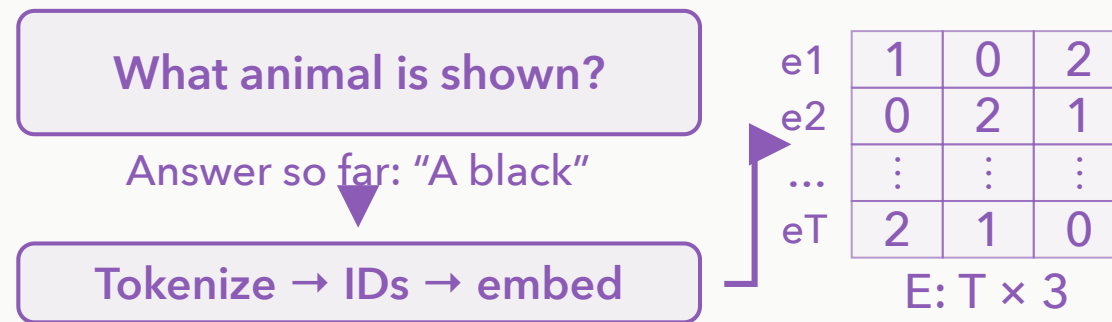
The third output coordinate adds the two inputs in this toy map.

Now this image vector has the required width

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Zoom into h1

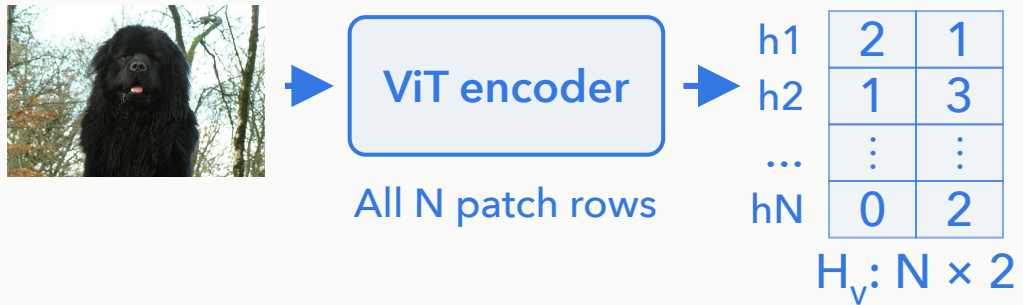
$$\begin{array}{c}
 \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\
 1 \times 2
 \end{array}
 \times
 \begin{array}{c}
 W_p \\
 \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\
 2 \times 3
 \end{array}
 =
 \begin{array}{c}
 z1 \\
 \begin{array}{|c|c|c|} \hline 2 & 1 & 3 \\ \hline \end{array} \\
 1 \times 3: \text{ same width as } e1
 \end{array}$$

The image row [2, 1] has become [2, 1, 3]. The text rows stay as they are.

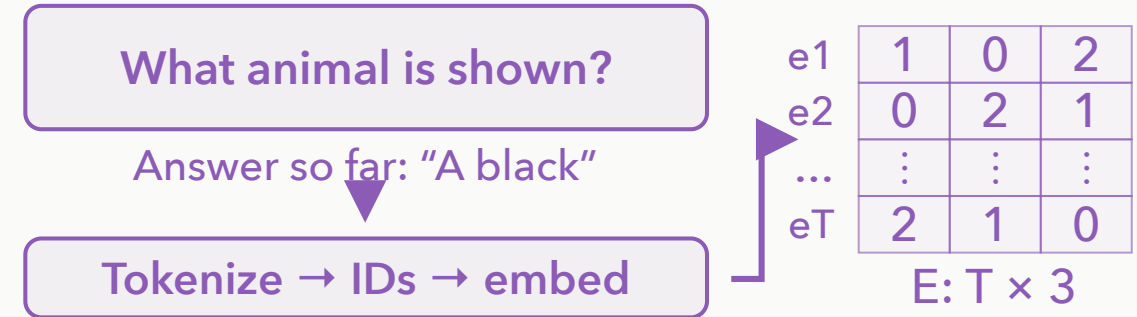
| The row [2, 1, 3] fits beside the 3-coordinate text embeddings.

Use the same matrix for all N image rows

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Every image row

h_1	2	1
h_2	1	3
...	$\vdots$	$\vdots$
h_N	0	2

$N \times 2$

Same W_p

1	0	1
0	1	1

2×3

=

All N projected rows

z_1	2	1	3
z_2	1	3	4
...	$\vdots$	$\vdots$	$\vdots$
z_N	0	2	2

$N \times 3$

Width now matches E

N is unchanged.

N rows stay N rows; every projected row now has 3 coordinates.

Check: did the projector create more visual tokens?

Before

576 visual rows.

Each row has 1,024 coordinates.

| Predict the row count before advancing.

After a row-wise projector

Each row has 4,096 coordinates.

How many rows now?

Still 576 rows: width and count are different

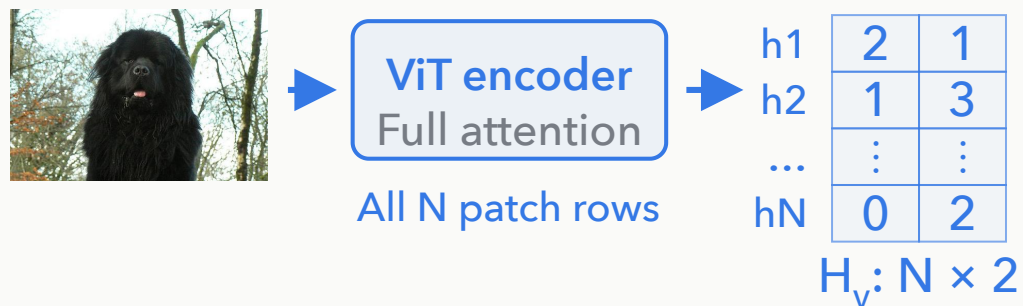
$$\underbrace{H_V}_{576 \times 1024} \quad \underbrace{W_P}_{1024 \times 4096} = \underbrace{Z}_{576 \times 4096}$$

Same positions. Different number of features at each position.

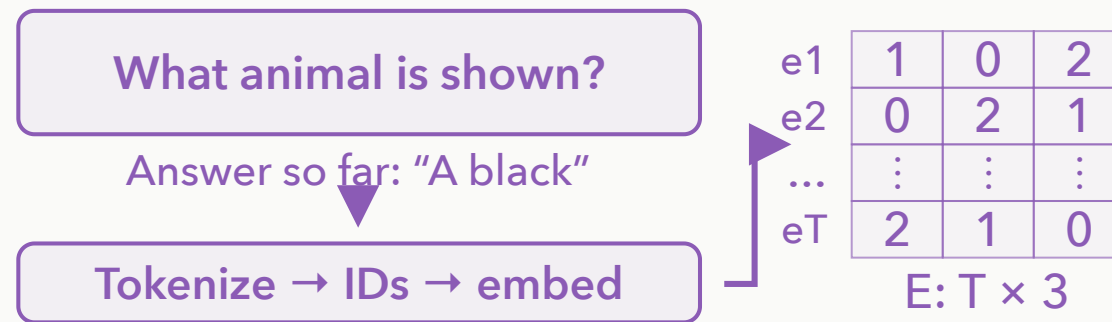
| We will learn how to reduce the row count in the compression section.

Now the two sequences fit one decoder input

IMAGE → FEATURE VECTORS



TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Image → shared $W_p \rightarrow Z$

N projected rows

+ T text rows

$(N + T) \times 3$

z1	2	1	3
...	⋮	⋮	⋮
zN	0	2	2
e1	1	0	2
...	⋮	⋮	⋮
eT	2	1	0



Language decoder



Head → next token

Causal from z_1 onward

The decoder mask covers image AND text rows.

The mask applies inside the decoder, including its visual positions.

Build the input row: one projected visual vector



Visual vectors

| Visual vectors and text embeddings share the decoder's input width.

Keep the rest of the projected visual sequence



Visual vectors

| Visual vectors and text embeddings share the decoder's input width.

Then place the user's text

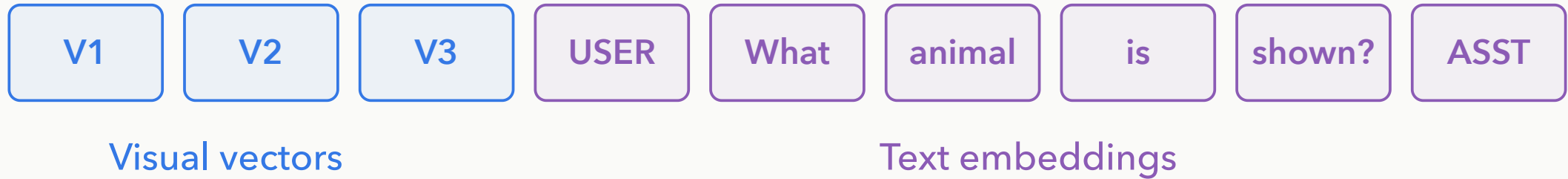


Visual vectors

Text embeddings

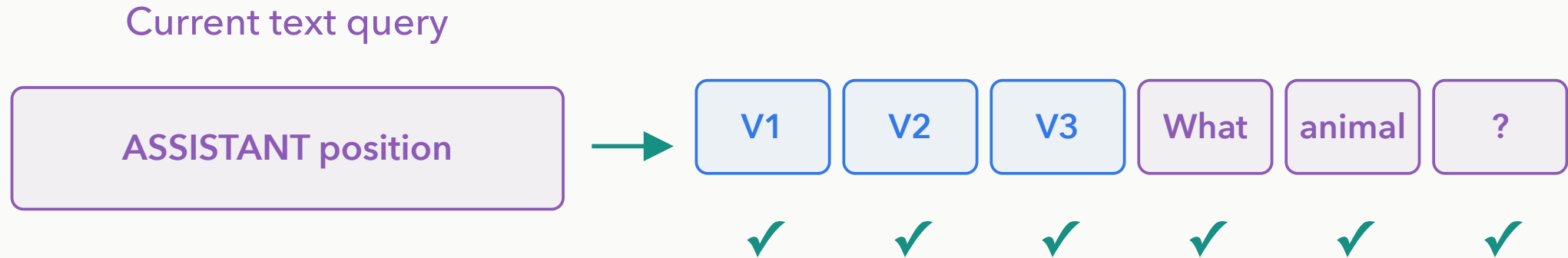
Visual vectors and text embeddings share the decoder's input width.

Finish the question; start the assistant's answer



Visual vectors and text embeddings share the decoder's input width.

What may the current text position read?

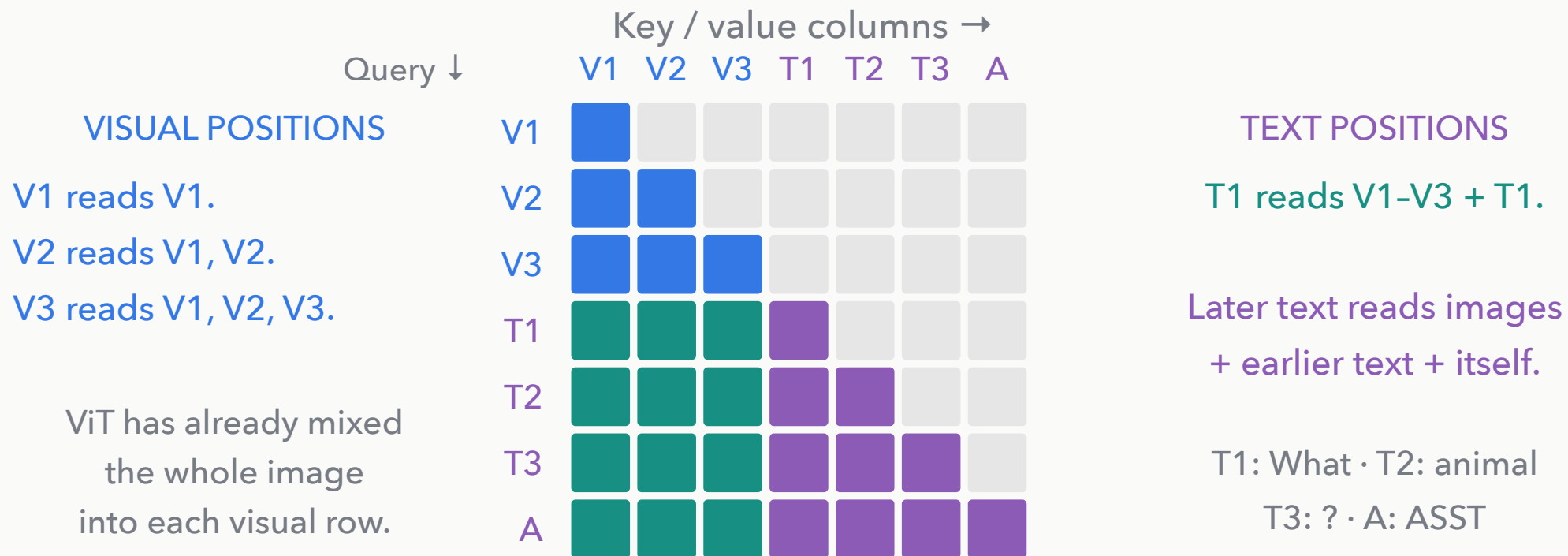


All six are earlier context.

No future text position is available.

| The assistant's state can use both the picture and the question.

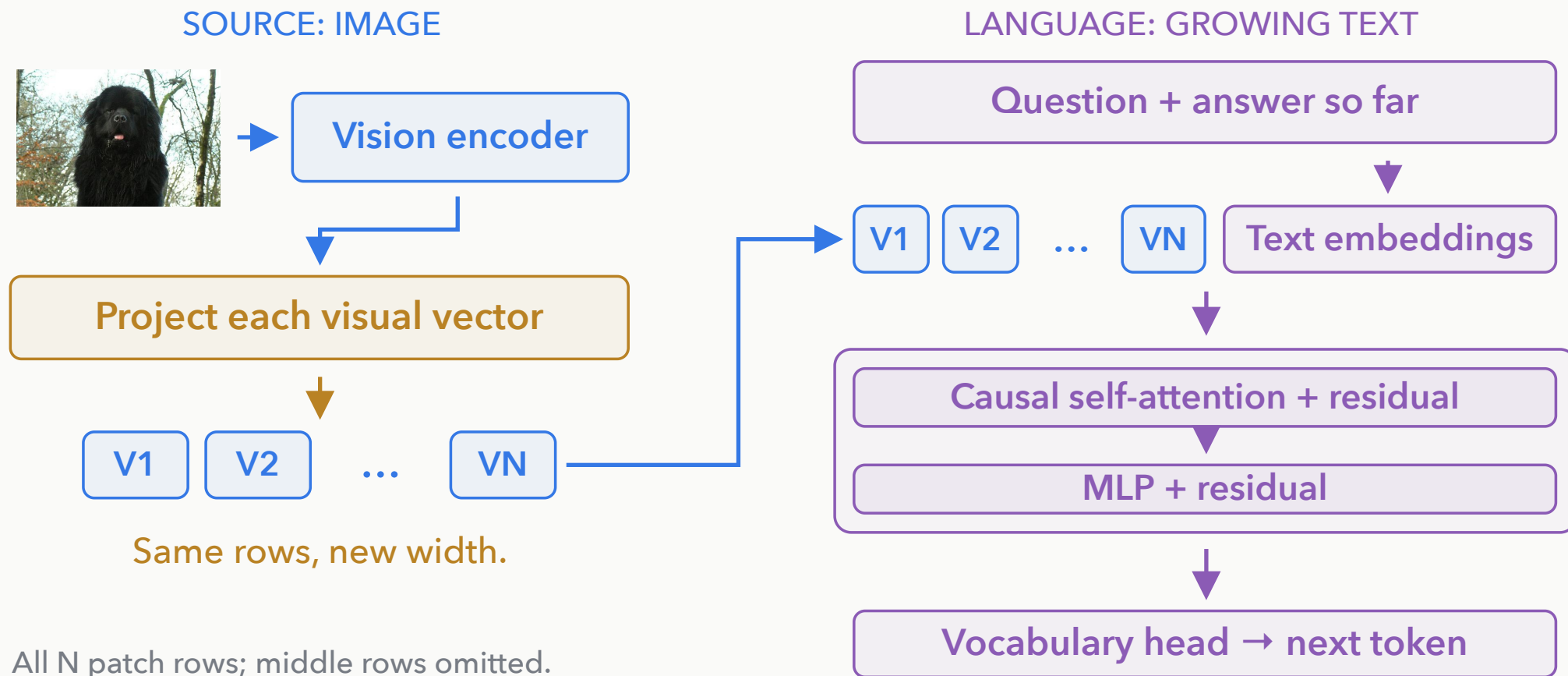
In this decoder, the causal mask starts at V1



Known image + prompt rows run together in one masked pass.

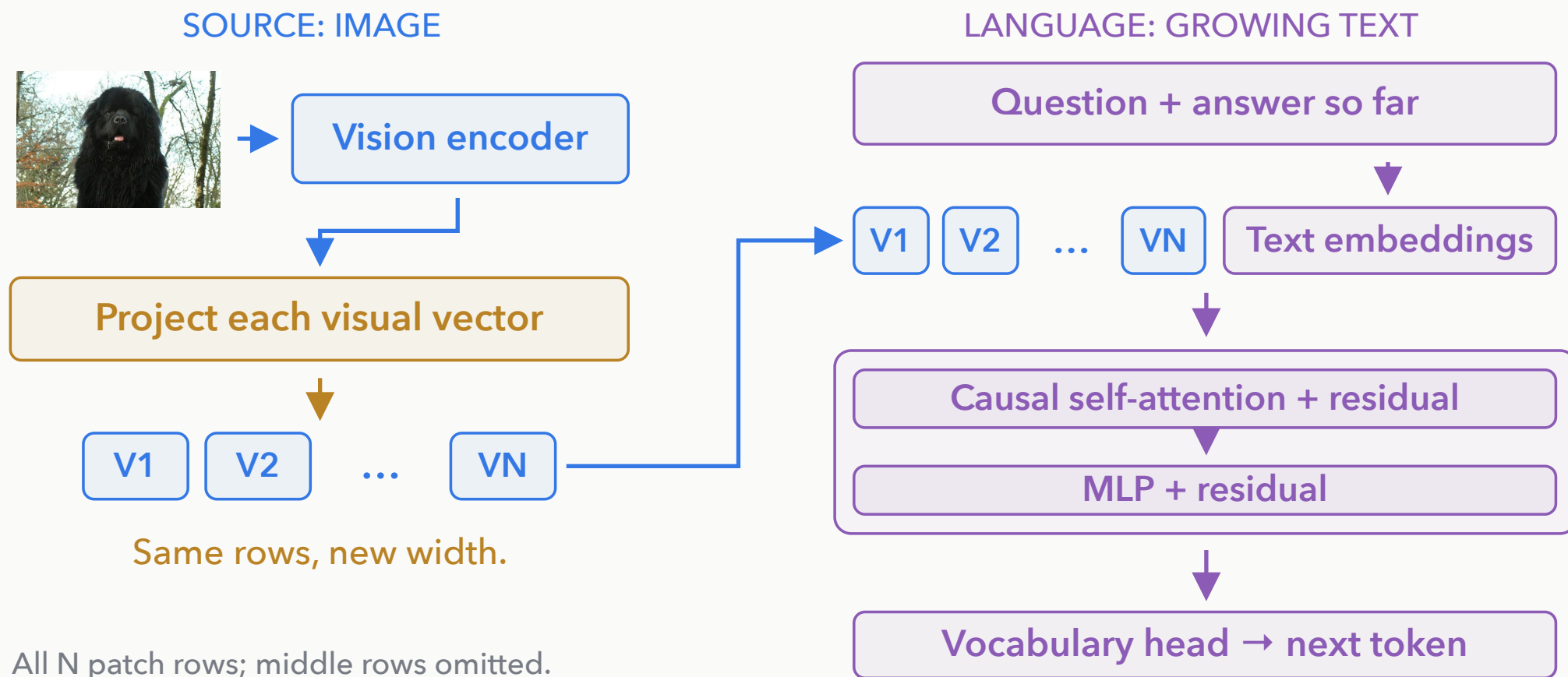
| Colored: allowed · grey: blocked. The triangle includes the visual positions.

Back to the big picture: the image is earlier context



We have built all three jobs: visual rows, matching width, and a prefix.

This is the basic LLaVA-style connection

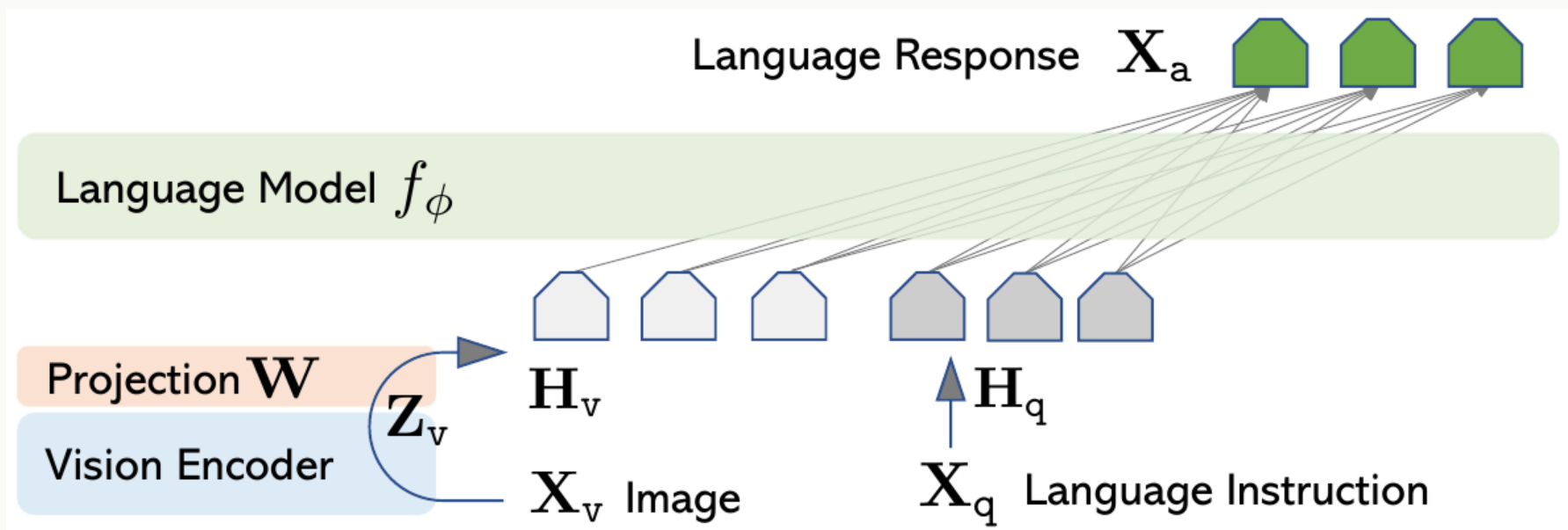


Projected patch vectors occupy earlier positions; they do not become word IDs.

Visual Instruction Tuning

Haotian Liu • Chunyuan Li • Qingyang Wu • Yong Jae Lee

[Read the paper ↗](#)



Original Figure 1 • Image features and instruction embeddings enter the same language model.

1 • Vision encoder

CLIP ViT-L/14 supplies image features Z_v .

2 • Projection W

The original paper uses a linear map to match the LM embedding width.

3 • Language model

Vicuna reads the visual vectors and text, then predicts the response.

Idea 1, end to end in one function

**image**

One image

question = "What animal is shown?"**answer_so_far** = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

<code>def next_token(image, question, answer_so_far):</code>	
<code> patches = vit(image).patch_states</code>	$N \times d_V$
<code> visual = projector(patches)</code>	$N \times d_{LM}$
<code> ids = tokenize(question, answer_so_far)</code>	T token IDs
<code> text = embed(ids)</code>	$T \times d_{LM}$
<code> inputs = concat([visual, text], axis="tokens")</code>	$(N + T) \times d_{LM}$
<code> hidden = causal_decoder(inputs)</code>	$(N + T) \times d_{LM}$
<code> scores = lm_head(hidden[-1])</code>	Vocabulary scores
<code> return argmax(scores)</code>	Next text token ID

causal_decoder stacks causal self-attention + MLP blocks over *all* input positions.

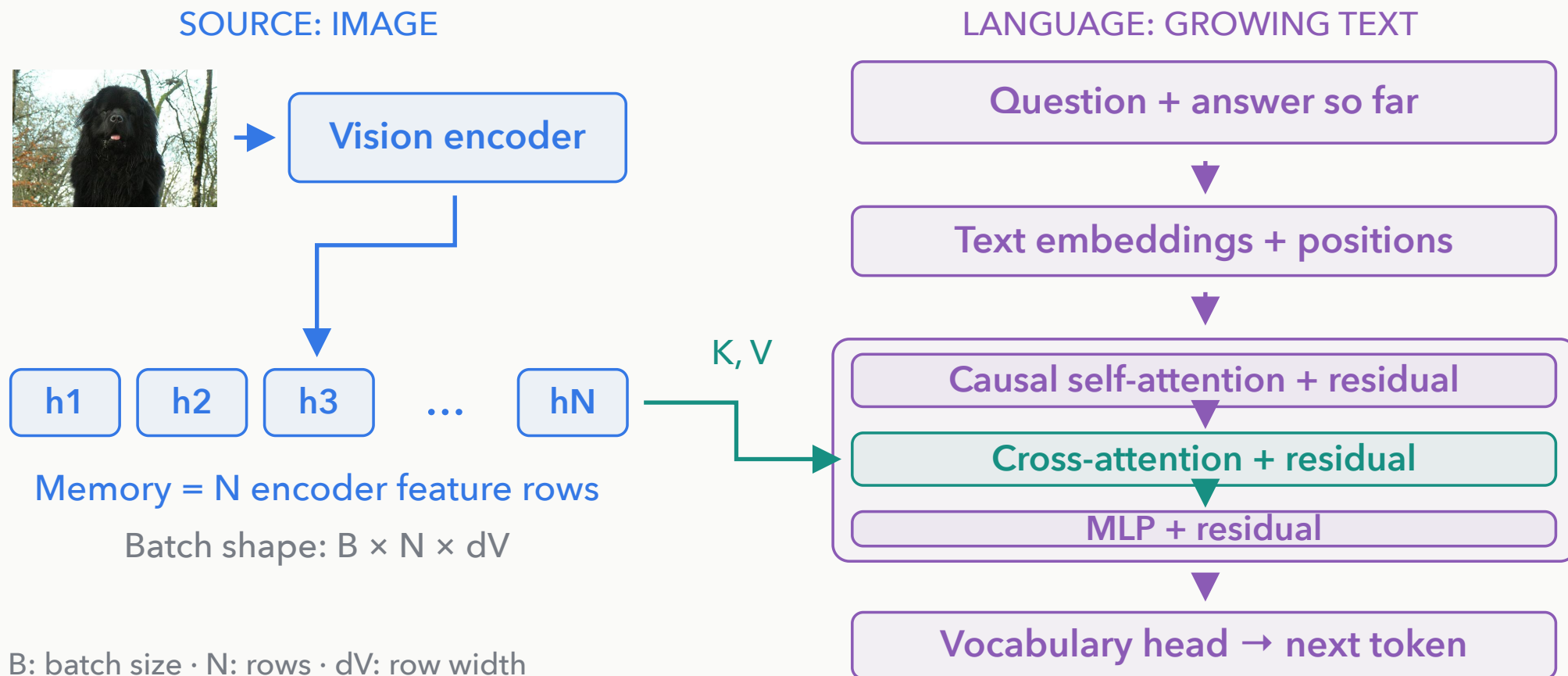
Display the returned token with the tokenizer; append its ID to the answer and predict again.

SECTION 04

Could vision remain separate?

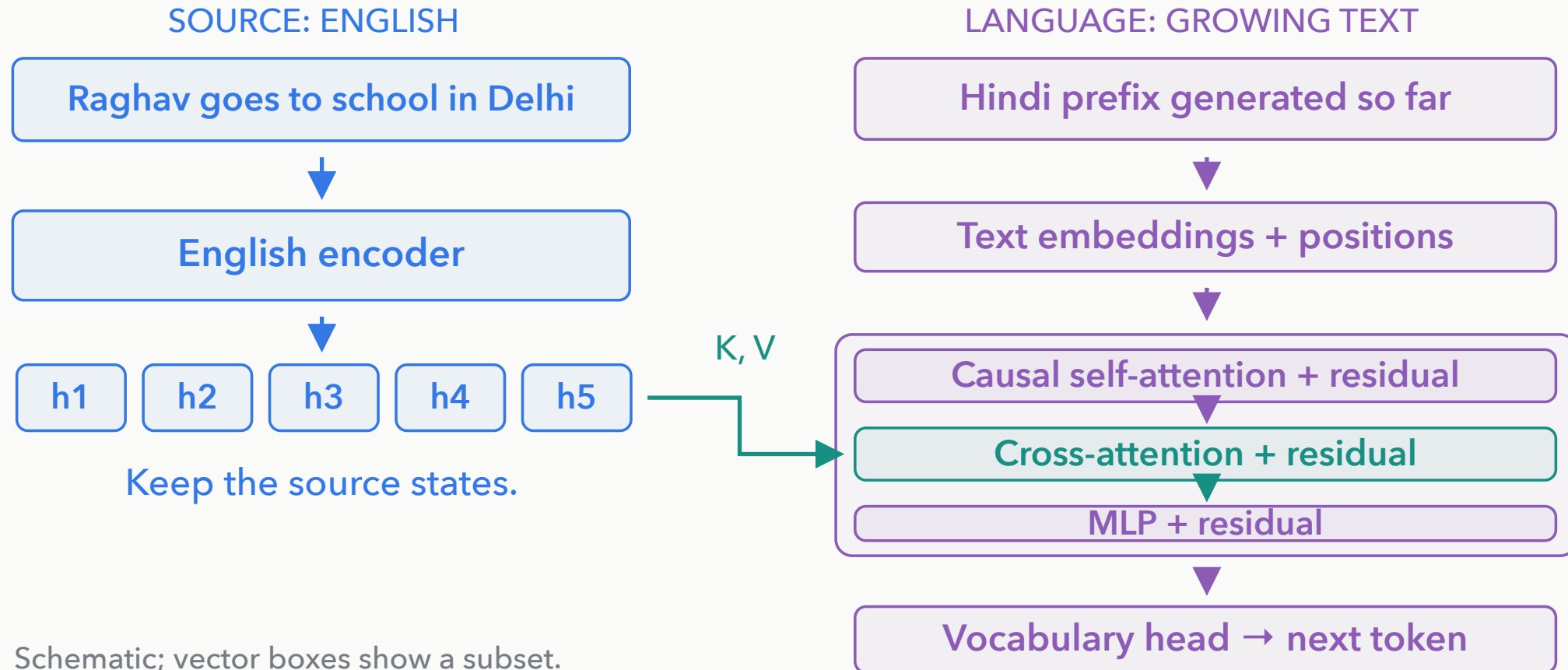
Return to Idea 2: how can language read separate visual memory?

Idea 2 in detail · read a separate visual memory



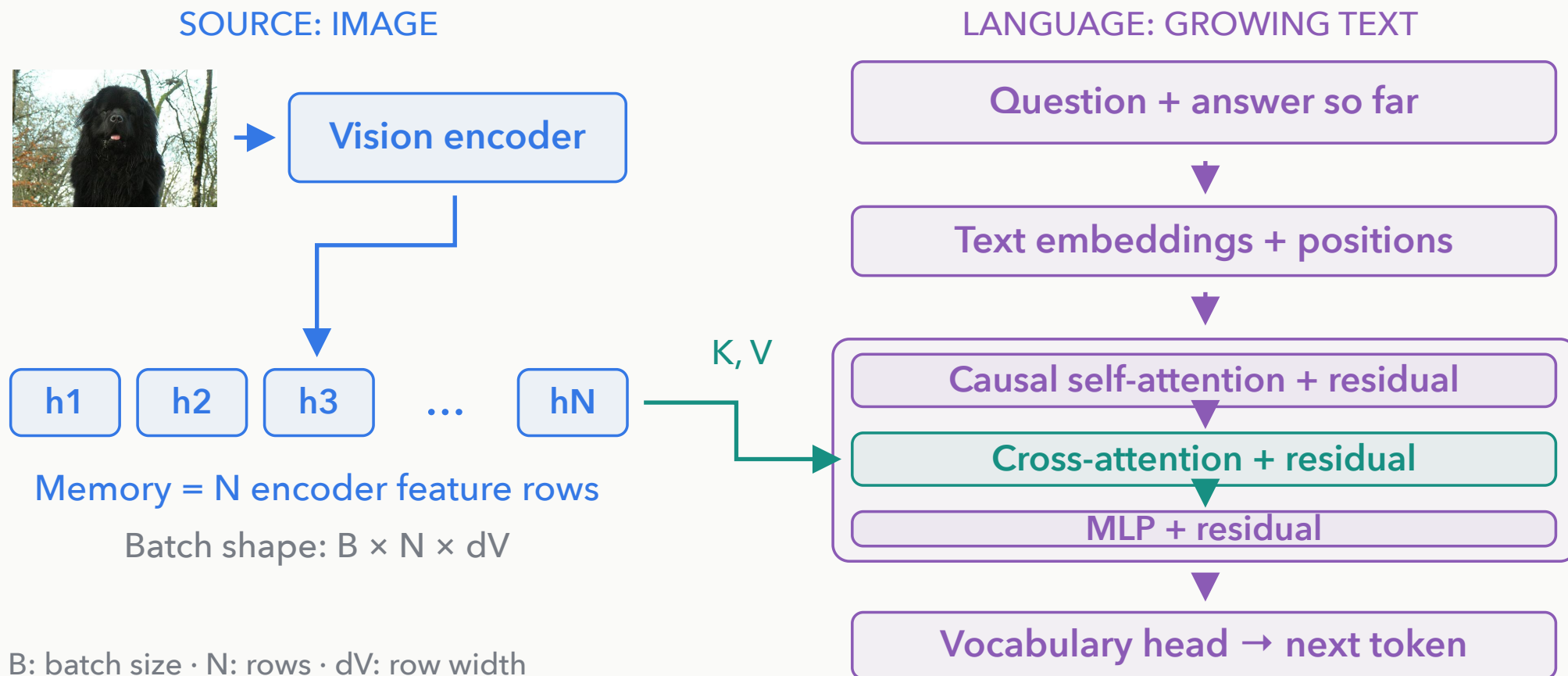
Causal self-attention reads text; cross-attention reads image memory.

We already did this with English-to-Hindi translation



English source states stayed available while the Hindi prefix grew.

Keep the decoder idea; change English to an image



| The source changes. The decoder still reads a separate memory.

What does the visual lookup need?

PART	ROLE IN THE FAMILIAR ATTENTION OPERATION
Query: from the current language state	What information would help this position?
Keys: from the visual states	Compute a match score for each memory row.
Values: from the visual states	Supply the features that the weights will mix.

| Q decides how to read; K is compared; V supplies the returned content.

The language state asks the question

VISUAL SOURCE STATES



LANGUAGE STATE · AFTER CAUSAL SELF-ATTENTION

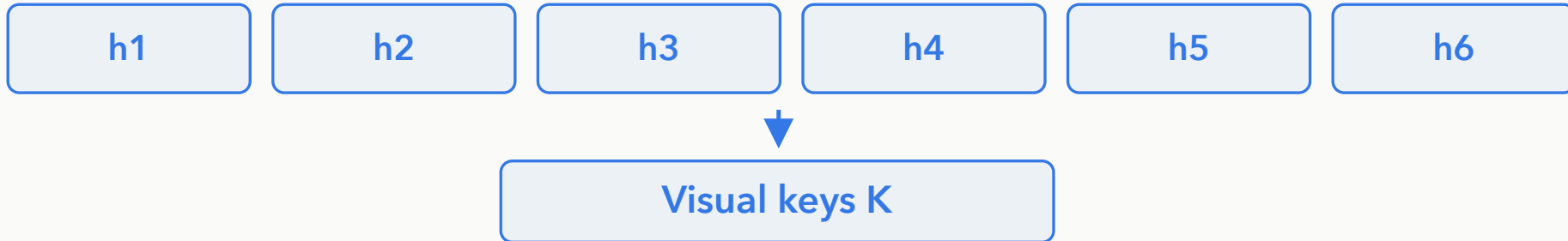


$$Q = H_L W_Q$$

| Queries come from the language state.

The image supplies the keys

VISUAL SOURCE STATES



LANGUAGE STATE · AFTER CAUSAL SELF-ATTENTION

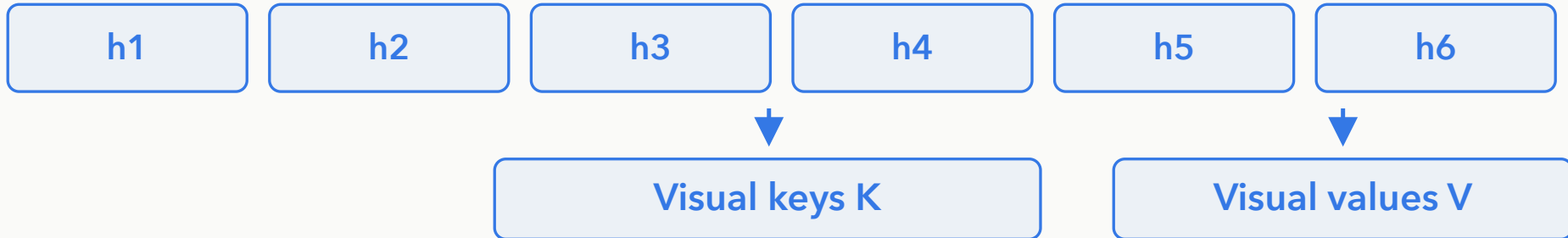


$$K = H_V W_K$$

| Keys describe what the visual memory can match.

The image also supplies the values

VISUAL SOURCE STATES



LANGUAGE STATE · AFTER CAUSAL SELF-ATTENTION

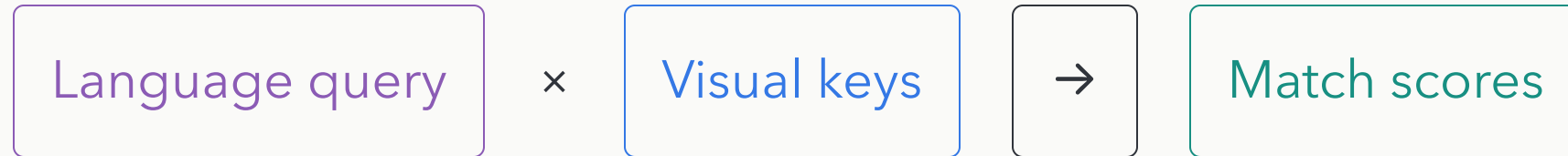


$$V = H_V W_V$$

Values provide the content that attention will mix.

Compare a query with every visual key

$$s = \frac{qK^T}{\sqrt{d_k}}$$



| The result is one score per visual state.

Turn the scores into mixing weights

$$\alpha = \text{softmax}(s)$$



Softmax normalizes across the visual memory positions.

Read a visual message

$$c = \alpha V = \sum_i \alpha_i v_i$$



| One query produces one context vector per head.

One tiny lookup: from a language query to a visual message

$$q = [1, 0], \quad d_k = 2$$

TEACHING LABEL	KEY	VALUE
face	[0.9, 0.1]	[1, 0]
fur	[0.6, 0.4]	[0, 2]
background	[0.1, 0.9]	[1, 1]

| Toy numbers and labels; no measured attention map is being shown.

Query × face key

$$q \cdot k_1 = 1(0.9) + 0(0.1) = 0.9$$

| The same language query is compared with another visual key.

Query × fur key

$$q \cdot k_2 = 1(0.6) + 0(0.4) = 0.6$$

| The same language query is compared with another visual key.

Query × background key

$$q \cdot k_3 = 1(0.1) + 0(0.9) = 0.1$$

| The same language query is compared with another visual key.

Scale, then normalize this row

$$s = [0.9, 0.6, 0.1] / \sqrt{2} \approx [0.636, 0.424, 0.071]$$

$$\alpha = \text{softmax}(s) \approx [0.421, 0.340, 0.239]$$

| The weights were calculated from Q and K.

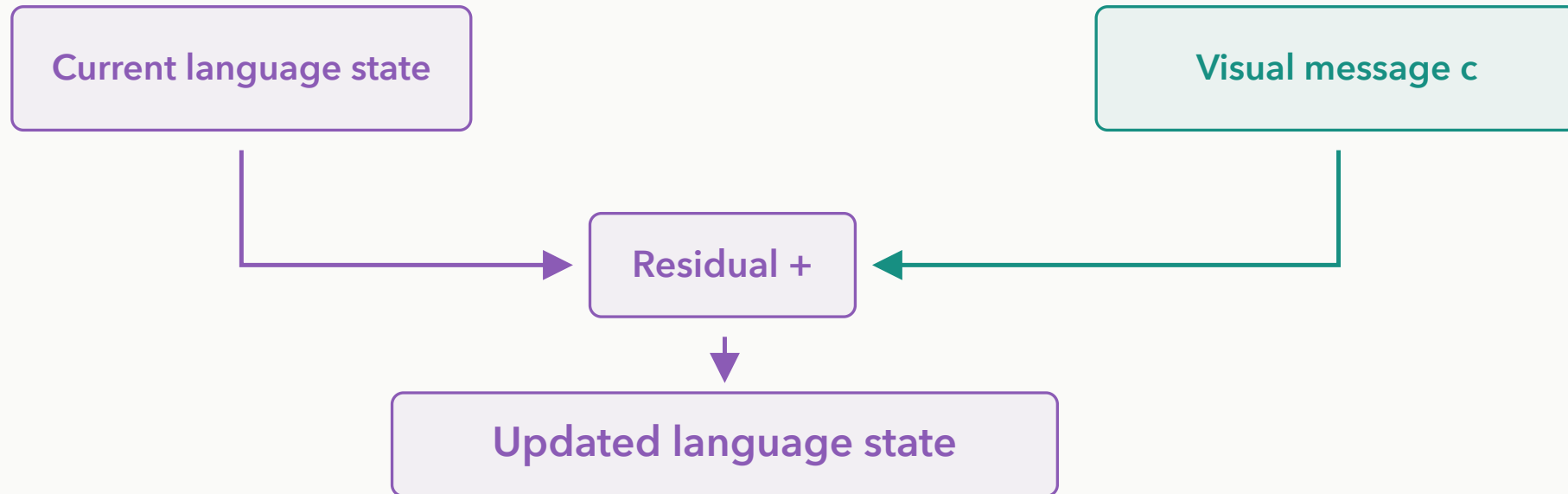
Use those weights to mix the values

$$c = 0.421[1, 0] + 0.340[0, 2] + 0.239[1, 1]$$

$$c \approx [0.660, 0.920]$$

| The result is a vector, not an output word.

The message updates the language state; it is not a word



$$H'_L = H_L + \text{CrossAttn}(H_L, H_V)$$

| The projected attention output updates the language state.

Check: has cross-attention already answered “dog”?

Lookup result

$$c \approx [0.660, 0.920]$$

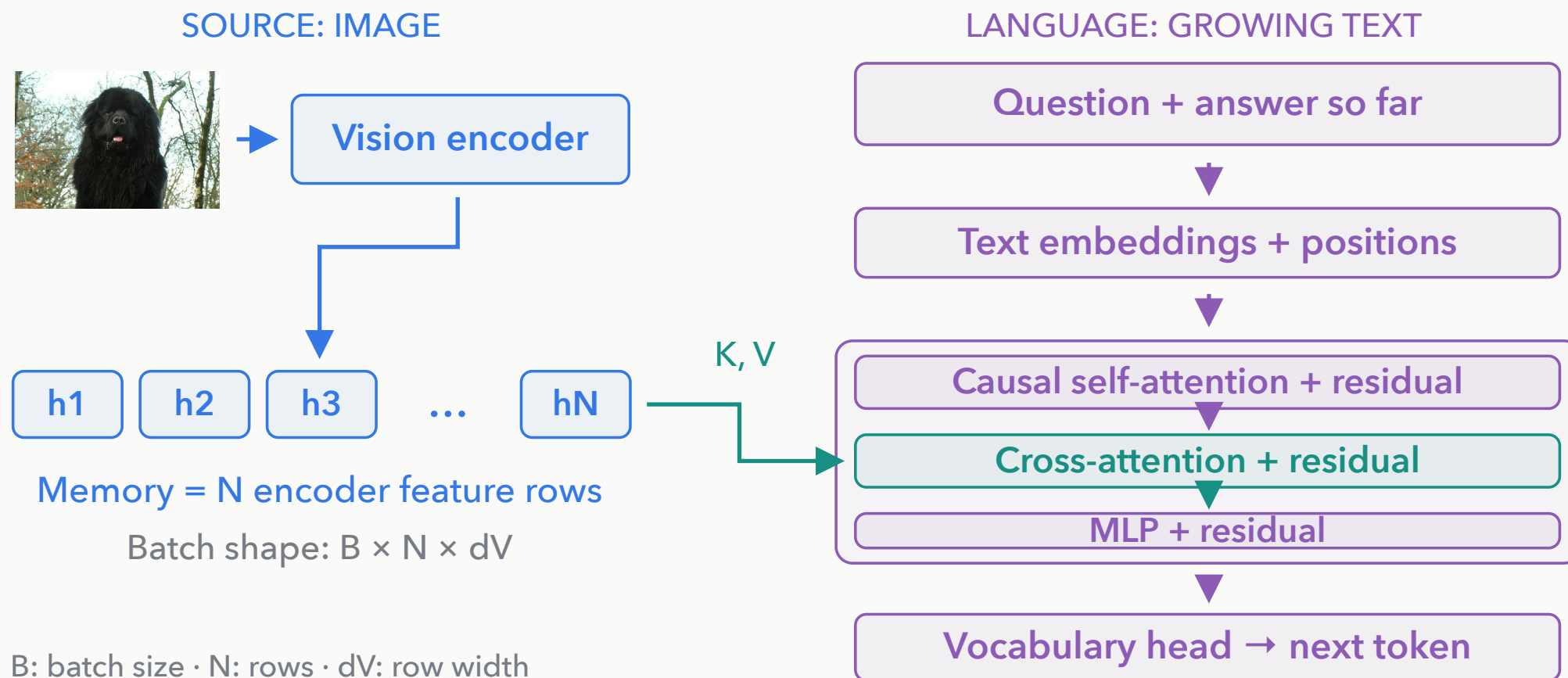
What happens next?

Does c name a word?

Or does it update the decoder’s current state?

| Predict before advancing: where does a vocabulary token actually appear?

The vocabulary head still produces the next-token scores



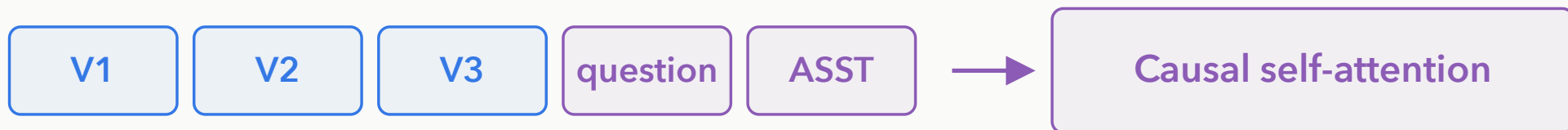
Visual lookup changes the language state; the language head chooses words.

Two ways for language to access vision

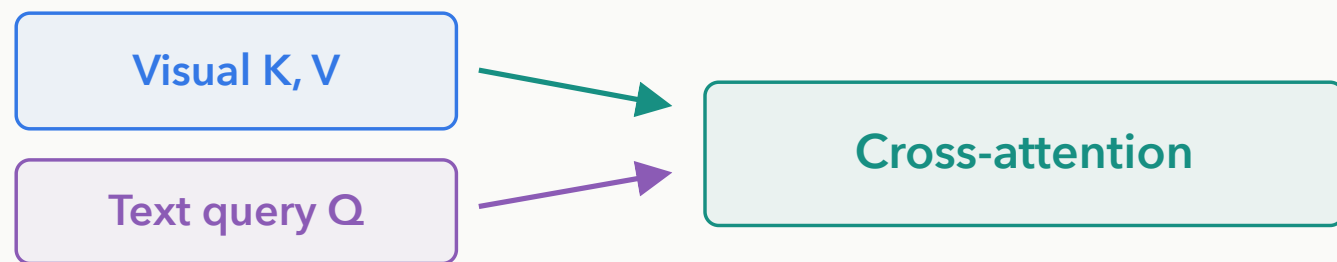


Same image. Same question: What animal is shown?

A · VISUAL PREFIX



B · SEPARATE MEMORY



In-sequence visual context or a separate visual memory.

Idea 2, end to end in one function

**image**

One image

question = "What animal is shown?"**answer_so_far** = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

<code>def next_token(image, question, answer_so_far):</code>	
<code>memory = vision_encoder(image).features</code>	$N \times d_V$
<code>ids = tokenize(question, answer_so_far)</code>	T token IDs
<code>text = embed(ids)</code>	$T \times d_{LM}$
for block in decoder_blocks:	
<code>text = block.causal_self_attn(text)</code>	$T \times d_{LM}$
<code>text = block.cross_attn(text, memory)</code>	$T \times d_{LM}$
<code>text = block.mlp(text)</code>	$T \times d_{LM}$
<code>return argmax(lm_head(text[-1]))</code>	Next text token ID

cross_attn: Q from text; K and V from image memory. The two sequences stay separate.

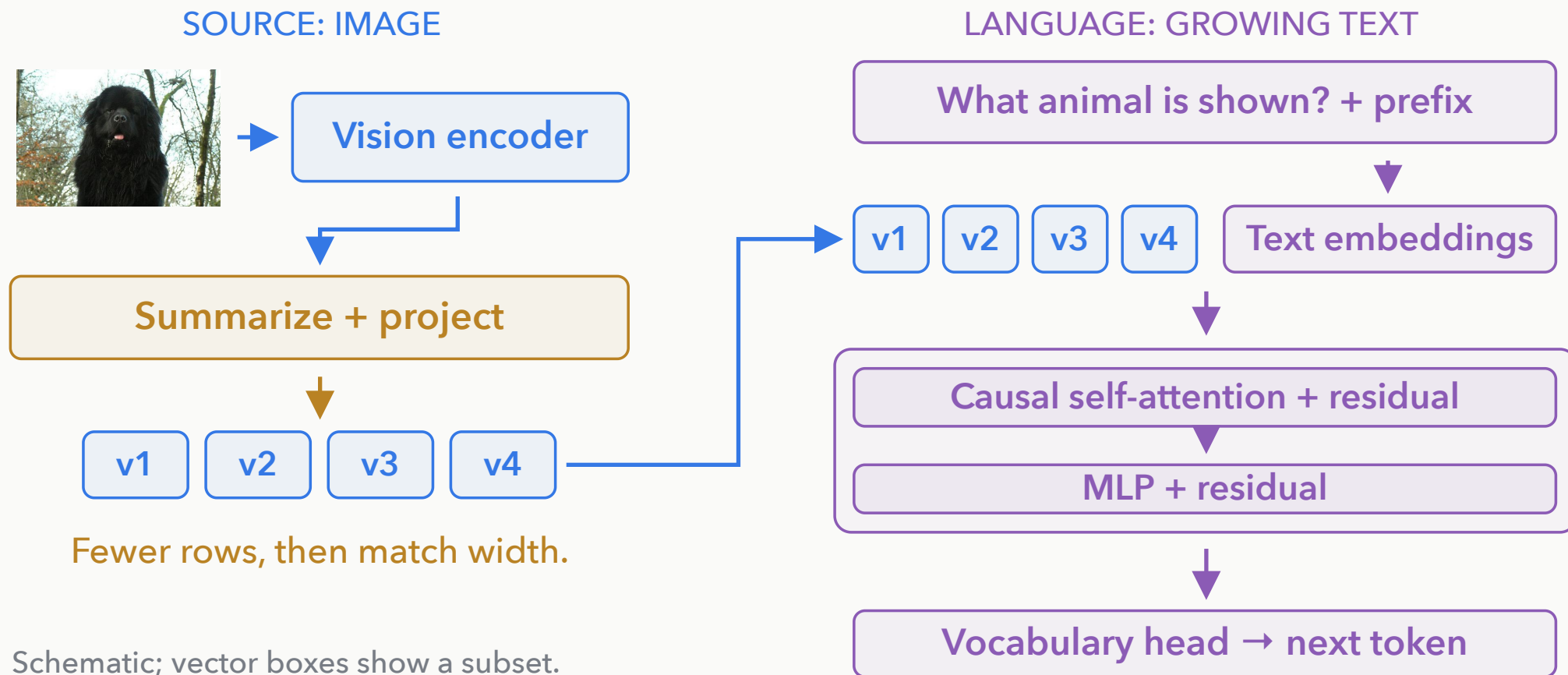
Sublayer helpers include residuals and norms. Append the returned token ID to the answer; repeat.

SECTION 05

Do we need every visual token?

Return to Idea 3: how can we make a shorter visual summary?

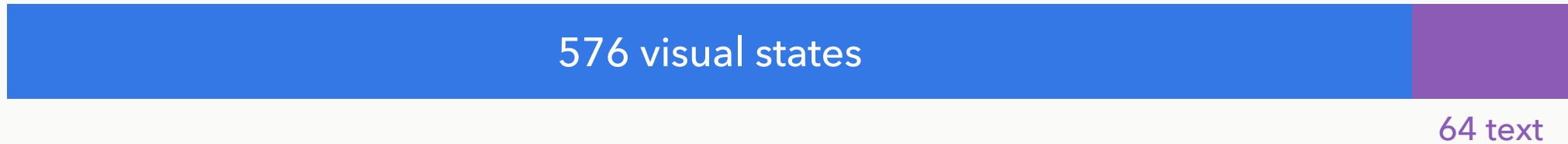
Idea 3 in detail · make the visual context shorter



Replace the row-wise projector with a learned summary followed by projection.

How long is the combined context?

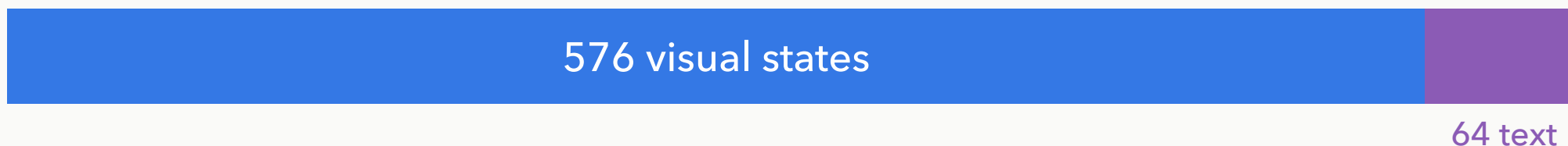
576 visual + 64 text = 640 positions



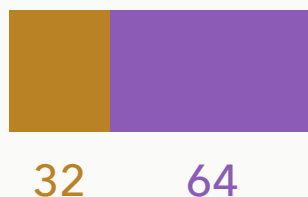
| 576 visual states + 64 text positions = 640 positions.

What if the image became 32 context vectors?

576 visual + 64 text = 640 positions



32 visual + 64 text = 96 positions



Same scale in both rows.

| 32 visual states + 64 text positions = 96 positions.

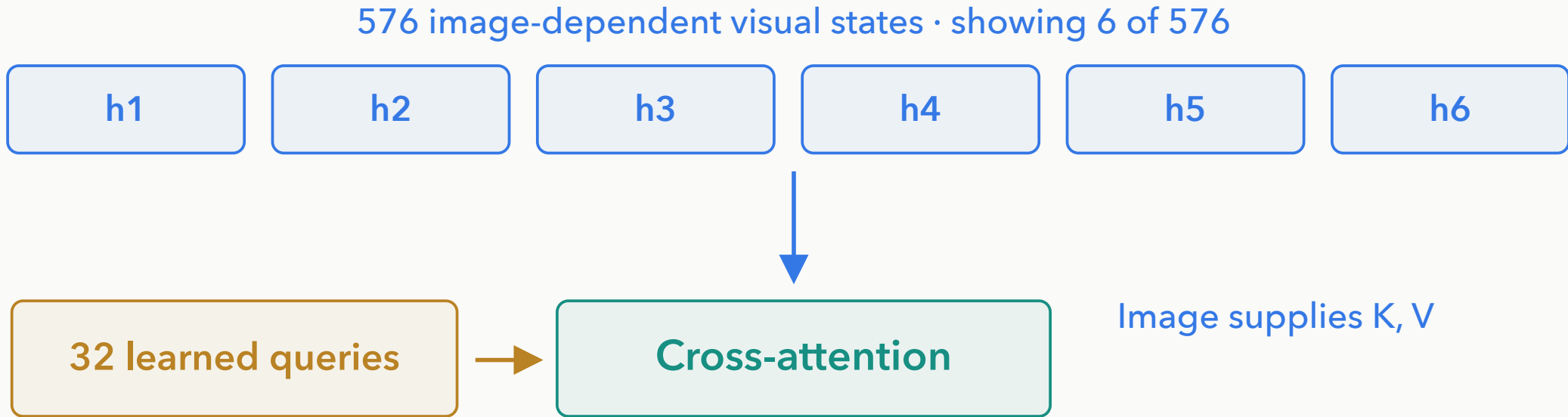
Now build the summary: start with the image states

576 image-dependent visual states · showing 6 of 576



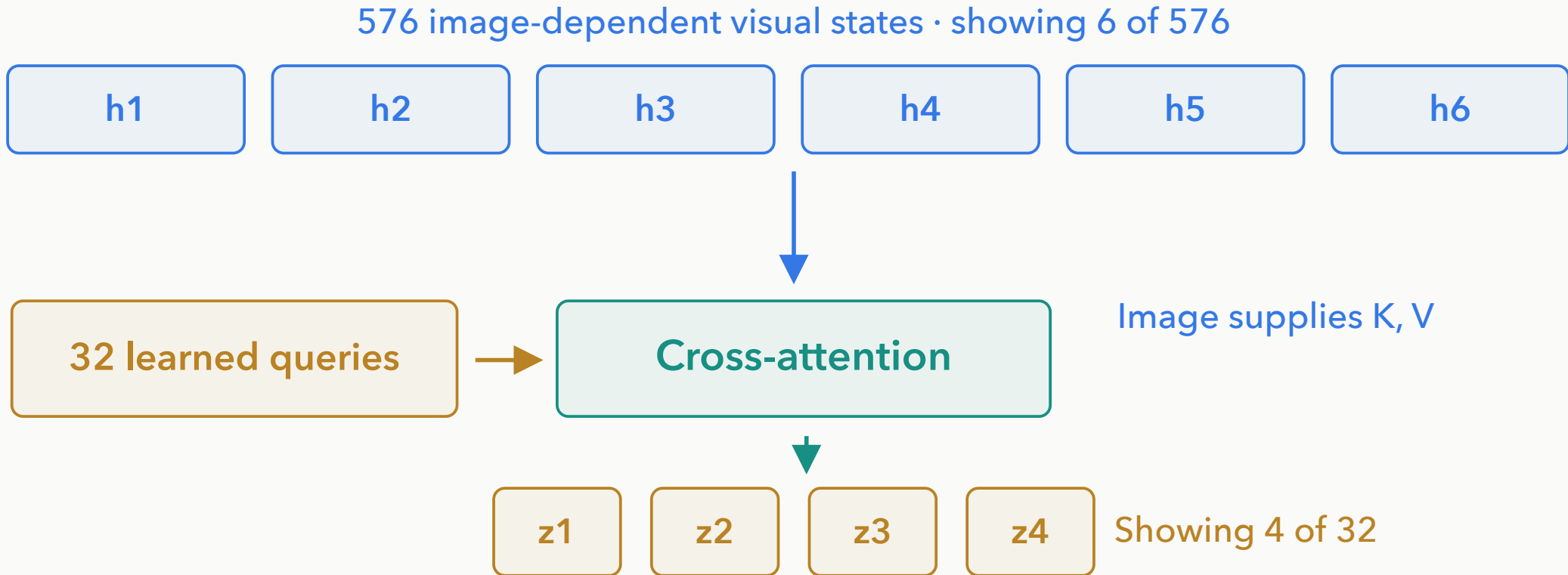
| How can we read a fixed-size summary of a variable visual memory?

Use 32 learned query vectors as 32 readers



| How can we read a fixed-size summary of a variable visual memory?

Each reader returns one image-dependent summary vector



| The query count fixes the number of output vectors.

Which detail survives the bottleneck?

CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

Subtotal ₹1,000

GST ₹180

TOTAL ₹1,180

32 outputs must carry the evidence needed by the question.

A shorter sequence can lose small text or spatial detail.

| Compression is a separate choice from prefix versus cross-attention access.

Attach names after the design choices

EXAMPLE	VISUAL STATES	CONNECTOR	ACCESS
LLaVA-style	ViT patches	Linear / MLP	Visual prefix
Flamingo	Vision feature grid	Perceiver Resampler	Gated cross-attention
BLIP-2	ViT patches	Q-Former + projection	Query-output prefix*

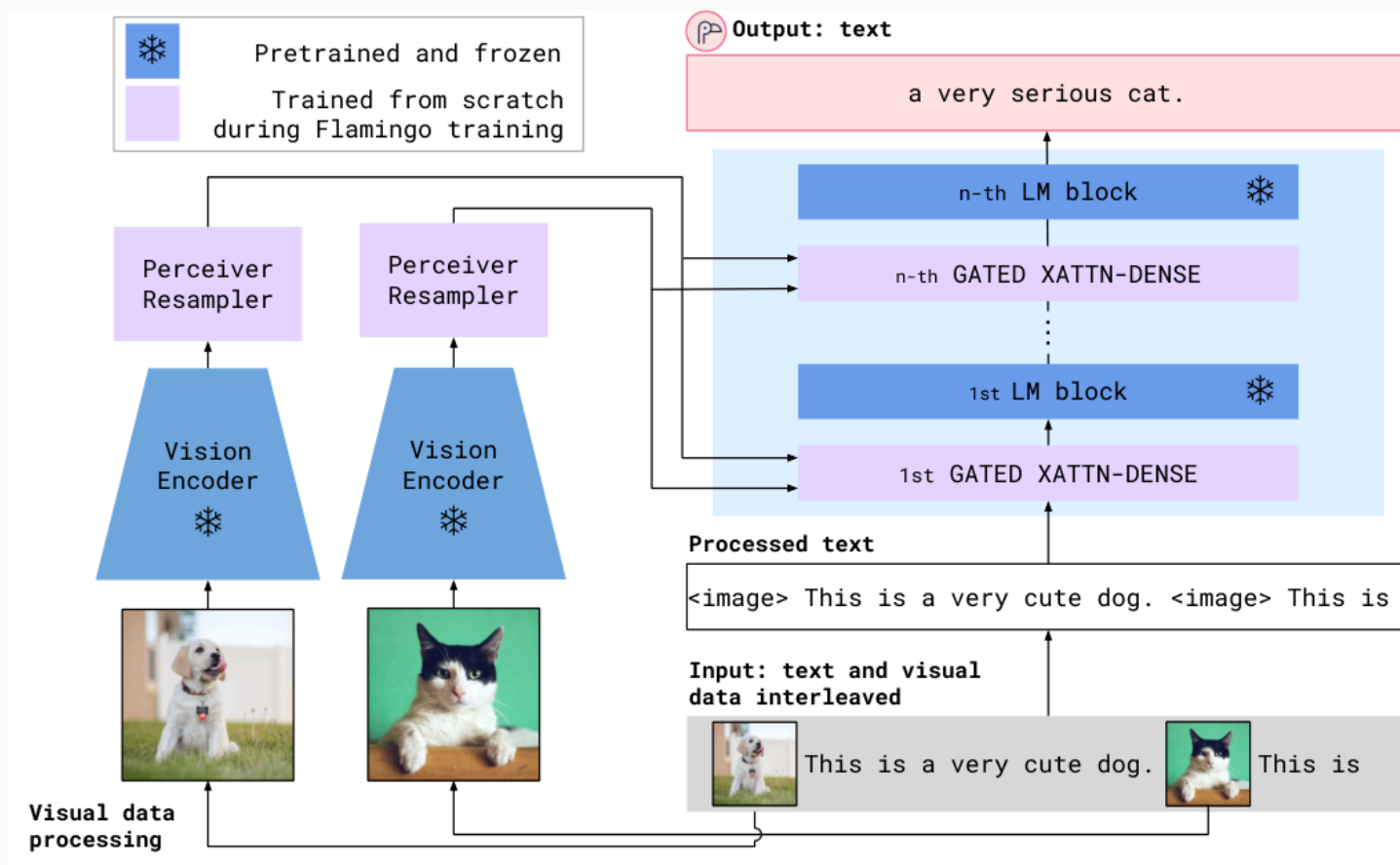
*Decoder-only LLM variant. Other language backbones use different interfaces.

Flamingo and BLIP-2 use learned-query mechanisms in different systems.

Flamingo: a Visual Language Model for Few-Shot Learning

Jean-Baptiste Alayrac · Jeff Donahue · Pauline Luc · et al.

[Read the paper ↗](#)



1 · Idea 3 · compress visual features

Frozen NFNNet features → Perceiver Resampler → fixed-size visual memory.

2 · Idea 2 · read separate memory

Gated cross-attention inserts visual information between LM blocks.

3 · Read the freeze symbols

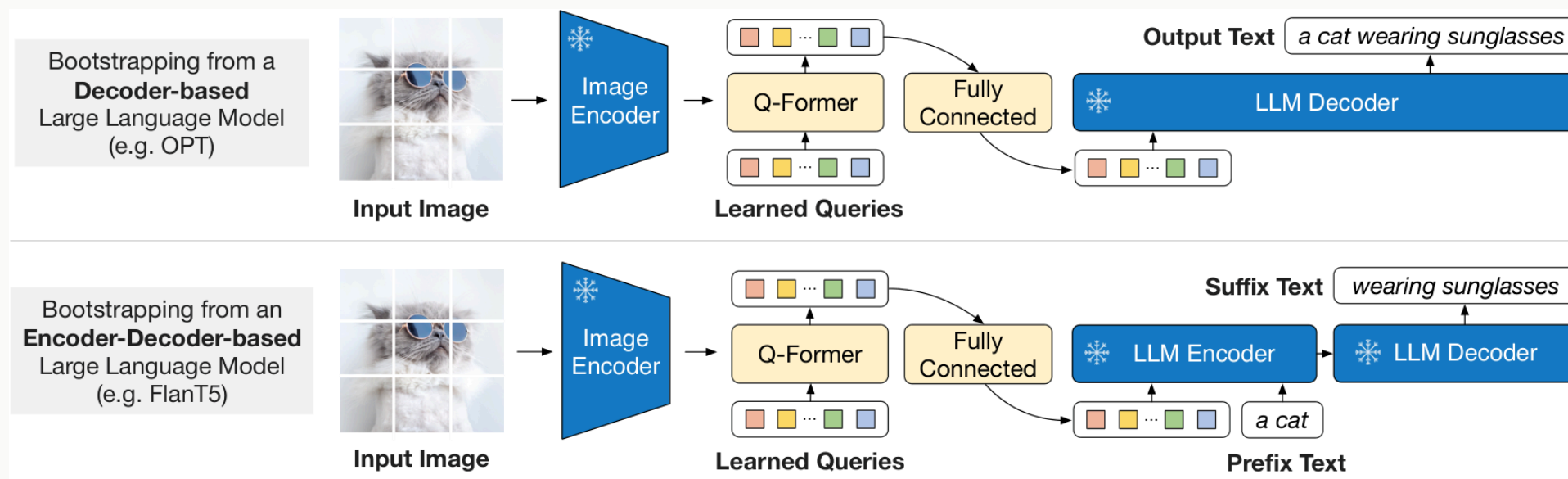
Vision encoder and LM stay frozen; the added bridge modules are trained.

Original Figure 3 · Image/video features condition a separate language stream.

BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models

Junnan Li · Dongxu Li · Silvio Savarese · Steven Hoi

[Read the paper ↗](#)



Original Figure 3 · Top: decoder-only LLM. Bottom: encoder-decoder LLM.

1 • Idea 3 • learned-query summaries

32 learned queries produce 32 image-dependent output vectors.

2 • Idea 1 • prefix for OPT (top)

Project the summaries; prepend them to text embeddings.

3 • Bottom: encoder-decoder

Visual vectors + text enter the encoder; its decoder generates the answer.

The learned readers stay; their image summaries change

LEARNED / STORED

Vision weights

Connector weights

Language weights + token embeddings

Cross-attention weights, if present

Learned queries, if present

COMPUTED FOR THIS INPUT

Image: visual states H and projected Z

Prompt: language hidden states

Positions: queries, keys and values

Attention weights + vocabulary logits

Selected tokens appended at inference

Parameters persist across examples; activations depend on the input.

Idea 3, end to end in one function



image

One image

question = "What animal is shown?"

answer_so_far = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

<code>def next_token(image, question, answer_so_far):</code>	
<code> patches = vit(image).patch_states</code>	$N \times d_V$
<code> summary = summarizer(queries, patches)</code>	$M \times d_S; M < N$
<code> visual = projector(summary)</code>	$M \times d_{LM}$
<code> ids = tokenize(question, answer_so_far)</code>	T token IDs
<code> text = embed(ids)</code>	$T \times d_{LM}$
<code> inputs = concat([visual, text], axis="tokens")</code>	$(M + T) \times d_{LM}$
<code> hidden = causal_decoder(inputs)</code>	$(M + T) \times d_{LM}$
<code> return argmax(lm_head(hidden[-1]))</code>	Next text token ID

summarizer: M learned queries read all N patch rows through cross-attention; outputs depend on the image.

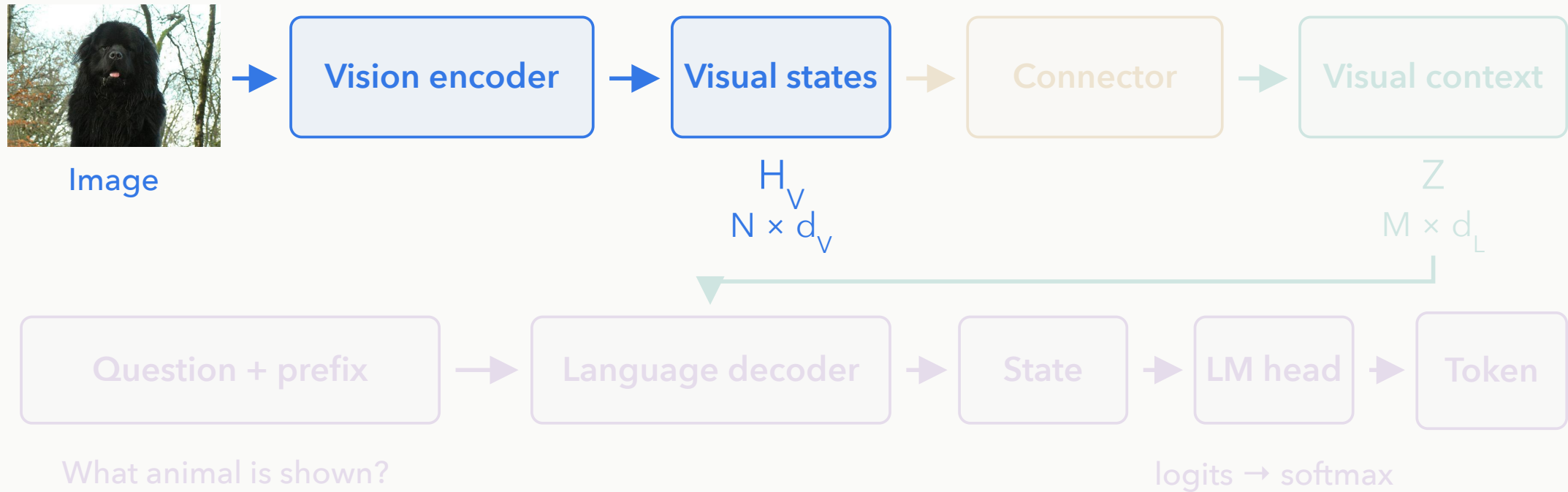
Shown: BLIP-2 / OPT-style prefix. For a Flamingo-style connection, use the summaries as Idea 2's memory.

SECTION 06

Follow one answer token end to end

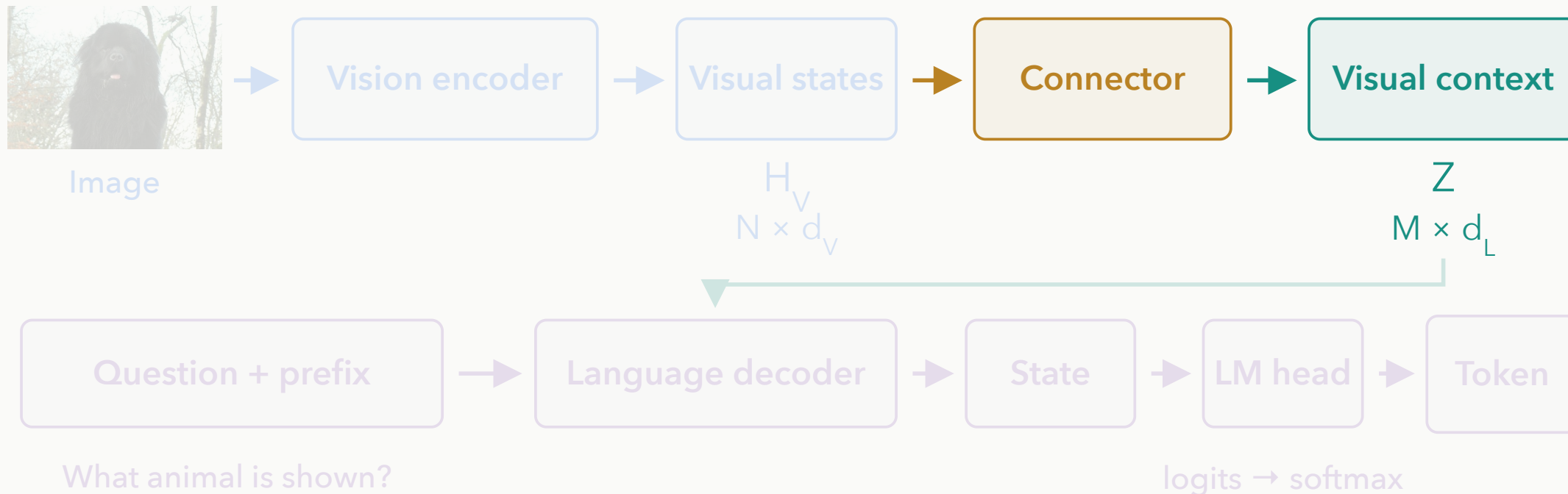
Now follow the connection we built all the way to a word.

The same Newfoundland image enters the encoder



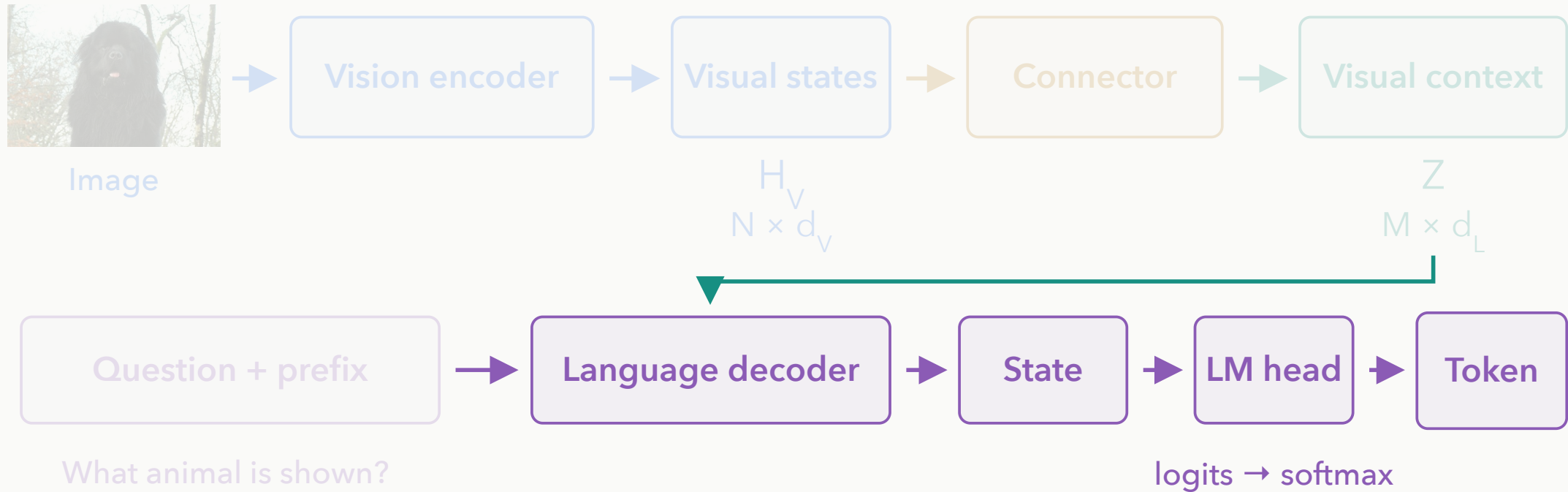
The encoder computes the visual state matrix.

The connector exposes usable visual context



| The visual context is available to the language computation.

Combine image, question and answer prefix



| The current language state will drive the vocabulary head.

The LM head scores vocabulary entries

$$\ell_t = W_{\text{vocab}} h_t + b$$

VOCABULARY ENTRY	LOGIT
dog	ℓ_{dog}
cat	ℓ_{cat}
horse / ...	$\ell_{\text{horse}} / \dots$

| Visual evidence can change h_t and therefore every vocabulary score.

From vocabulary scores to a selected token

$$p(y_t \mid y_{<t}, I, P) = \text{softmax}(\ell_t)$$

Vocabulary distribution

→

select "dog"

| Selection can be greedy or sampled; the output is a vocabulary token.

Start with image + prompt



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

⟨empty⟩

$$p(y_t \mid y_{<t}, I, P)$$

| Illustrative trace: the image and prompt remain fixed.

Append the first token



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

A

$$p(y_t \mid y_{<t}, I, P)$$

| Illustrative trace: the image and prompt remain fixed.

Use the growing prefix



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

A

black

$$p(y_t \mid y_{<t}, I, P)$$

Illustrative trace: the image and prompt remain fixed.

Append the object word



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

A

black

dog

$$p(y_t \mid y_{<t}, I, P)$$

Illustrative trace: the image and prompt remain fixed.

Finish the sentence



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

A

black

dog

.

$$p(y_t \mid y_{<t}, I, P)$$

Illustrative trace: the image and prompt remain fixed.

Stop at the end token



FIXED PROMPT

Describe the animal.

GENERATED PREFIX

A

black

dog

.

EOS

$$p(y_t \mid y_{<t}, I, P)$$

Illustrative trace: the image and prompt remain fixed.

Keep the prompt fixed. Change only the image.

What animal is shown?



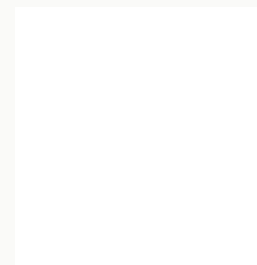
Dog

| There is a black dog in the foreground.



Cat

| A cat is visible.



Blank

| A squirrel.

$$p(y_t \mid y_{<t}, I, P)$$

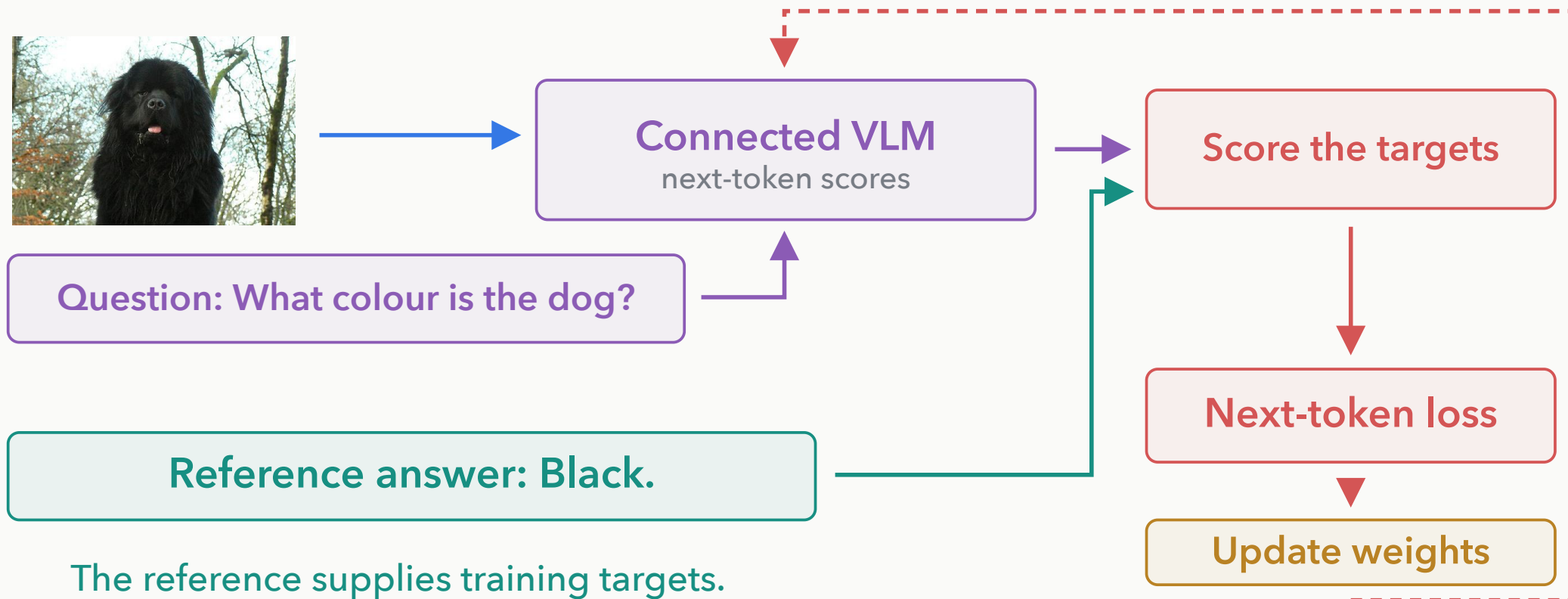
| Actual saved answers from the pinned browser model; probabilities are not exposed.

SECTION 07

How do we train the connection?

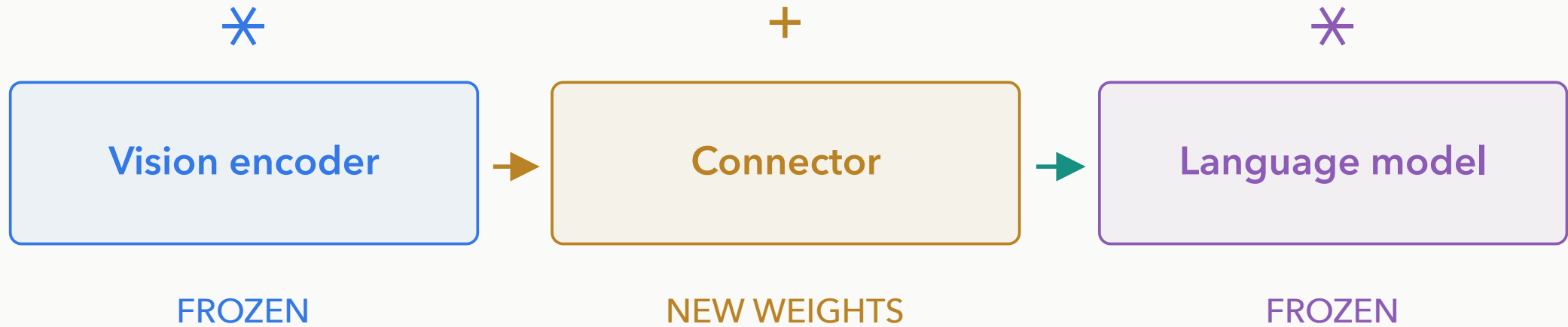
Make the visual connection useful by training on reference answers.

The wiring fits. How does it learn to use the image?



Use the correct answer tokens to train the visual-to-language connection.

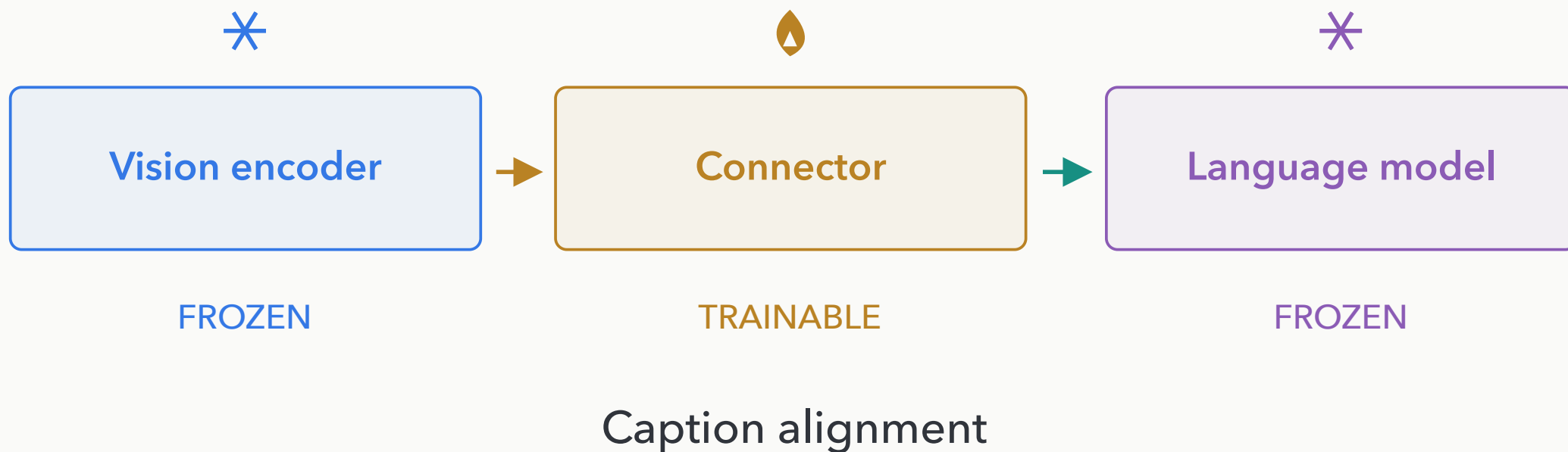
Start from two pretrained components



New connector weights; pretrained vision and language.

| The connector is new; vision and language already have learned weights.

Stage 1 · align visual features with language



Example recipe: caption alignment; freezing choices depend on the model.

Supply the correct previous caption tokens



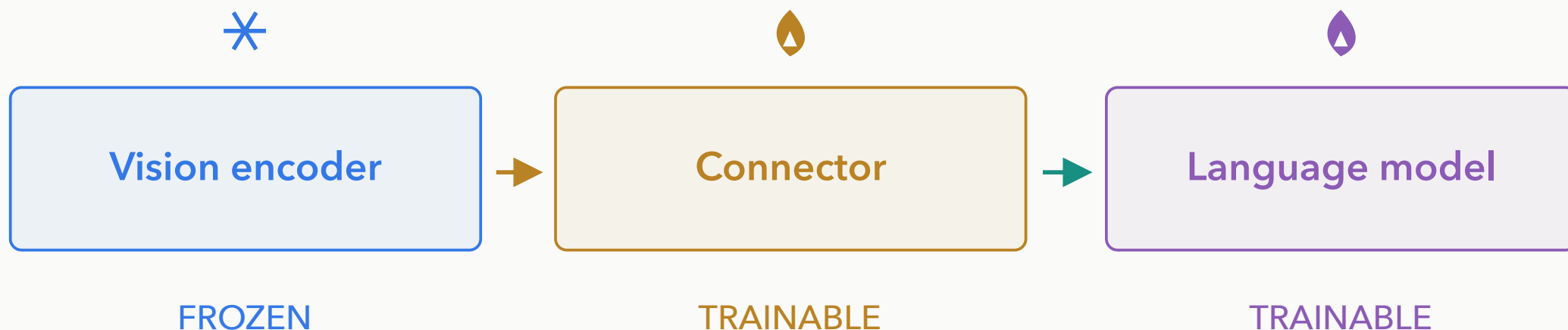
REFERENCE CAPTION

A black dog . EOS

$$\mathcal{L} = - \sum_t \log p(y_t^* \mid y_{<t}^*, I)$$

| Teacher forcing supplies the reference prefix during training.

Stage 2 · learn to follow visual instructions



Visual instruction tuning

Example recipe: image + question + answer; train the LLM or its adapters.

The same image can support different questions



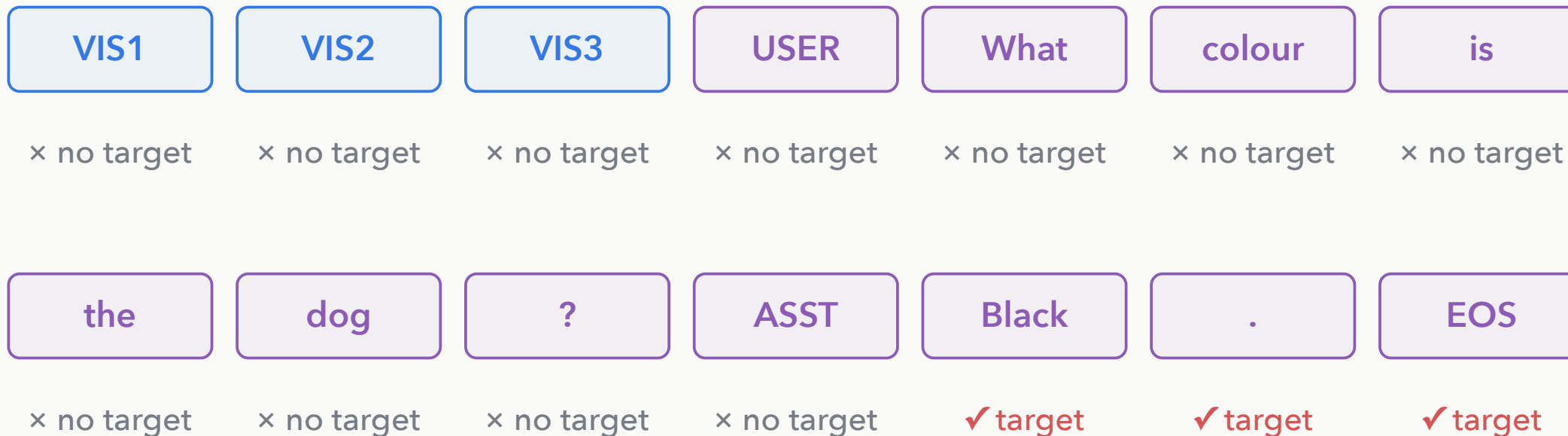
USER: What colour is the dog?

ASSISTANT: Black.

$$\mathcal{L} = - \sum_{t \in \mathcal{A}} \log p(y_t^* \mid y_{<t}^*, I, P)$$

| The loss supervises the answer targets selected by the training recipe.

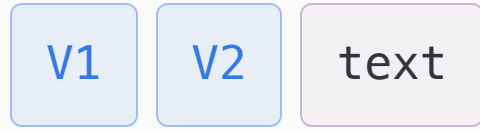
Mark the target at every position



Only Black, period and EOS are targets; preceding-position logits predict them.

Two masks answer different questions

Attention mask



↑ available to the current query

Which earlier states may this query read?

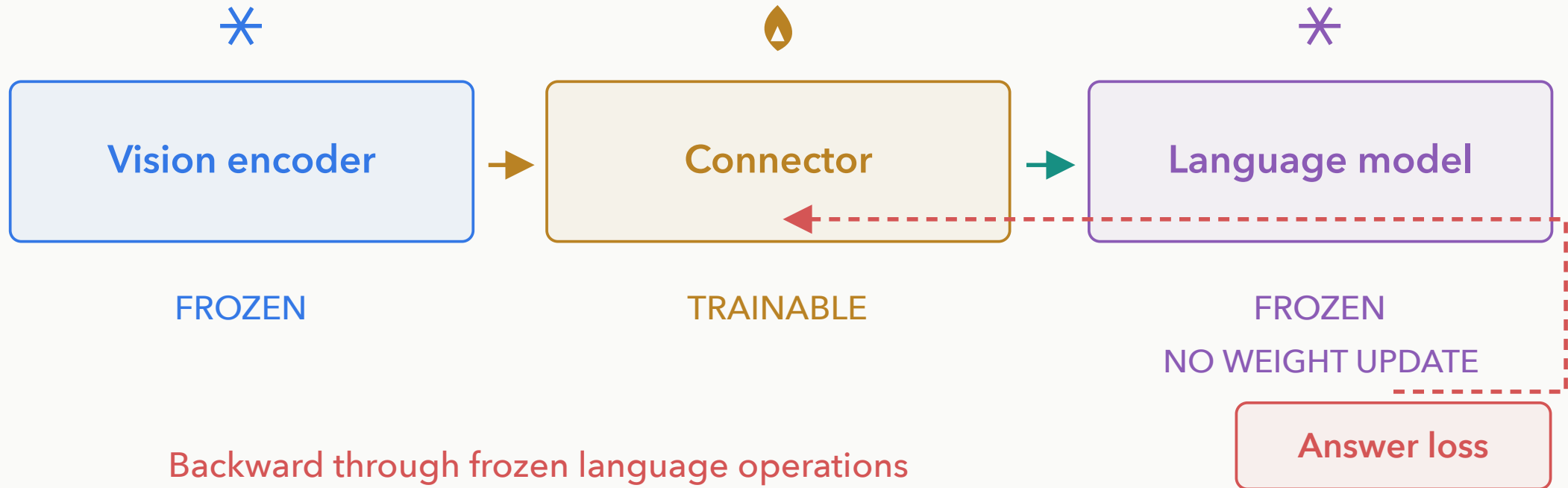
Removing a target loss does not hide a context token.

Loss mask



Which next-token targets contribute to the loss?

A frozen LLM still passes the gradient



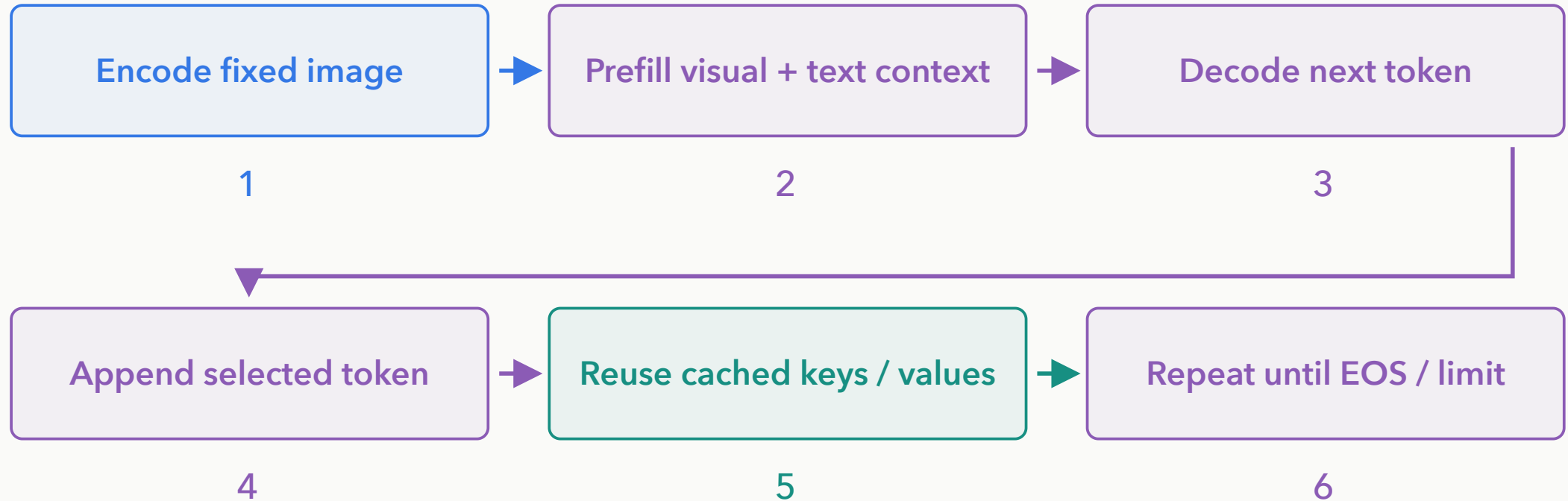
| Frozen ≠ stop-gradient.

SECTION 08

What happens at inference?

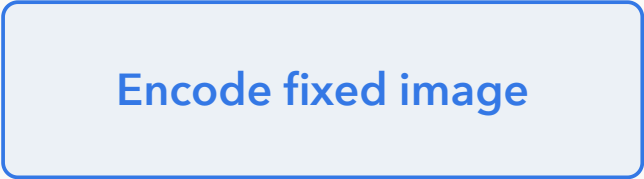
At inference: prepare the image once, then grow the answer.

The whole generation loop before its individual steps



Prepare the fixed context once; repeatedly predict and append a token.

Encode the fixed image once

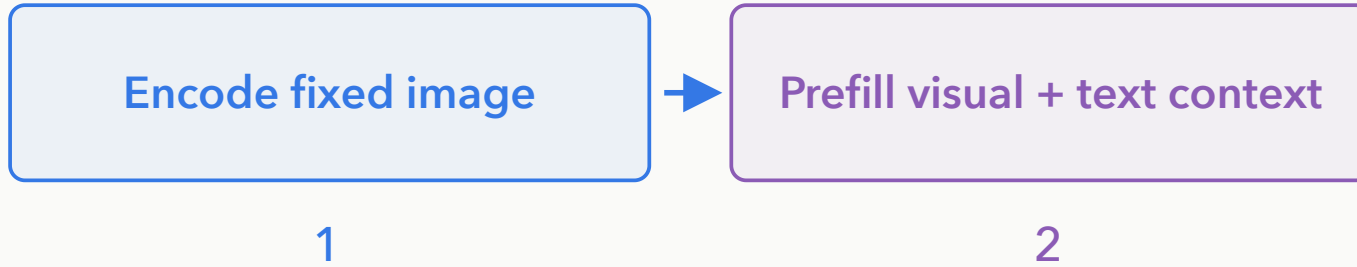


Encode fixed image

1

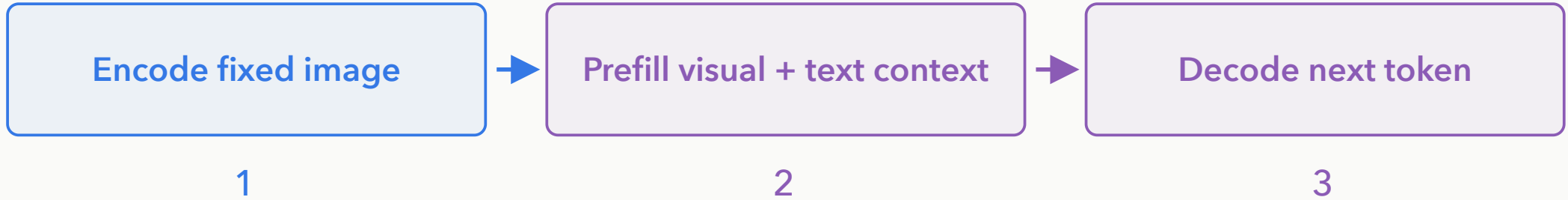
| Only the generated prefix changes at each decoding step.

Prefill the image and question context



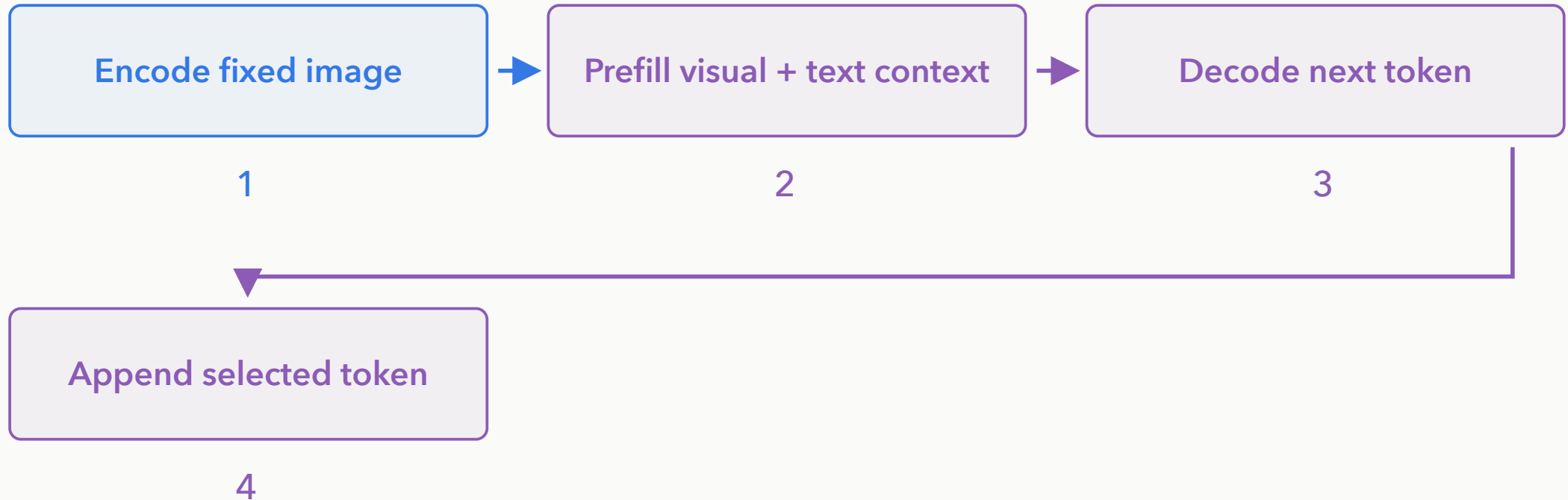
| Only the generated prefix changes at each decoding step.

Decode one new token



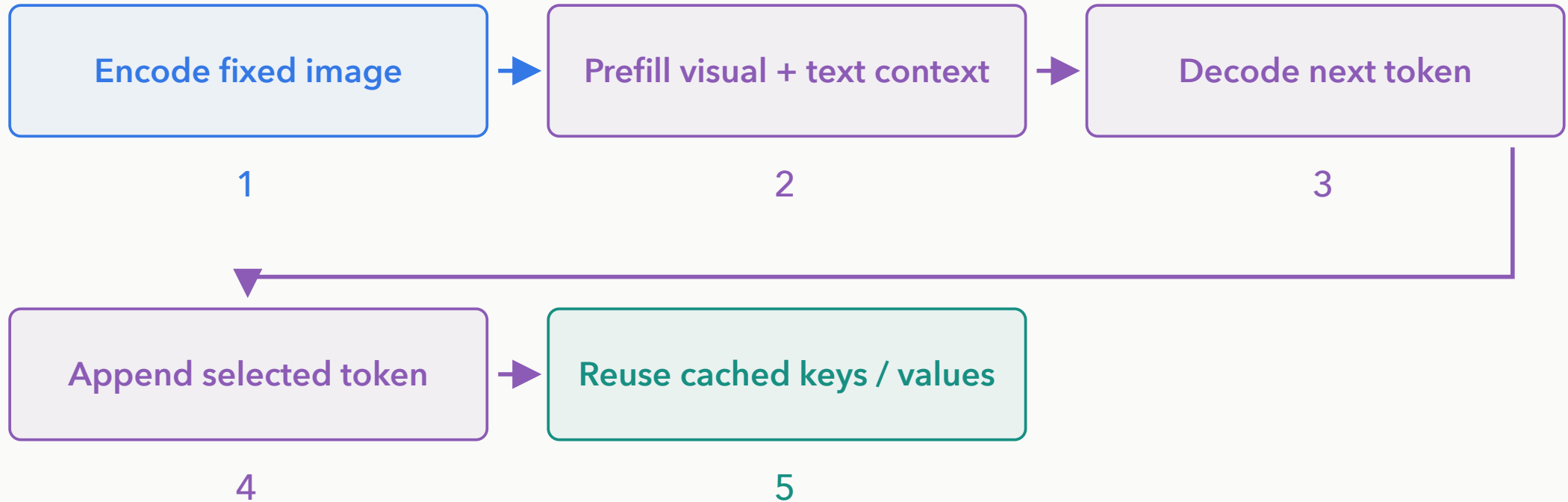
| Only the generated prefix changes at each decoding step.

Append the selected token



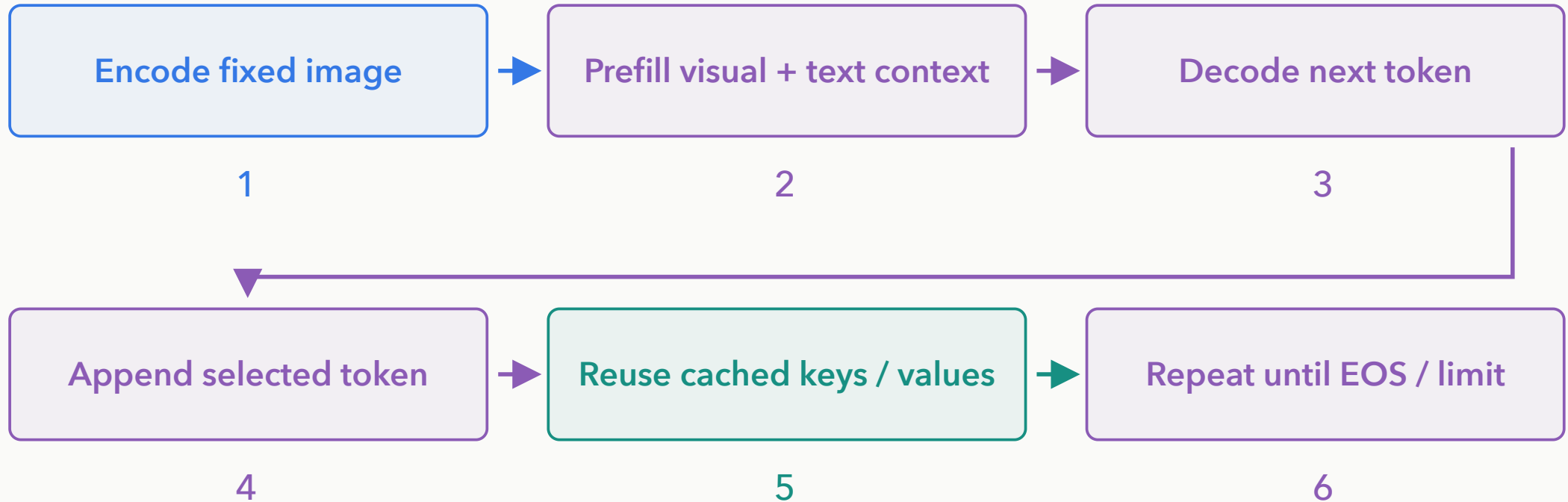
| Only the generated prefix changes at each decoding step.

Reuse the cached keys and values



| Only the generated prefix changes at each decoding step.

Repeat until EOS or the generation limit



| Only the generated prefix changes at each decoding step.

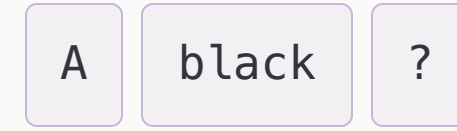
Same conditional model, different available prefix

Training



Reference prefix supplied.
Target loss and gradients.

Inference



Generated prefix grows.
No reference answer or backward pass.

| Known targets allow parallel teacher-forced positions; generation is sequential.

Reuse the image; keep the conversation context



User: What animal is this?

Assistant: A dog.

User: What colour is its fur?

Assistant: Black.

Illustrative conversation, not a captured model transcript.

| Visual features can persist. Decoder cache reuse also depends on the exact prefix.

SECTION 09

Can we trust the answer?

Check evidence use, not just fluent language.

A fluent sentence can have no visual support



What color is the bicycle?

Reference: No bicycle is visible.

| Recorded baseline: "The bicycle is red."

| Fluent ≠ grounded.

Keep a small diagnostic suite

IMAGE	FIXED QUESTION	REFERENCE
Dog	What animal is this?	Dog
Desk	Is the laptop plugged in?	Yes, visible cable connected
Shapes	What is immediately left of the mug?	Red ball
Invoice	What is the invoice number?	ES667-042
Chart	Which month has the highest value?	March
Coffee	What color is the bicycle?	No bicycle visible

Run / record the diagnostic suite ↗

[Offline fallback: recorded model outputs](#)

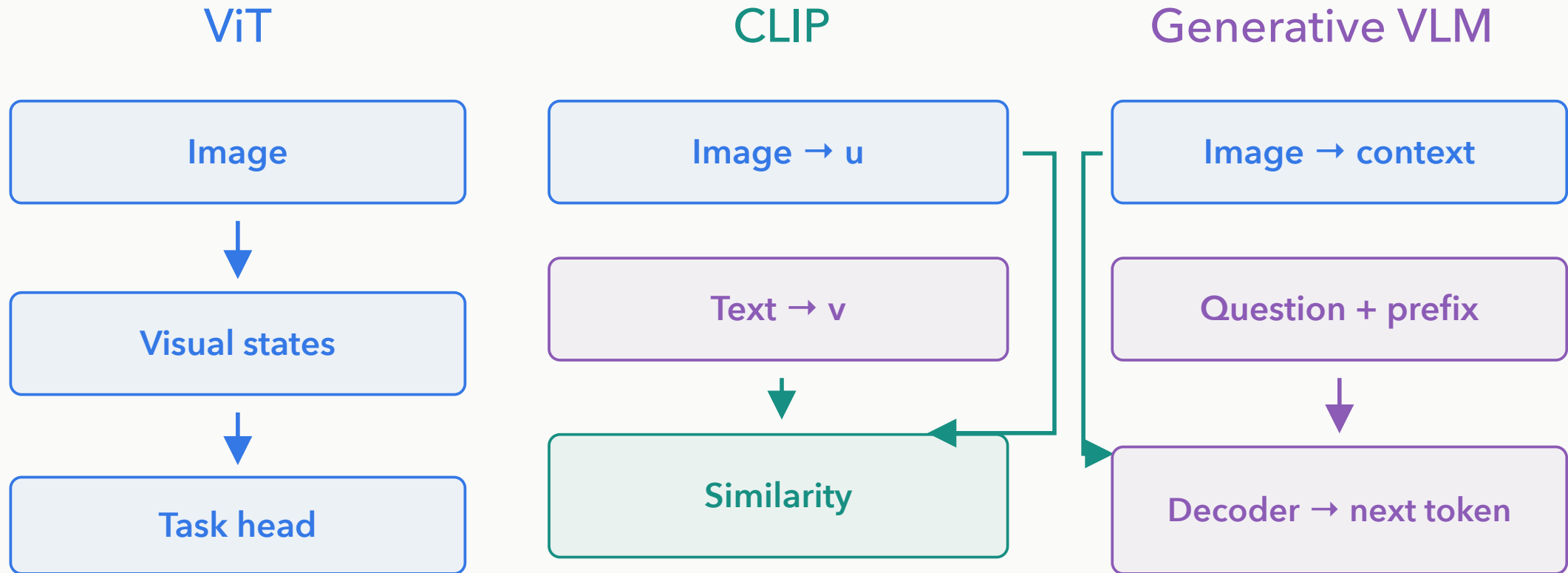
Preserve the raw answer, reveal the reference, then judge support.

SECTION 10

Summary

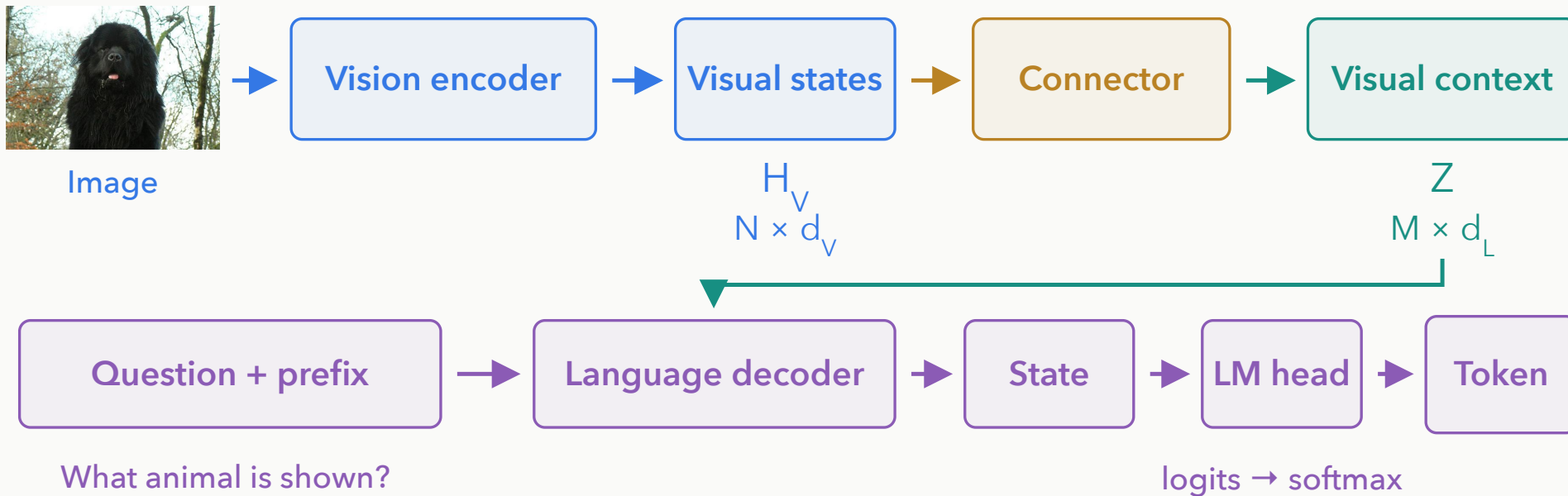
Represent the image, expose context, predict the next token.

ViT → CLIP → generative VLM



Visual representation → cross-modal matching → conditioned generation.

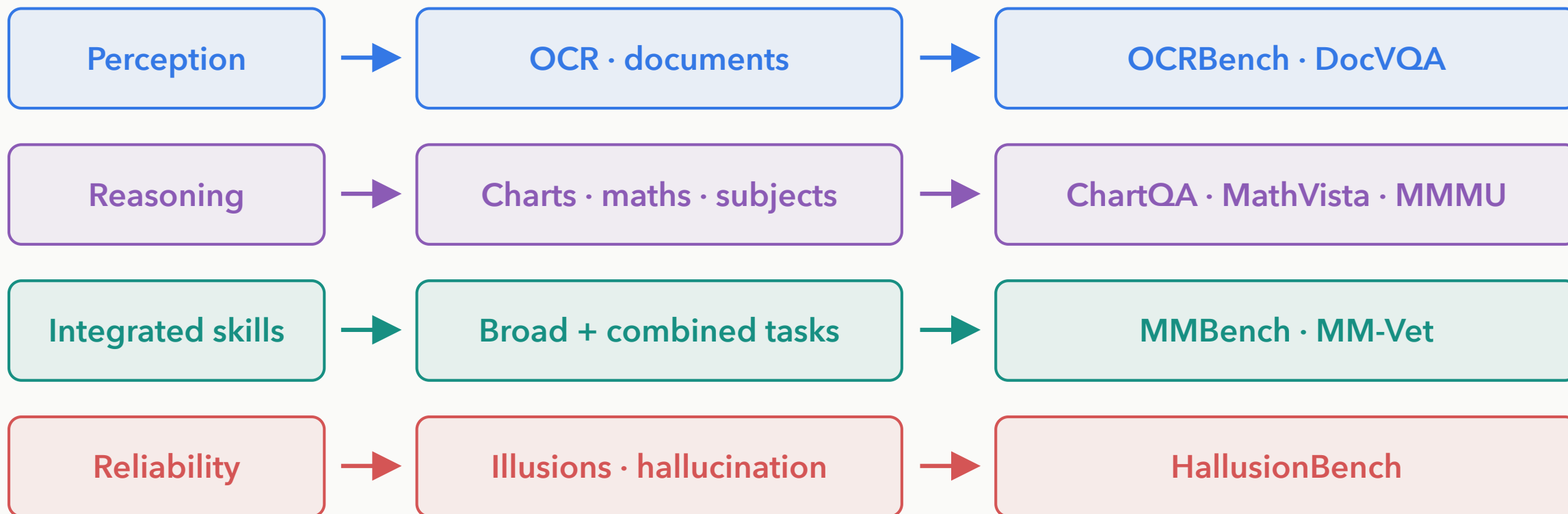
One computation to remember



$$p(y_t \mid y_{<t}, I, P)$$

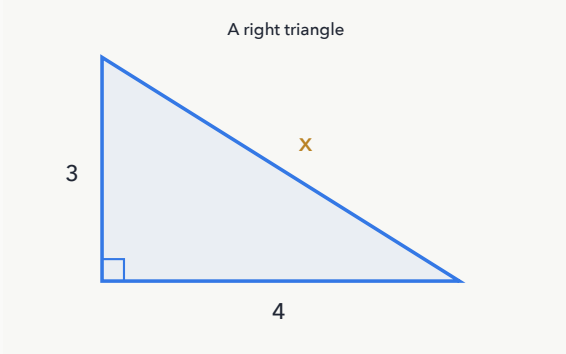
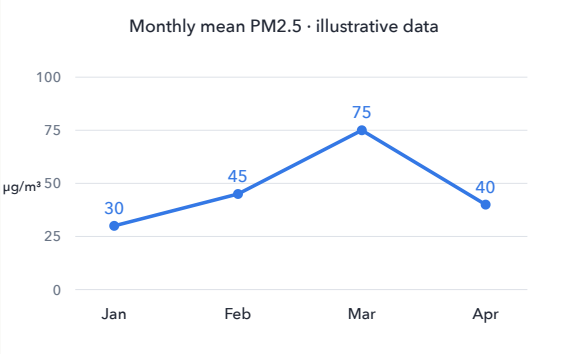
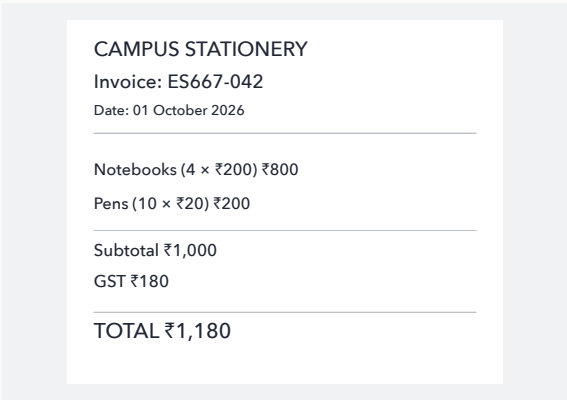
Visual information becomes context for next-token prediction.

Different benchmarks probe different capabilities



| Choose the evaluation from the capability you need.

Inspect the questions before interpreting a score



Read: What is the invoice ID?

Calculate: March minus January?

Reason: What is x?

Reject: What colour is the bicycle?

Authored classroom examples, not copied benchmark items.

A wrong answer may come from perception, reasoning, or accepting a false premise.

Use a metric that matches the answer format

OUTPUT / USE	MEASURE	WHAT STILL NEEDS INSPECTION
Fixed answer	Accuracy	Ambiguity and answer normalization
Document text	Exact / normalized string match; ANLS	OCR errors and layout dependence
Open explanation	Human or specified model rubric	Judge bias and unsupported details
Grounding coordinates	IoU / pointing accuracy	Object identity and spatial accuracy
Deployment	Latency, memory, output tokens/s	Hardware and decoding settings

Report the evaluation protocol along with the number.

Further training can change how the model answers

Behaviour

Instruction following
Preference optimisation
Safer responses

Evidence use

Better grounding data
Hard negative examples
Missing-evidence responses

| The basic inference target remains image-conditioned next-token prediction.

Sources • architectures and browser implementation

[CLIP • Radford et al., 2021](#)

<https://arxiv.org/abs/2103.00020>

[LLaVA • Liu et al., 2023](#)

<https://arxiv.org/abs/2304.08485>

[Flamingo • Alayrac et al., 2022](#)

<https://arxiv.org/abs/2204.14198>

[BLIP-2 • Li et al., 2023](#)

<https://arxiv.org/abs/2301.12597>

[SmolVLM-256M • model card](#)

<https://huggingface.co/HuggingFaceTB/SmolVLM-256M-Instruct>

[Official Transformers.js browser example](#)

<https://github.com/huggingface/transformers.js-examples/tree/main/smolvlm-webgpu>

[Oxford-IIIT Pet • CC BY-SA 4.0](#)

<https://www.robots.ox.ac.uk/~vgg/data/pets/>

Primary sources; no leaderboard scores are used in this lecture.

Sources • evaluation

[OCRBench](#)

<https://arxiv.org/abs/2305.07895>

[DocVQA](#)

<https://arxiv.org/abs/2007.00398>

[ChartQA](#)

<https://arxiv.org/abs/2203.10244>

[MathVista](#)

<https://arxiv.org/abs/2310.02255>

[MMMU](#)

<https://arxiv.org/abs/2311.16502>

[MMBench](#)

<https://arxiv.org/abs/2307.06281>

[MM-Vet](#)

<https://arxiv.org/abs/2308.02490>

[HallusionBench](#)

<https://arxiv.org/abs/2310.14566>

| The local six-image suite is an instructional diagnostic, not one of these benchmarks.